Java Core Exercises

Exercises and solutions

Nicola Bicocchi

Contents

E2 - Java Basics 1

E3 - Java Object-Oriented Programming 18

E4 - Java Collections 50

E5 - Java Exceptions 67

E6 - Java Generics 85

E7 - Java Functional Programming 100

E8 - Java Concurrency 116

E2 - Java Basics

Esercizio 1 — Aggiornamento delle misure

Tempo indicativo: 5–7 minuti · Java 25

Un'applicazione raccoglie misure numeriche in un array condiviso da più parti del programma. Una schermata conserva un riferimento all'array per mostrarne il contenuto aggiornato. `increase` deve aumentare di uno tutti gli elementi, mentre `reset` deve azzerarli senza cambiare l'oggetto array osservato dalla schermata. Prima di un incremento, le misure sono interi tra 0 e 99; l'array è non nullo, anche eventualmente vuoto. Il programma è monothread.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato, con `java.util.Arrays` importata. I numeri di riga non fanno parte del codice.

```
01  class Measures {
02      static void increase(int[] values) {
03          for (int i = 0; i < values.length; i++) {
04              values[i]++;
05          }
06      }
07      static void reset(int[] values) {
08          values = new int[values.length];
09      }
10  }
```

```
11  int[] original = {2, 5};
12  int[] alias = original;
13  Measures.increase(original);
14  IO.println(Arrays.toString(alias));
15  IO.println(original == alias);
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le due righe nell'ordine.

- **A.** `[2, 5]`, poi `false`.
- **B.** `[3, 6]`, poi `false`.
- **C.** `[3, 6]`, poi `true`.
- **D.** `[2, 5]`, poi `true`.

2. Risposta breve

Considera separatamente `int[] data = {4, 7}; Measures.reset(data);`. Il metodo rispetta il proprio contratto? Motiva facendo riferimento al passaggio dei parametri e, se necessario, correggine il corpo mantenendo la firma.

Lunghezza indicativa: 1–2 frasi e massimo 3 righe di codice.

Mostra la soluzione dell'esercizio 1

1. Scelta multipla: C

`original` e `alias` indicano lo stesso array. Il metodo modifica i suoi elementi, quindi si stampano `[3, 6]` e `true`.

- **A errata:** gli elementi cambiano e i riferimenti restano uguali.
- **B errata:** l'incremento non crea un array diverso.
- **D errata:** gli elementi non restano invariati.

2. Risposta breve

Il contratto non è rispettato: viene riassegnata solo la copia locale del riferimento e `data` conserva `[4, 7]`. Occorre modificare gli elementi dell'array ricevuto.

```
Arrays.fill(values, 0);
```

Alternativa valida: un ciclo che assegna zero a ogni elemento. Entrambe le soluzioni funzionano anche con un array vuoto.

Esercizio 2 — Righe di una tabella

Tempo indicativo: 5–7 minuti · Java 25

Un servizio registra le quantità raccolte da più postazioni in una tabella. Ogni riga rappresenta una postazione, che può aver prodotto un numero diverso di misure. Il metodo `sum` deve sommare tutti gli elementi senza modificare la tabella, restituendo zero se non ci sono valori. Un'altra operazione crea una copia con `clone()` e ne modifica alcune parti per preparare una simulazione. La tabella e le righe sono non nulle, anche vuote; i valori sono piccoli interi e non si considera overflow.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```java
class Tables {
    static int sum(int[][] values) {
        int total = 0;
        for (int r = 0; r < values.length; r++) {
            for (int c = 0; c < values[0].length; c++) {
                total += values[r][c];
            }
        }
        return total;
    }
}
```

```java
int[][] original = {{1, 2}, {3}};
int[][] copy = original.clone();
copy[0][0] = 9;
copy[1] = new int[]{7};
IO.println(original[0][0]);
IO.println(original[1][0]);
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le due righe nell'ordine.

- **A.** 1, poi 3.
- **B.** 9, poi 3.
- **C.** 9, poi 7.
- **D.** 1, poi 7.

2. Risposta breve

Valuta separatamente `Tables.sum(new int[][]{{1}, {2, 3}})` rispetto al contratto. Indica il risultato richiesto e quello effettivo; se necessario, modifica una sola riga del metodo perché funzioni con tutte le forme ammesse.

Lunghezza indicativa: 1–2 frasi e una riga di codice.

Mostra la soluzione dell'esercizio 2

1. Scelta multipla: B

`clone()` copia l'array esterno, ma condivide inizialmente le righe. La modifica di `copy[0][0]` raggiunge la prima riga originale; riassegnare `copy[1]` sostituisce invece soltanto una posizione nell'array esterno copiato. Si stampano 9 e 3.

- **A errata:** la prima riga non è stata copiata in profondità.
- **C errata:** la sostituzione di `copy[1]` non sostituisce `original[1]`.
- **D errata:** entrambe le previsioni ignorano la distinzione fra modifica di una riga e sostituzione del suo riferimento.

2. Risposta breve

Il risultato richiesto è 6, ma il metodo restituisce 3: usa sempre la lunghezza della prima riga e non visita il secondo elemento della seconda. La riga 5 deve usare la lunghezza della riga corrente:

```
for (int c = 0; c < values[r].length; c++) {
```

Il ciclo esterno non entra per una tabella vuota; quello interno non entra per una riga vuota.

Esercizio 3 — Ricerca di un codice

Tempo indicativo: 5–7 minuti · Java 25

Un archivio conserva codici nell'ordine in cui sono stati acquisiti. Il metodo `firstIndex` deve individuare la prima posizione di un codice in quell'ordine, oppure restituire `-1` se manca. Il chiamante continua a usare l'array dopo la ricerca, quindi il suo contenuto non deve cambiare. Un frammento separato prepara un array più lungo per acquisizioni future. Gli array sono non nulli, anche vuoti; i codici possono essere duplicati e non ordinati. Per un array ordinato, `Arrays.binarySearch` restituisce un indice corrispondente, non necessariamente il primo tra duplicati, oppure un valore negativo se l'elemento manca.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato, con `java.util.Arrays` importata. I numeri di riga non fanno parte del codice.

```
01  class Codes {
02      static int firstIndex(int[] values, int code) {
03          Arrays.sort(values);
04          int position = Arrays.binarySearch(values, code);
05          return position < 0 ? -1 : position;
06      }
07  }
```

```
08  int[] source = {4, 7};
09  int[] expanded = Arrays.copyOf(source, 4);
10  expanded[0] = 9;
11  IO.println(Arrays.toString(expanded));
12  IO.println(Arrays.toString(source));
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le due righe nell'ordine.

- **A.** `[9, 7, 0, 0]`, poi `[9, 7]`.
- **B.** `[9, 7]`, poi `[4, 7]`.

- **C.** Solleva un'eccezione perché la nuova lunghezza supera quella originale.
- **D.** `[9, 7, 0, 0]`, poi `[4, 7]`.

2. Risposta breve

Valuta `firstIndex` rispetto al contratto. Fornisci un input di tre elementi distinti che permetta di controllare sia l'indice restituito sia lo stato dell'array dopo la chiamata. Descrivi poi un algoritmo che rispetti anche il requisito sui duplicati, senza scrivere il codice.

Lunghezza indicativa: un input con il codice cercato e 2–3 frasi.

Mostra la soluzione dell'esercizio 3

1. Scelta multipla: D

`Arrays.copyOf` crea un array indipendente e riempie le posizioni aggiuntive di un `int[]` con zero. La modifica riguarda solo expanded: si stampano `[9, 7, 0, 0]` e `[4, 7]`.

- **A errata:** il nuovo array non condivide le proprie posizioni con quello originale.
- **B errata:** la lunghezza richiesta è quattro.
- **C errata:** una lunghezza maggiore è ammessa.

2. Risposta breve

Con `values = {9, 2, 5}` e codice 9, l'indice richiesto è `0`, ma il metodo restituisce 2 e lascia l'array ordinato come `{2, 5, 9}`. Un algoritmo conforme visita l'array da sinistra a destra e restituisce subito l'indice del primo elemento uguale al codice. Se la visita termina senza trovarlo, restituisce `-1`: non modifica l'input e gestisce anche duplicati e array vuoti.

Alternativa valida: altre strategie che preservino l'array e restituiscano il minimo indice originale; non serve introdurre ordinamento o strutture aggiuntive.

Esercizio 4 — Riconoscimento di un comando

Tempo indicativo: 5–7 minuti · Java 25

Una console riceve comandi testuali da un operatore. `isStart` deve riconoscere START ignorando maiuscole/minuscole e spazi iniziali e finali. Il testo ricevuto viene conservato anche in un registro e deve rimanere invariato. La console può creare nuove stringhe durante la preparazione dei comandi; il risultato non deve dipendere dall'identità degli oggetti. Gli input sono non nulli e contengono solo lettere ASCII e spazi.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato, con `java.util.Locale` importata. I numeri di riga non fanno parte del codice.

```java
class Command {
    static boolean isStart(String text) {
        String clean = new String(text.strip().toUpperCase(Locale.ROOT));
        return clean == "START";
    }
}
```

```java
String original = " start ";
String saved = original;
original = original.strip();
original.toUpperCase(Locale.ROOT);
IO.println("[" + original + "]");
IO.println("[" + saved + "]");
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le due righe nell'ordine.

- **A.** `[START]`, poi `[START]`.
- **B.** `[start]`, poi `[ start ]`.
- **C.** `[START]`, poi `[ start ]`.
- **D.** `[ start ]`, poi `[ start ]`.

2. Risposta breve

Valuta se `isStart` rispetta il contratto per tutti gli input ammessi. Se non lo rispetta, fornisci un input che lo dimostri e una modifica minima al metodo; altrimenti giustificane la correttezza. Non basarti su ipotesi sul riuso degli oggetti da parte dei metodi di trasformazione.

Lunghezza indicativa: un input, una riga di codice e 1–2 frasi.

Mostra la soluzione dell'esercizio 4

1. Scelta multipla: B

L'assegnazione conserva in `original` il risultato di `strip()`. Il risultato di `toUpperCase` viene invece scartato; `saved` mantiene il riferimento al testo iniziale. Si stampano `[start]` e `[ start ]`.

- **A errata:** una trasformazione non modifica in-place le stringhe condivise.
- **C errata:** il risultato della conversione in maiuscolo non viene assegnato.
- **D errata:** il risultato di `strip()` viene assegnato a `original`.

2. Risposta breve

Per esempio, `isStart(" start ")` restituisce `false`, mentre il contratto richiede `true`. `new String(...)` crea esplicitamente un oggetto distinto dal letterale: `==` confronta i riferimenti, non il contenuto. Sostituire la riga 4 con:

```
return clean.equals("START");
```

Alternativa valida: `return text.strip().equalsIgnoreCase("START");`, eliminando anche la variabile locale non più necessaria. Nessuna delle due soluzioni modifica il testo ricevuto.

Esercizio 5 — Valore di un campo

Tempo indicativo: 5–7 minuti · Java 25

Un lettore di configurazioni riceve una riga alla volta nel formato `chiave:valore`. Il metodo `value` deve estrarre tutto ciò che segue il primo separatore, anche quando il valore è vuoto o contiene altri `:`. Il chiamante usa il risultato come testo e non richiede ulteriori controlli sul contenuto della chiave. Una riga priva di separatore non è valida e deve produrre `IllegalArgumentException`, non essere interpretata come un valore. La riga è non nulla e contiene solo caratteri ASCII; `indexOf` restituisce `-1` se il carattere cercato manca.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Field {
02      static String value(String line) {
03          int separator = line.indexOf(':');
04          return line.substring(separator + 1);
05      }
06  }
```

```
07  String line = "role:admin:local";
08  IO.println(Field.value(line));
09  IO.println(Field.value("role:").length());
```

1. Scelta multipla

Seleziona l’unico output corretto, indicando le due righe nell’ordine.

- **A.** `admin`, poi `0`.
- **B.** `local`, poi `1`.
- **C.** `admin:local`, poi `0`.
- **D.** `admin:local`, poi un’eccezione alla riga 9.

2. Risposta breve

Valuta separatamente la chiamata `Field.value("admin")` rispetto al contratto. Se necessario, modifica il metodo conservando il comportamento richiesto per i valori vuoti e per i valori che contengono altri `:`.

Lunghezza indicativa: una frase e massimo 4 righe per il corpo del metodo.

Mostra la soluzione dell'esercizio 5

1. Scelta multipla: C

Il primo `:` si trova all'indice 4; `substring(5)` conserva tutto il suffisso `admin:local`. Per `"role:"`, la sottostringa inizia alla fine del testo ed è vuota, quindi ha lunghezza `0`.

- **A errata:** `substring` non interrompe il risultato al secondo separatore.
- **B errata:** viene cercato il primo separatore, non l'ultimo; la stringa vuota non ha lunghezza uno.
- **D errata:** iniziare la sottostringa alla lunghezza della stringa è ammesso.

2. Risposta breve

Con `"admin"`, `separator` vale `-1` e il metodo restituisce l'intera riga invece di segnalare un errore. Un corpo conforme è:

```
int separator = line.indexOf(':');
if (separator < 0) throw new IllegalArgumentException("Separatore mancante");
return line.substring(separator + 1);
```

Il controllo non rifiuta né il suffisso vuoto né eventuali separatori successivi. Sono accettabili anche le parentesi graffe attorno al corpo dell'`if`.

Esercizio 6 — Registro dei nomi

Tempo indicativo: 5–7 minuti · Java 25

Un'applicazione mantiene un registro testuale dei partecipanti a un evento. Il registro cresce tramite `StringBuilder`, mentre una stringa salvata rappresenta una fotografia del testo in un determinato momento. Il metodo `readName` acquisisce un nuovo nome leggendo un'intera riga, inclusi eventuali spazi interni. La riga vuota è ammessa e deve essere restituita senza saltarla. Lo scanner è fornito dal chiamante e non deve essere chiuso dal metodo; si assume che la prossima riga esista e che lo scanner sia posizionato al suo inizio.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato, con `java.util.Scanner` importata. I numeri di riga non fanno parte del codice.

```java
class Names {
    static String readName(Scanner input) {
        return input.next();
    }
}
```

```java
StringBuilder register = new StringBuilder();
register.append("Ada");
String saved = register.toString();
register.append(" Linus");
IO.println(saved);
IO.println(register.length());
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le due righe nell'ordine.

- **A.** Ada, poi 9.
- **B.** Ada Linus, poi 9.
- **C.** Ada, poi 3.
- **D.** AdaLinus, poi 8.

2. Risposta breve

Valuta `readName` con uno scanner costruito dalla stringa `"Ada Lovelace\n"`. Indica ciò che viene letto rispetto al requisito e proponi, se necessaria, una modifica minima. La soluzione deve funzionare anche quando la prossima riga è vuota.

Lunghezza indicativa: una riga di codice e 1–2 frasi.

Mostra la soluzione dell'esercizio 6

1. Scelta multipla: A

`toString()` produce una stringa che non cambia quando viene modificato il builder. `saved` contiene `Ada`; il builder finale contiene `Ada Linus`, lungo nove caratteri, incluso lo spazio.

- **B errata:** saved non è una vista modificabile del builder.
- **C errata:** il secondo append modifica il builder e ne aumenta la lunghezza.
- **D errata:** la stringa aggiunta contiene uno spazio e saved non contiene il testo finale.

2. Risposta breve

`next()` legge solo `Ada`, non l'intera riga. Il corpo richiesto è:

```
return input.nextLine();
```

`nextLine()` legge anche gli spazi interni e restituisce `""` per una riga vuota. Non si deve chiudere lo scanner né usare `nextLine()` una seconda volta per saltare righe: il contratto lo colloca già all'inizio della riga da leggere.

Esercizio 7 — Numero di tentativi

Tempo indicativo: 5–7 minuti · Java 25

Un client permette di configurare quanti tentativi effettuare per un'operazione. Il valore `null` significa «non specificato» e, nella versione attuale, deve produrre il valore predefinito 3. Un valore esplicito, incluso zero, deve invece essere rispettato. Il metodo `effective` restituisce un `int` da usare nel resto dell'applicazione, senza modificare la configurazione ricevuta. Gli eventuali valori non nulli sono tra 0 e 10; il frammento sulle variabili `stored` e `requested` è una prova separata dei confronti numerici e non rappresenta una configurazione valida del client.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Attempts {
02      static int effective(Integer supplied) {
03          int value = supplied;
04          if (supplied == null) {
05              return 3;
06          }
07          return value;
08      }
09  }
```

```
10  Integer stored = Integer.valueOf(2000);
11  int requested = 2000;
12  IO.println(stored == requested);
```

1. Scelta multipla

Che cosa accade eseguendo le righe 10–12? Seleziona una sola risposta.

- **A.** Stampa `false`, perché uno degli operandi è un oggetto.
- **B.** Stampa `true` soltanto se il wrapper proviene dalla cache.
- **C.** Non compila perché i tipi dei due operandi sono diversi.
- **D.** Stampa sempre `true`.

2. Risposta breve

Esamina separatamente `Attempts.effective(null)` e `Attempts.effective(0)`: indica il comportamento effettivo e se rispetta il contratto. Ora il valore predefinito deve essere scelto da ciascun chiamante, senza stato globale. Proponi la nuova firma e un corpo che conservi la distinzione tra assenza e zero; assumi che anche il predefinito ricevuto sia tra 0 e 10.

Lunghezza indicativa: 1–2 frasi e massimo 4 righe di codice, inclusa la firma.

Mostra la soluzione dell'esercizio 7

1. Scelta multipla: D

Nel confronto con un `int`, `stored` viene convertito nel valore primitivo tramite unboxing. I valori sono entrambi `2000`, quindi si stampa `true`, indipendentemente dalla cache dei wrapper.

- **A errata:** questo confronto non avviene fra due riferimenti.
- **B errata:** la cache non determina il risultato di un confronto fra valori primitivi.
- **C errata:** il confronto è consentito grazie all'unboxing.

2. Risposta breve

`effective(null)` solleva `NullPointerException` alla riga 3, prima del controllo, e viola il contratto; `effective(0)` restituisce correttamente zero. Il nuovo parametro consente di scegliere il predefinito per chiamata, controllando l'assenza prima dell'unboxing:

```
static int effective(Integer supplied, int defaultValue) {
    return supplied == null ? defaultValue : supplied;
}
```

Per esempio, `effective(null, 5)` restituisce `5`, mentre `effective(0, 5)` restituisce `0`. È equivalente una versione con `if` esplicito; non serve uno stato statico modificabile.

Esercizio 8 — Conversione di un'età

Tempo indicativo: 5–7 minuti · Java 25

Un modulo di iscrizione riceve l'età come testo e deve convertirla prima di salvarla. Il metodo `parseAge` accetta età intere tra 0 e 130, inclusi, ignorando spazi iniziali e finali. Un input non valido deve produrre `IllegalArgumentException` o una sua sottoclasse, così che il chiamante possa richiedere una correzione senza salvare valori predefiniti. Un frammento separato confronta due rappresentazioni numeriche già presenti nel sistema. Il testo ricevuto da `parseAge` è non nullo; `NumberFormatException`, sollevata dal parsing di testi non numerici o fuori dall'intervallo di `int`, è una sottoclasse di `IllegalArgumentException`.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Ages {
02      static int parseAge(String text) {
03          return Integer.parseInt(text.strip());
04      }
05  }
```

```
06  Integer entered = Integer.valueOf(7);
07  Long archived = Long.valueOf(7);
08  IO.println(entered.equals(archived));
09  IO.println(entered.intValue() == archived.longValue());
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le due righe nell'ordine.

- **A.** `true`, poi `true`.
- **B.** `false`, poi `false`.
- **C.** Non compila perché `equals` non accetta un `Long`.
- **D.** `false`, poi `true`.

2. Risposta breve

Valuta `parseAge` per gli input `"  42  "` e `"131"`. Se necessario, riscrivine il corpo perché rispetti l'intero contratto, senza trasformare un dato non valido in un valore predefinito.

Lunghezza indicativa: una frase e massimo 5 righe di codice.

Mostra la soluzione dell'esercizio 8

1. Scelta multipla: D

`Integer.equals` non considera uguale un oggetto `Long`, anche se rappresenta lo stesso numero. Il secondo confronto usa valori primitivi, promuovendo l'`int` a `long`, e restituisce `true`.

- **A errata:** `equals` tiene conto del tipo del wrapper.
- **B errata:** il confronto numerico tra i valori primitivi restituisce `true`.
- **C errata:** `equals` accetta un argomento di tipo `Object`, quindi la chiamata compila.

2. Risposta breve

`"  42  "` produce correttamente 42; `"131"` produce invece un valore fuori dal dominio ammesso. Un corpo conforme è:

```java
int age = Integer.parseInt(text.strip());
if (age < 0 || age > 130) {
    throw new IllegalArgumentException("Età fuori intervallo");
}
return age;
```

Gli input non numerici e quelli fuori dall'intervallo di `int` continuano a produrre `NumberFormatException`, già ammessa dal contratto. Non occorre intercettarla né trasformarla in un valore predefinito.

Esercizio 9 — Aggiornamento delle scorte

Tempo indicativo: 5–7 minuti · Java 25

Un magazzino rappresenta le quantità disponibili tramite numeri interi. Un'operazione di incremento deve permettere al chiamante di ottenere la quantità aumentata di uno. La quantità può essere conservata in una variabile `Integer`, ma non si vuole introdurre un oggetto mutabile per aggiornarla. Un frammento separato usa una lista per rimuovere una quantità da una sequenza di scorte. Tutte le quantità sono piccoli interi non negativi e non nulli; non si considera overflow.

Il metodo va dichiarato nel corpo di una classe e le istruzioni in un suo metodo statico, con `java.util.List` e `java.util.ArrayList` importate. Per `List<Integer>`, `remove(int index)` rimuove per posizione, mentre `remove(Object value)` rimuove la prima occorrenza uguale al valore. I numeri di riga non fanno parte del codice.

```
01  static void increment(Integer value) {
02      value++;
03  }
```

```
04  List<Integer> stock = new ArrayList<>(List.of(10, 20, 30));
05  stock.remove(1);
06  IO.println(stock);
```

1. Scelta multipla

Qual è l'output delle righe 4–6? Seleziona una sola risposta.

- **A.** `[10, 20, 30]`.
- **B.** `[10, 30]`.
- **C.** `[20, 30]`.
- **D.** Solleva un'eccezione perché 1 non compare nella lista.

2. Risposta breve

Considera separatamente `Integer quantity = 3; increment(quantity);`. Il chiamante osserva l'incremento nella variabile `quantity`? Motiva e proponi una modifica al metodo e alla chiamata per ottenere l'aggiornamento, senza introdurre un contenitore mutabile.

Lunghezza indicativa: 1–2 frasi e massimo 4 righe di codice complessive.

Mostra la soluzione dell'esercizio 9

1. Scelta multipla: B

L'argomento `1` ha tipo `int`, quindi viene scelto `remove(int index)`: si elimina l'elemento all'indice 1, cioè `20`. La lista stampata è `[10, 30]`.

- **A errata:** l'operazione rimuove un elemento.
- **C errata:** gli indici partono da zero; l'indice 1 non indica il primo elemento.
- **D errata:** la ricerca del valore 1 richiederebbe l'overload con argomento oggetto.

2. Risposta breve

Il chiamante conserva 3: `Integer` è immutabile e `value++` riassegna solo il parametro locale dopo unboxing, incremento e boxing. Il metodo può restituire il nuovo valore, che il chiamante deve assegnare:

```
static int increment(int value) { return value + 1; }
Integer quantity = 3;
quantity = increment(quantity);
```

Alternativa valida: mantenere un parametro `Integer` e restituire un `Integer`, rispettando l'assunzione di non nullità. Il punto essenziale è il contratto di restituzione e la riassegnazione nel chiamante, non il tipo wrapper o primitivo della firma.

Esercizio 10 — Media dei consumi

Tempo indicativo: 5–7 minuti · Java 25

Un'applicazione riceve i consumi giornalieri di un dispositivo. Un valore negativo indica una lettura mancante e deve essere ignorato. Il componente deve calcolare la media delle sole letture disponibili, senza modificare l'array. La media può avere una parte frazionaria. L'array è non nullo, anche vuoto, e contiene al massimo cento interi tra -1 e 100; non si considera overflow.

Il frammento è eseguito in un metodo. I numeri di riga non fanno parte del codice.

```java
int[] readings = {2, -1, 3};
int total = 0;
int count = 0;
for (int value : readings) {
    if (value < 0) continue;
    total += value;
    count++;
}
double average = total / count;
IO.println(average);
```

1. Scelta multipla

Qual è l'output? Seleziona una sola risposta.

- **A.** `2.5`.
- **B.** `1.0`.
- **C.** Solleva `ArithmeticException`.
- **D.** `2.0`.

2. Risposta aperta

Proponi firma e contratto di un metodo che riceva l'array e restituisca la media frazionaria, segnalando con `IllegalArgumentException` l'assenza di letture valide. Indica quali parti del ciclo riutilizzare e come terminare il metodo. Non riscrivere il ciclo.

Lunghezza indicativa: una firma, massimo 3 righe finali e una frase.

Mostra la soluzione dell'esercizio 10

1. Scelta multipla: D

La somma è 5 e il conteggio è 2. La divisione tra interi produce 2, poi convertito in `double`.

- **A errata:** l'assegnazione a `double` non cambia il tipo della divisione già eseguita.
- **B errata:** la lettura negativa non partecipa né alla somma né al conteggio.
- **C errata:** il denominatore è 2, non zero.

2. Risposta aperta

La firma può essere `static double average(int[] readings)`. Riutilizzo il ciclo con somma e conteggio locali; dopo il ciclo eseguo:

```java
if (count == 0) throw new IllegalArgumentException("Nessuna lettura");
return (double) total / count;
```

Il controllo copre array vuoti e array di sole letture mancanti; il cast rende frazionaria la divisione. È equivalente accumulare la somma in un `double`.

Esercizio 11 — Ricerca tra i codici

Tempo indicativo: 5–7 minuti · Java 25

Un catalogo conserva i codici dei prodotti in un array. Il metodo `contains` deve indicare se un codice è presente, senza modificare il contenuto o l'ordine dell'array ricevuto. Il chiamante può quindi continuare a usare l'ordine originale per la visualizzazione. Due test verificano aspetti del contratto.

Gli array e le stringhe sono non nulli. `Arrays.sort` ordina l'array ricevuto; `Arrays.binarySearch` cerca nell'array ordinato e restituisce un indice non negativo se trova il valore, un numero negativo altrimenti.

Il frammento appartiene a una classe di test; gli import sono omessi. `@Test` identifica un test, `assertTrue` richiede una condizione vera e `assertArrayEquals(atteso, effettivo)` confronta contenuto e ordine degli array. Ogni test usa i propri dati. I numeri di riga non fanno parte del codice.

```
01  static boolean contains(String[] codes, String searched) {
02      Arrays.sort(codes);
03      return Arrays.binarySearch(codes, searched) >= 0;
04  }
05
06  @Test
07  void existingCode() {
08      String[] codes = {"B", "A", "C"};
09      assertTrue(contains(codes, "A"));
10  }
11
12  @Test
13  void catalogOrder() {
14      String[] codes = {"B", "A", "C"};
15      contains(codes, "A");
16      assertArrayEquals(new String[]{"B", "A", "C"}, codes);
17  }
```

1. Scelta multipla

Qual è l'esito dei due test?

- **A.** Passano entrambi.
- **B.** Passa soltanto `catalogOrder`.
- **C.** Passa soltanto `existingCode`.
- **D.** Falliscono entrambi.

2. Risposta aperta

Un collega propone di usare soltanto array già ordinati nei test del metodo. Questa scelta permette di verificare adeguatamente il requisito sulla conservazione dell'input? Motiva in **1–3 frasi**, senza scrivere codice.

Mostra la soluzione dell’esercizio 11

1. Scelta multipla: C

La ricerca trova `A`, quindi `existingCode` passa. `catalogOrder` fallisce: dopo la chiamata l'array contiene `{"A", "B", "C"}`, mentre il test si aspetta `{"B", "A", "C"}`.

- **A errata:** il risultato della ricerca è corretto, ma l'ordine dell'input è stato modificato.
- **B errata:** è proprio il test sull'ordine a fallire.
- **D errata:** l'ordinamento rende valida la ricerca binaria e il codice cercato viene trovato.

2. Risposta aperta

No: su un array già ordinato, `sort` non cambia l'ordine osservabile e il test non distingue questa implementazione da una che lascia davvero invariato l'input. Occorre includere un array non ordinato e confrontarne lo stato dopo la chiamata con una copia dello stato precedente; verificare soltanto il booleano restituito non basta.

E3 - Java Object-Oriented Programming

Esercizio 1 — Controllo del volume

Tempo indicativo: 5–7 minuti · Java 25

Un lettore musicale espone un controllo del volume usato da più comandi della schermata. Ogni controllo conserva un livello tra 0 e 10. Il costruttore e `setLevel` rifiutano valori esterni a questo intervallo con `IllegalArgumentException`, senza modificare lo stato di un oggetto già costruito. Due variabili possono riferirsi allo stesso controllo, senza creare una copia. Il programma è monothread.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  final class Volume {
02      private int level;
03      Volume(int initial) { setLevel(initial); }
04      public int getLevel() { return level; }
05      public void setLevel(int value) {
06          if (value < 0 || value > 10)
07              throw new IllegalArgumentException();
08          level = value;
09      }
10  }
```

```
11  Volume first = new Volume(3);
12  Volume second = first;
13  second.setLevel(5);
14  IO.println(first.getLevel());
```

1. Scelta multipla

Che cosa accade eseguendo le righe 11–14? Seleziona una sola risposta.

- **A.** Stampa 5.
- **B.** Stampa 3.
- **C.** Solleva `IllegalArgumentException`.
- **D.** Non compila perché `level` è privato.

2. Risposta aperta

Per una nuova versione, dopo la costruzione il chiamante deve poter leggere il livello e variarlo soltanto di un'unità per operazione, in aumento o in diminuzione. Ai limiti, l'operazione non ammessa deve sollevare `IllegalArgumentException` lasciando invariato lo stato.

Proponi la nuova API: indica firme e visibilità dei metodi necessari, quali membri attuali cambieresti e dove collocheresti la validazione. Non implementare i corpi e non cambiare il tipo del campo.

Lunghezza indicativa: massimo 4 firme di metodi e 2 frasi. Non occorre aggiornare le istruzioni dimostrative.

Mostra la soluzione dell'esercizio 1

1. Scelta multipla: A

Si stampa 5: le due variabili indicano lo stesso oggetto.

- **B errata:** non viene creata una copia del volume.
- **C errata:** 5 è ammesso.
- **D errata:** il campo viene usato attraverso metodi accessibili.

2. Risposta aperta

Conservo `getLevel()` pubblico; sostituisco il setter pubblico con `public void increase()` e `public void decrease()`. Mantengo il costruttore con il livello iniziale e rendo il setter privato, riutilizzandone il controllo prima dell'assegnazione. I due nuovi metodi gli passano rispettivamente `level + 1` e `level - 1`, così il fallimento non modifica lo stato.

Alternativa valida: controlli nel costruttore e nei due metodi, senza setter; l'API deve impedire aggiornamenti arbitrari dall'esterno.

Esercizio 2 — Numero di una sessione

Tempo indicativo: 5–7 minuti · Java 25

Un servizio crea una sessione per ogni nuovo collegamento. La classe mantiene un conteggio delle sessioni costruite durante l'esecuzione. Il metodo `number` viene invocato su una singola sessione per consultarne l'informazione corrente. Il programma è monothread, parte senza sessioni preesistenti e ne crea al massimo mille. Non vengono distrutte o riutilizzate sessioni per questo conteggio.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```java
final class Session {
    private static int count;
    Session() { count++; }
    public int number() { return count; }
}
```

```java
Session first = new Session();
Session second = new Session();
IO.println(first.number());
IO.println(second.number());
```

1. Scelta multipla

Seleziona l’unico output corretto, indicando le due righe nell’ordine.

- **A.** 1, poi 2.
- **B.** 1, poi 1.
- **C.** 2, poi 2.
- **D.** `0`, poi `0`.

2. Risposta aperta

Per una nuova versione, ogni sessione deve ricevere alla costruzione un numero distinto e stabile: 1 per la prima, 2 per la seconda, e così via. Il numero non deve poter essere cambiato dal chiamante e deve rimanere lo stesso anche dopo altre costruzioni.

Scegli quali informazioni debbano appartenere alla classe e quali a ciascuna istanza. Mostra i campi, il costruttore e `number()` della nuova versione, senza aggiungere setter.

Lunghezza indicativa: massimo 5 righe di codice e una frase di motivazione.

Mostra la soluzione dell’esercizio 2

1. Scelta multipla: C

Entrambe le chiamate leggono il campo statico, che dopo due costruzioni vale 2.

- **A errata:** il codice non salva un numero distinto in ogni istanza.
- **B errata:** il contatore è condiviso e viene incrementato due volte.
- **D errata:** i costruttori aggiornano il contatore prima delle letture.

2. Risposta aperta

```
private static int count;
private final int number;
Session() { number = ++count; }
public int number() { return number; }
```

Il contatore appartiene alla classe; il numero assegnato appartiene all'istanza ed è stabile perché `final` e senza setter. L'assunzione monothread evita problemi di assegnazione concorrente.

Esercizio 3 — Dimensioni di una scatola

Tempo indicativo: 5–7 minuti · Java 25

Un catalogo descrive scatole rettangolari attraverso larghezza e altezza. Dopo la costruzione, le dimensioni non devono essere riassegnate. Per le scatole quadrate è disponibile un costruttore con un solo argomento. Il metodo `area` viene usato per confrontare gli ingombri. Le dimensioni ricevute sono piccoli interi, ma possono essere non positive; non considerare overflow.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  final class Box {
02      private final int width;
03      private final int height;
04      Box(int side) { this(side, side); }
05      Box(int width, int height) {
06          this.width = width;
07          this.height = height;
08      }
09      public int area() { return width * height; }
10  }
```

```
11  Box square = new Box(3);
12  IO.println(square.area());
```

1. Scelta multipla

Che cosa accade eseguendo le righe 11–12? Seleziona una sola risposta.

- **A.** Stampa 9.
- **B.** Stampa 6.
- **C.** Stampa `0`, perché i campi sono `final`.
- **D.** Non compila perché un costruttore non può chiamarne un altro.

2. Risposta breve

Una review richiede di garantire larghezza e altezza strettamente positive fin dalla costruzione, rifiutando gli altri valori con `IllegalArgumentException`. Indica dove inseriresti il controllo per coprire entrambi i costruttori senza duplicarlo e mostra il controllo. Spiega se i campi `final` siano da soli sufficienti a garantire questo vincolo.

Lunghezza indicativa: massimo 2 righe di codice e 2 frasi.

Mostra la soluzione dell'esercizio 3

1. Scelta multipla: A

Il costruttore delega a quello con due parametri, assegnando 3 a entrambe le dimensioni: l'area è 9.

- **B errata:** l'area è un prodotto, non una somma.
- **C errata:** `final` non impedisce l'inizializzazione nel costruttore.
- **D errata:** la delega con `this(...)` è consentita.

2. Risposta breve

Inserisco il controllo nel costruttore a due parametri, prima delle assegnazioni: anche quello a un parametro vi delega.

```
if (width <= 0 || height <= 0) throw new IllegalArgumentException();
```

`final` impedisce riassegnazioni successive, ma non rifiuta valori iniziali non validi.

Esercizio 4 — Tariffe di consegna

Tempo indicativo: 5–7 minuti · Java 25

Un negozio offre diversi tipi di consegna. Ogni consegna espone la propria tariffa attraverso `fee`. Il componente `Quotes` deve fornire un preventivo uguale alla tariffa dell'oggetto ricevuto, anche quando vengono introdotti nuovi sottotipi di `Delivery`. Il chiamante può conservare una consegna mediante un riferimento al tipo base. Gli argomenti sono non nulli e le tariffe sono piccoli interi non negativi.

Le istruzioni dopo le classi sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```java
class Delivery {
    public int fee() { return 5; }
}
class Express extends Delivery {
    @Override public int fee() { return 9; }
}
class Quotes {
    static int quote(Delivery delivery) {
        if (delivery instanceof Express) return 9;
        return 5;
    }
}
```

```java
Delivery delivery = new Express();
IO.println(delivery.fee());
IO.println(new Delivery().fee());
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le due righe nell'ordine.

- **A.** 5, poi 5.
- **B.** 5, poi 9.
- **C.** 9, poi 9.
- **D.** 9, poi 5.

2. Risposta breve

Valuta se `quote` rispetta il requisito per ogni possibile sottotipo. Se non è così, descrivi un sottotipo e una chiamata che lo dimostrino, indicando risultato richiesto e risultato effettivo. Non modificare il codice esistente.

Lunghezza indicativa: 2–3 frasi oppure una piccola classe e una frase.

Mostra la soluzione dell'esercizio 4

1. Scelta multipla: D

La chiamata sul primo oggetto usa l'override di `Express`, restituendo 9; il secondo oggetto è un `Delivery` e restituisce 5.

- **A errata:** il tipo dichiarato non elimina il dispatch dinamico.
- **B errata:** scambia le due implementazioni.
- **C errata:** il secondo oggetto non è un `Express`.

2. Risposta breve

Un sottotipo `Pickup extends Delivery` che ridefinisce `fee()` restituendo zero è un controesempio: `Quotes.quote(new Pickup())` restituisce 5 anziché `0`. Anche un sottotipo di `Express` con tariffa diversa verrebbe gestito male. La selezione dei due valori fissi non rispetta il contratto generale.

Esercizio 5 — Confronto di biglietti

Tempo indicativo: 5–7 minuti · Java 25

Un sistema di prenotazione rappresenta ciascun biglietto con un codice intero. Più oggetti possono contenere lo stesso codice perché arrivano da richieste diverse. Alcuni client conservano i biglietti attraverso variabili di tipo `Object`. Il metodo di confronto mostrato fa parte dell'API corrente; la classe non dichiara altri metodi oltre a quelli riportati. I codici non cambiano dopo la costruzione.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  final class Ticket {
02      private final int code;
03      Ticket(int code) { this.code = code; }
04      public boolean equals(Ticket other) {
05          return other != null && code == other.code;
06      }
07  }
```

```
08  Ticket first = new Ticket(7);
09  Object second = new Ticket(7);
10  IO.println(first.equals(second));
```

1. Scelta multipla

Che cosa accade eseguendo le righe 8–10? Seleziona una sola risposta.

- **A.** Stampa `true`, perché i codici sono uguali.
- **B.** Non compila perché `Ticket` dichiara soltanto `equals(Ticket)`.
- **C.** Solleva `ClassCastException`.
- **D.** Stampa `false`.

2. Risposta aperta

Si decide di definire l'uguaglianza logica di due biglietti attraverso il codice, anche nei client che li trattano come `Object` o li inseriscono in una collezione hash. Un biglietto non deve risultare uguale a `null` o a un oggetto di un altro tipo.

Indica le firme dei metodi da ridefinire e descrivi le rispettive regole. Precisa la relazione richiesta tra i loro risultati, senza implementare i corpi.

Lunghezza indicativa: 2 firme e 2–3 frasi.

Mostra la soluzione dell'esercizio 5

1. Scelta multipla: D

L'argomento ha tipo dichiarato `Object`: viene scelto `equals(Object)` ereditato da `Object`, che confronta l'identità. I due oggetti sono distinti.

- **A errata:** `equals(Ticket)` non ridefinisce `equals(Object)`.
- **B errata:** il metodo ereditato è comunque disponibile.
- **C errata:** non viene eseguito alcun cast.

2. Risposta aperta

Ridefinisco `public boolean equals(Object other)` e `public int hashCode()`. `equals` accetta soltanto un `Ticket` con lo stesso codice, rifiutando `null` e altri tipi. `hashCode` dipende dal codice: oggetti uguali devono avere lo stesso hash, mentre la coincidenza degli hash non implica uguaglianza.

Alternativa valida: trasformare il tipo in un record con il solo codice, se è ammesso cambiare la dichiarazione e l'API preserva il requisito; i metodi generati forniscono la semantica richiesta.

Esercizio 6 — Report del personale

Tempo indicativo: 5–7 minuti · Java 25

Un ufficio produce un report delle persone registrate. L'elenco può contenere persone generiche e dipendenti. Soltanto i dipendenti espongono una retribuzione, rappresentata da un piccolo intero non negativo. Il codice conserva tutti gli elementi mediante il tipo comune `Person`, ma deve distinguere quali operazioni siano disponibili sugli oggetti effettivi. Gli elementi sono non nulli.

Le istruzioni dopo le classi sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Person { }
02  class Employee extends Person {
03      private final int salary;
04      Employee(int salary) { this.salary = salary; }
05      public int salary() { return salary; }
06  }
```

```
07  Employee original = new Employee(20);
08  Person person = original;
09  Employee recovered = (Employee) person;
10  IO.println(original == recovered);
```

1. Scelta multipla

Che cosa accade eseguendo le righe 7–10? Seleziona una sola risposta.

- **A.** Stampa `false`, perché il cast crea un nuovo oggetto.
- **B.** Stampa `true`.
- **C.** Non compila perché non si può tornare da `Person` a `Employee`.
- **D.** Solleva `ClassCastException`.

2. Risposta breve

In un'esecuzione separata il report usa:

```
11  Person[] people = {new Employee(20), new Person()};
12  for (Person value : people) {
13      IO.println(((Employee) value).salary());
14  }
```

Il requisito è stampare la retribuzione dei soli dipendenti, ignorando le persone generiche. Valuta il ciclo e, se necessario, correggilo senza aggiungere una retribuzione fittizia a `Person`.

Lunghezza indicativa: una frase e massimo 4 righe di codice.

Mostra la soluzione dell'esercizio 6

1. Scelta multipla: B

Il cast riesce perché l'oggetto effettivo è un `Employee` e non crea una copia: i riferimenti restano uguali.

- **A errata:** un cast non costruisce un oggetto.
- **C errata:** il cast esplicito è ammesso.
- **D errata:** l'oggetto soddisfa il tipo richiesto dal cast.

2. Risposta breve

Il ciclo originale stampa `20` e poi fallisce con `ClassCastException` sulla persona generica; non soddisfa il requisito. Si può verificare il tipo prima dell'accesso:

```
for (Person value : people) {
    if (value instanceof Employee employee)
        IO.println(employee.salary());
}
```

Alternativa valida: `instanceof Employee` seguito dal cast esplicito nello stesso ramo protetto.

Esercizio 7 — Invio di notifiche

Tempo indicativo: 5–7 minuti · Java 25

Un'applicazione di prenotazione deve avvisare un operatore quando si verifica un evento che richiede attenzione. La classe `Alarm` rappresenta il componente che attiva l'avviso, mentre il canale ricevuto dal costruttore si occupa dell'invio del testo. Il chiamante sceglie il canale quando costruisce ciascun allarme e può conservarlo attraverso il tipo `Notifier`. Nell'esempio sono disponibili email e SMS, così da poter configurare allarmi per destinatari diversi senza cambiare `Alarm`. Per rendere osservabile il comportamento senza servizi esterni, l'invio è simulato mediante una stampa sulla console.

Assunzioni: il canale e i testi sono non nulli; l'invio termina normalmente. Non sono richiesti rete, concorrenza o gestione degli errori di consegna.

Le istruzioni dopo le dichiarazioni sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  interface Notifier {
02      void send(String text);
03  }
04  class Email implements Notifier {
05      @Override
06      public void send(String text) {
07          IO.println("EMAIL: " + text);
08      }
09  }
10  class Sms implements Notifier {
11      @Override
12      public void send(String text) {
13          IO.println("SMS: " + text);
14      }
15  }
16  class Alarm {
```

```
17      private final Notifier channel;
18      Alarm(Notifier channel) {
19          this.channel = channel;
20      }
21      void fire() {
22          channel.send("Alert");
23      }
24  }
```

```
25  Notifier channel = new Email();
26  Alarm first = new Alarm(channel);
27  channel = new Sms();
28  Alarm second = new Alarm(channel);
29  first.fire();
30  second.fire();
```

1. Scelta multipla

Qual è l'output delle righe 25–30? Seleziona una sola risposta, considerando le due righe nell'ordine.

- **A.** `SMS: Alert`, poi `SMS: Alert`.
- **B.** `EMAIL: Alert`, poi `SMS: Alert`.
- **C.** `EMAIL: Alert`, poi `EMAIL: Alert`.
- **D.** `SMS: Alert`, poi `EMAIL: Alert`.

2. Risposta aperta

Si vogliono aggiungere canali di notifica, per esempio email e SMS, che inseriscano un prefisso prima del testo da inviare. Il prefisso viene scelto quando si crea il canale: con il prefisso `"[URGENTE] "`, la chiamata `send("Alert")` deve inviare il testo `"[URGENTE] Alert"`. Il prefisso è sempre non nullo e può essere diverso per due oggetti dello stesso tipo.

La preparazione del testo deve essere scritta una sola volta e riutilizzata dai diversi canali; cambia soltanto il modo in cui il testo viene inviato. Non puoi modificare `Alarm` o `Notifier`.

Proponi un'organizzazione del codice indicando:

- dove conservare il prefisso e come fornirlo alla creazione del canale;
- dove preparare il testo completo senza duplicare questa operazione;
- quale metodo deve occuparsi dell'invio specifico per email, SMS o altri canali.

Lunghezza indicativa: uno schema di classi o interfacce di circa 6 righe e una frase di spiegazione. Indica le relazioni tra i tipi e le firme dei metodi, ma non implementare i corpi o gli invii reali. Sono ammesse soluzioni diverse che rispettino i requisiti.

Mostra la soluzione dell'esercizio 7

1. Scelta multipla: B

L'output è `EMAIL: Alert`, poi `SMS: Alert`. Ogni allarme conserva nel proprio campo il riferimento ricevuto alla costruzione: il primo a un'istanza di `Email`, il secondo a un'istanza di `Sms`. Le chiamate a `send` eseguono le rispettive implementazioni.

- **A errata:** riassegnare la variabile locale `channel` non cambia il campo del primo allarme.
- **C errata:** il secondo allarme riceve il riferimento al nuovo oggetto `Sms`, non al precedente oggetto `Email`.
- **D errata:** le stampe seguono le chiamate a `first.fire()` e `second.fire()`, non l'ordine inverso.

2. Risposta aperta

Risposta sufficiente Una possibile risposta è questo schema, accompagnato dalla spiegazione che il prefisso appartiene a ciascun oggetto e la preparazione comune risiede in `send`:

```
PrefixedNotifier: classe astratta che implementa Notifier
- private final String prefix
- protected PrefixedNotifier(String prefix)
- public final void send(String text): prepara il testo e chiama deliver
- protected abstract void deliver(String preparedText)
Canali concreti: estendono PrefixedNotifier e implementano deliver
```

Approfondimento L'implementazione seguente esplicita il funzionamento dello schema; non è richiesta nella risposta dello studente.

```java
abstract class PrefixedNotifier implements Notifier {
    private final String prefix;

    protected PrefixedNotifier(String prefix) {
        this.prefix = prefix;
    }

    @Override
    public final void send(String text) {
        deliver(prefix + text);
    }

    protected abstract void deliver(String preparedText);
}
```

Il campo non è `static`: ogni oggetto può avere un prefisso diverso. `send` contiene la preparazione comune e, essendo `final`, non può essere ridefinito dalle sottoclassi per aggirarla. Un canale concreto implementa soltanto `deliver`, per esempio:

```java
final class PrefixedEmail extends PrefixedNotifier {
    PrefixedEmail(String prefix) {
        super(prefix);
    }

    @Override
    protected void deliver(String preparedText) {
        IO.println("EMAIL: " + preparedText);
    }
}
```

Un `PrefixedEmail("[URGENTE] ")` può essere passato al costruttore di `Alarm` senza modificarlo. Il testo affidato all'invio è `[URGENTE] Alert`; la simulazione sulla console aggiunge anche l'indicazione `EMAIL:`.

Alternativa per composizione È altrettanto valida una classe che implementa `Notifier` e riceve nel costruttore un prefisso e un altro `Notifier`, conservandoli in due campi di istanza. Il suo send prepara il testo e chiama `delegate.send(prefix + text)`: il delegato può essere un'istanza dell'attuale `Email` o `Sms`. Questa descrizione, con le firme essenziali, è sufficiente come risposta alternativa: la preparazione è riutilizzata per composizione invece che per ereditarietà.

Esercizio 8 — Etichette dei componenti

Tempo indicativo: 5–7 minuti · Java 25

Una schermata mostra etichette testuali per diversi componenti di un catalogo. Il codice che costruisce la schermata riceve riferimenti di tipo `Named` e usa `label()` per ottenere il testo da visualizzare, senza esaminare la classe concreta. Il contratto offre un'etichetta generica alle implementazioni che non ne definiscono una propria. La classe `Tag` conserva invece il nome di una voce del catalogo e fornisce il testo usato per quella voce. Il frammento mostra come viene richiesto il testo di un componente prima di stampare la schermata.

Assunzioni: i nomi e i riferimenti sono non nulli; le interfacce non hanno relazioni di ereditarietà ulteriori rispetto a quelle dichiarate.

Le istruzioni dopo le dichiarazioni sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```java
01  interface Named {
02      default String label() {
03          return "generic";
04      }
05  }
06  class Tag implements Named {
07      private final String name;
```

```
08      Tag(String name) {
09          this.name = name;
10      }
11      @Override
12      public String label() {
13          return "tag:" + name;
14      }
15  }
```

```
16  Named component = new Tag("invoice");
17  IO.println(component.label());
```

1. Scelta multipla

Che cosa accade eseguendo le righe 16–17? Seleziona una sola risposta.

- **A.** Stampa `generic`, perché il riferimento ha tipo `Named`.
- **B.** Non compila: un metodo `default` non può essere ridefinito.
- **C.** Stampa `tag:invoice`.
- **D.** Stampa prima `generic` e poi `tag:invoice`.

2. Risposta breve

Considera questa seconda versione, che sostituisce completamente la prima `Tag`. `Named` resta quella definita alle righe 1–5; `Audited` è un'interfaccia distinta, senza relazioni di ereditarietà con `Named`.

```
18  interface Audited {
19      default String label() {
20          return "audit";
21      }
22  }
23  class Tag implements Named, Audited {
24      private final String name;
25      Tag(String name) {
26          this.name = name;
27      }
28  }
```

Valuta la compilazione della seconda versione con la chiamata delle righe 16–17. Se necessario, spiega come intervenire su `Tag` per fare in modo che usi il comportamento definito da `Named`, senza cambiare le interfacce. Puoi descrivere la modifica a parole: non è necessario ricordare la sintassi esatta della chiamata al metodo dell'interfaccia.

Lunghezza indicativa: 2–3 frasi, oppure 1–2 frasi e poche righe di codice.

Mostra la soluzione dell'esercizio 8

1. Scelta multipla: C

L'output è `tag:invoice`. L'oggetto è un `Tag` e il suo metodo `label` ridefinisce quello dell'interfaccia.

- **A errata:** il tipo del riferimento non impone di eseguire l'implementazione `default`.
- **B errata:** i metodi `default` possono essere ridefiniti dalle classi che implementano l'interfaccia.
- **D errata:** viene eseguito un solo metodo; non c'è una chiamata automatica anche al metodo dell'interfaccia.

2. Risposta breve

Risposta sufficiente La seconda versione non compila perché eredita due metodi `default` con la stessa firma da interfacce non imparentate. `Tag` deve dichiarare un proprio metodo pubblico `label` che risolva il conflitto richiamando esplicitamente l'implementazione di `Named`.

Questa spiegazione è sufficiente anche senza ricordare la sintassi della chiamata.

Approfondimento Il metodo da aggiungere alla seconda versione di `Tag` è:

```
@Override
public String label() {
    return Named.super.label();
}
```

La chiamata delle righe 16–17 stampa ora `generic`. Il campo `name` non viene usato da questa versione del metodo.

Restituire direttamente `"generic"` riprodurrebbe l'output attuale, ma non riutilizzerebbe il comportamento dell'interfaccia: `Named.super.label()` esprime la scelta richiesta senza duplicarne l'implementazione.

Esercizio 9 — Costo di un piano

Tempo indicativo: 5–7 minuti · Java 25

Un servizio calcola il costo di utilizzo di diverse offerte commerciali. Ogni offerta comprende una quota fissa e una parte che dipende dal numero di unità consumate. La classe `Plan` conserva la quota fissa e definisce il punto di ingresso usato dal codice di fatturazione. Le implementazioni concrete determinano il contributo variabile attraverso `extra`, mentre il chiamante richiede il totale senza conoscere la formula specifica. L'offerta `Metered` usa un prezzo unitario scelto alla costruzione e permette quindi di rappresentare più tariffe con la stessa classe.

Requisiti e assunzioni:

- Un numero di unità negativo deve essere rifiutato con `IllegalArgumentException`; `extra` deve ricevere soltanto unità valide.
- Quote fisse, prezzi unitari e contributi variabili sono non negativi; i costruttori ricevono valori validi.
- Tutti i valori sono piccoli e non producono overflow. Non sono richiesti sconti, arrotondamenti o valute diverse.

Le istruzioni dopo le classi sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  abstract class Plan {
02      private final int base;
03      Plan(int base) {
04          this.base = base;
05      }
06      public final int total(int units) {
07          if (units < 0) throw new IllegalArgumentException();
08          return base + extra(units);
09      }
10      protected abstract int extra(int units);
11  }
12  class Metered extends Plan {
13      private final int unitPrice;
14      Metered(int unitPrice) {
15          super(2);
16          this.unitPrice = unitPrice;
17      }
18      @Override
19      protected int extra(int units) {
20          return unitPrice * units;
21      }
22  }
```

```
23  Plan plan = new Metered(3);
24  IO.println(plan.total(4));
```

1. Scelta multipla

Che cosa accade eseguendo le righe 23–24? Seleziona una sola risposta.

- **A.** Stampa 14.
- **B.** Stampa 12.
- **C.** Stampa 2.
- **D.** Non compila perché `Plan` è astratta.

2. Risposta breve

Aggiungi un piano gratuito che rispetti lo stesso contratto sugli input: zero per ogni numero di unità valido ed eccezione per un numero negativo. Indica quali membri sono necessari nella

nuova classe e quali puoi riutilizzare senza modificarli. Non cambiare `Plan` o `Metered`.

Lunghezza indicativa: 2–3 frasi oppure uno schema di massimo 5 righe; non occorre una classe completa.

Mostra la soluzione dell'esercizio 9

1. Scelta multipla: A

L'output è 14: la quota fissa è 2, mentre il contributo variabile è `3 × 4 = 12`. `total` esegue il controllo e richiama l'implementazione di `extra` fornita da `Metered`.

- **B errata:** considera soltanto la parte variabile, omettendo la quota fissa.
- **C errata:** considera soltanto la quota fissa, omettendo la parte variabile.
- **D errata:** l'oggetto costruito è un `Metered`, una classe concreta; `Plan` è soltanto il tipo del riferimento.

2. Risposta breve

Risposta sufficiente Definisco una sottoclasse di `Plan` il cui costruttore chiama `super(0)` e che implementa `extra` restituendo zero. Riutilizzo `total` senza modificarlo: somma due contributi nulli per gli input validi e conserva il controllo sui valori negativi. Non servono nuovi campi.

Approfondimento Una possibile implementazione completa è:

```
final class FreePlan extends Plan {
    FreePlan() {
        super(0);
    }

    @Override
    protected int extra(int units) {
        return 0;
    }
}
```

Il metodo `total` resta quello ereditato: restituisce zero per ogni numero di unità valido, ma continua a sollevare `IllegalArgumentException` per un valore negativo, prima di invocare `extra`.

Esercizio 10 — Campioni di una rilevazione

Tempo indicativo: 5–7 minuti · Java 25

Uno strumento produce rilevazioni composte da campioni interi. `Reading` riceve i campioni dal chiamante, che può continuare a usare e modificare il proprio array. Il record espone i dati per la consultazione attraverso l'accessor generato. Nella versione mostrata si considera soltanto l'accesso ai campioni, non l'uguaglianza tra rilevazioni. L'array è non nullo, anche eventualmente vuoto.

Le istruzioni dopo il record sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  record Reading(int[] samples) {
02      Reading {
03          samples = samples.clone();
04      }
05  }
```

```
06  int[] source = {4, 7};
07  Reading reading = new Reading(source);
08  source[0] = 9;
09  IO.println(reading.samples()[0]);
```

1. Scelta multipla

Che cosa accade eseguendo le righe 6–9? Seleziona una sola risposta.

- **A.** Stampa 9.
- **B.** Non compila perché un record non può contenere un array.
- **C.** Stampa 4.
- **D.** Solleva un'eccezione alla riga 8 perché il componente è `final`.

2. Risposta aperta

Una nuova versione deve essere immutabile e considerare uguali due rilevazioni con la stessa sequenza di valori, anche quando costruite da array distinti. Il chiamante può continuare a modificare l'array passato al costruttore. Puoi cambiare tipo del componente e API pubblica.

Scegli una rappresentazione interna e descrivi come costruzione, lettura, `equals` e `hashCode` soddisferanno i requisiti. Non implementare i metodi: motiva le scelte ed evita modifiche non necessarie.

Lunghezza indicativa: 3 frasi oppure uno schema di massimo 5 righe.

Mostra la soluzione dell'esercizio 10

1. Scelta multipla: C

Il costruttore conserva un clone: la modifica di `source` non raggiunge quell'array e si stampa 4.

- **A errata:** la copia in ingresso separa i due array.
- **B errata:** un componente di un record può essere un array.
- **D errata:** `final` non rende immutabile il contenuto di un array.

2. Risposta aperta

Una possibilità è un record con componente `List<Integer>`, costruita copiando i valori dell'array e poi resa non modificabile con `List.copyOf`. La lettura può esporre tale lista perché gli elementi `Integer` sono immutabili; i metodi generati del record usano l'uguaglianza e l'hash della lista, basati sulla sequenza dei valori. Le successive modifiche all'array del chiamante non incidono sul record.

Alternativa valida: conservare l'array con copie difensive sia in ingresso sia nell'accessor e ridefinire equals/hashCode tramite `Arrays.equals`/`Arrays.hashCode`. Il solo clone nel costruttore non basta: l'accessor generato espone l'array interno e l'uguaglianza predefinita degli array è per identità.

Esercizio 11 — Indicatore di un contatore

Tempo indicativo: 5–7 minuti · Java 25

Una schermata ottiene un indicatore da un contatore. Il contatore può ricevere nuovi valori dopo che l'indicatore è stato creato. La schermata usa `read` per consultare il valore esposto dall'indicatore. La versione mostrata usa una classe interna per organizzare i due componenti. Tutte le operazioni avvengono nello stesso thread e i valori sono piccoli interi.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Counter {
02      private int value;
03      void set(int value) { this.value = value; }
04      class Indicator {
05          int read() { return value; }
06      }
07      Indicator indicator() { return new Indicator(); }
08  }
```

```
09  Counter counter = new Counter();
10  counter.set(4);
11  Counter.Indicator indicator = counter.indicator();
12  counter.set(7);
13  IO.println(indicator.read());
```

1. Scelta multipla

Che cosa accade eseguendo le righe 9–13? Seleziona una sola risposta.

- **A.** Stampa 4.
- **B.** Stampa 0.
- **C.** Non compila perché `Indicator` legge un campo privato di `Counter`.
- **D.** Stampa 7.

2. Risposta aperta

Per una nuova versione, l'indicatore deve conservare soltanto il valore numerico esistente al momento della creazione, senza cambiare in seguito e senza mantenere un riferimento al contatore. Deve rimanere un tipo annidato in `Counter`.

Scegli la forma della classe annidata, lo stato necessario e il modo in cui il valore le verrà fornito. Mostra le dichiarazioni essenziali e descrivi l'adeguamento di `indicator()`; non riscrivere `Counter`.

Lunghezza indicativa: uno schema di massimo 5 righe e una frase.

Mostra la soluzione dell'esercizio 11

1. Scelta multipla: D

L'indicatore interno legge il campo del contatore a cui è associato; dopo `set(7)` legge 7.

- **A errata:** il costruttore implicito non fotografa il valore 4.
- **B errata:** il campo è già stato aggiornato.
- **C errata:** la classe annidata può accedere ai membri privati dell'esterna.

2. Risposta aperta

```
static final class Indicator {
    private final int snapshot;
    Indicator(int snapshot) { this.snapshot = snapshot; }
    int read() { return snapshot; }
}
```

`indicator()` restituisce `new Indicator(value)`: la classe annidata statica non ha un riferimento implicito al contatore e conserva solo il valore ricevuto. È valida anche una dichiarazione di record annidato con componente numerico e metodo `read` equivalente.

Esercizio 12 — Stato di una lavorazione

Tempo indicativo: 5–7 minuti · Java 25

Un laboratorio tiene traccia dello stato di una lavorazione. Ogni nuova istanza parte da `NEW` e permette di leggere lo stato corrente. Nella versione mostrata il chiamante assegna direttamente uno degli stati attraverso `setState`. L'unico controllo del setter attuale riguarda la non nullità. Il programma è monothread e non sono richiesti altri stati oltre a quelli elencati.

Le istruzioni dopo le dichiarazioni sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  enum State { NEW, RUNNING, DONE }
02  final class Job {
03      private State state = State.NEW;
04      public State state() { return state; }
05      public void setState(State next) {
06          if (next == null) throw new IllegalArgumentException();
07          state = next;
08      }
09  }
```

```
10  Job job = new Job();
11  job.setState(State.DONE);
12  IO.println(job.state() == State.DONE);
```

1. Scelta multipla

Che cosa accade eseguendo le righe 10–12? Seleziona una sola risposta.

- **A.** Non compila: gli enum devono essere confrontati soltanto con `equals`.
- **B.** Stampa `true`.
- **C.** Stampa `false`, perché ogni accesso a `State.DONE` crea un nuovo oggetto.
- **D.** Solleva un'eccezione perché non è stato attraversato lo stato `RUNNING`.

2. Risposta aperta

Una nuova versione deve consentire soltanto le transizioni `NEW → RUNNING → DONE`; ogni altra transizione deve essere rifiutata con `IllegalStateException` lasciando invariato lo stato. Le nuove istanze devono partire sempre da `NEW`.

Proponi l'API pubblica e le condizioni da verificare, indicando che cosa cambieresti nell'attuale setter. Non implementare i corpi; la lettura dello stato deve restare possibile.

Lunghezza indicativa: massimo 3 firme di metodi e 2 frasi.

Mostra la soluzione dell'esercizio 12

1. Scelta multipla: B

Il setter accetta `DONE`, che non è nullo; il confronto tra la costante enum letta e `State.DONE` vale `true`.

- **A errata:** == è appropriato per le costanti enum.
- **C errata:** un accesso a una costante enum non crea una nuova istanza.
- **D errata:** il setter corrente non impone un percorso tra gli stati.

2. Risposta aperta

Conservo `public State state()` e sostituisco il setter pubblico con `public void start()` e `public void finish()`. `start` verifica `NEW` prima di assegnare `RUNNING`; `finish` verifica `RUNNING` prima di assegnare `DONE`, altrimenti sollevano `IllegalStateException` senza assegnazioni.

Alternativa valida: mantenere `setState(State next)` validando esattamente le due coppie di transizione ammesse prima di aggiornare il campo e rifiutando ogni altra richiesta, incluso `null`, con l'eccezione prevista.

Esercizio 13 — Calcolo delle spese

Tempo indicativo: 5–7 minuti · Java 25

Un negozio calcola il totale di un ordine sommando subtotale e spese di spedizione. `Checkout` riceve il nome della modalità scelta dal cliente. La versione mostrata riconosce due modalità, ciascuna con una spesa fissa. Le modalità non riconosciute devono essere segnalate invece di applicare una tariffa arbitraria. Subtotali e risultati sono piccoli interi non negativi, senza overflow; `shipping` è non nullo.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Checkout {
02      int total(int subtotal, String shipping) {
03          int fee;
04          switch (shipping) {
05              case "standard" -> fee = 5;
06              case "express" -> fee = 9;
07              default -> throw new IllegalArgumentException();
08          }
09          return subtotal + fee;
10      }
11  }
```

```
12  Checkout checkout = new Checkout();
13  IO.println(checkout.total(20, "express"));
```

1. Scelta multipla

Che cosa accade sostituendo soltanto `"express"` alla riga 13 con `"pickup"`? Seleziona una sola risposta.

- **A.** Stampa `20`.
- **B.** Stampa `25`.
- **C.** Non compila perché manca un ramo dello `switch`.
- **D.** Solleva `IllegalArgumentException` senza stampare un totale.

2. Risposta aperta

Sono previste nuove regole di spedizione, anche dipendenti dal subtotale, che devono poter essere aggiunte senza modificare nuovamente `Checkout`. Il chiamante deve poter scegliere la regola per ogni calcolo. Si assume che ogni regola ricevuta rispetti il contratto di restituire una spesa non negativa.

Proponi una modifica al contratto di `total` e le sole dichiarazioni di supporto indispensabili. Spiega dove risiederanno le nuove regole. Non implementare regole concrete e non limitarti a nominare un principio SOLID.

Lunghezza indicativa: massimo 6 righe di codice e una frase.

Mostra la soluzione dell’esercizio 13

1. Scelta multipla: D

`pickup` seleziona il ramo `default`, che solleva l’eccezione prima che `println` riceva un risultato.

- **A errata:** non è prevista una tariffa zero per il ritiro.
- **B errata:** `pickup` non coincide con `standard`.
- **C errata:** il ramo `default` gestisce i valori non elencati.

2. Risposta aperta

```
interface ShippingRule { int fee(int subtotal); }
int total(int subtotal, ShippingRule shipping) {
    return subtotal + shipping.fee(subtotal);
}
```

Le nuove regole risiedono in implementazioni di `ShippingRule`, scelte dal chiamante per ogni calcolo; `Checkout` dipende dal contratto, non dai loro nomi. È valida anche un'interfaccia funzionale standard appropriata, se nel contratto viene mantenuto il vincolo di spesa non negativa.

Esercizio 14 — Operazioni su un conto

Tempo indicativo: 5–7 minuti · Java 25

Un servizio bancario consulta saldi ed esegue prelievi. Il contratto di `Account.withdraw` richiede che un prelievo positivo non superiore al saldo abbia successo e riduca il saldo della quantità richiesta. Una versione del progetto introduce anche conti bloccati. Il frammento mostra la relazione attuale tra i due tipi di conto. Considera soltanto prelievi che rispettano la precondizione indicata e un programma monothread.

Le istruzioni dopo le classi sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Account {
02      private int balance = 10;
03      public int balance() { return balance; }
04      public void withdraw(int amount) { balance -= amount; }
05  }
06  class LockedAccount extends Account {
07      @Override public void withdraw(int amount) {
08          throw new UnsupportedOperationException();
09      }
10  }
```

```
11  Account account = new LockedAccount();
12  IO.println(account.balance());
```

1. Scelta multipla

Che cosa accade eseguendo le righe 11–12? Seleziona una sola risposta.

- **A.** Stampa `0`.

- **B.** Solleva `UnsupportedOperationException`.
- **C.** Stampa `10`.
- **D.** Non compila perché la sottoclasse non ridefinisce `balance`.

2. Risposta aperta

Il progetto deve supportare sia conti prelevabili sia conti bloccati: entrambi permettono di leggere il saldo, ma soltanto i primi possono essere passati a un servizio di prelievo. Valuta la gerarchia attuale rispetto al contratto e proponi tipi o ruoli che esprimano la distinzione a compilazione, senza operazioni fittizie o cast nel servizio.

Lunghezza indicativa: uno schema di massimo 5 righe con le firme essenziali e 1–2 frasi. Non implementare i conti.

Mostra la soluzione dell'esercizio 14

1. Scelta multipla: C

Viene letto il saldo iniziale `10`; il frammento non chiama `withdraw`.

- **A errata:** la sottoclasse non azzera il saldo.
- **B errata:** il metodo che solleva l'eccezione non viene invocato.
- **D errata:** il metodo `balance` viene ereditato.

2. Risposta aperta

La gerarchia viola il contratto: un `LockedAccount` rifiuta anche un prelievo valido, come `withdraw(1)` con saldo `10`. Si possono distinguere i ruoli:

```
interface BalanceView { int balance(); }
interface Withdrawable extends BalanceView { void withdraw(int amount); }
// Conto bloccato: implementa BalanceView.
// Conto prelevabile: implementa Withdrawable.
void processWithdrawal(Withdrawable account, int amount);
```

Il servizio riceve solo conti che promettono davvero il prelievo. Sono valide altre gerarchie o composizioni che mantengano questa distinzione statica.

Esercizio 15 — Dispositivi da ufficio

Tempo indicativo: 5–7 minuti · Java 25

Un ufficio usa dispositivi per stampare e acquisire documenti. Alcuni dispositivi offrono soltanto una delle due capacità. L'interfaccia corrente le raggruppa entrambe, mentre l'implementazione mostrata sa solo stampare. Il servizio di stampa riceve un testo e lo invia al dispositivo scelto. Testi e dispositivi sono non nulli e la stampa simulata termina normalmente.

Le istruzioni dopo le dichiarazioni sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  interface Device {
02      void print(String text);
03      String scan();
04  }
05  class PrintOnly implements Device {
06      public void print(String text) { IO.println(text); }
07      public String scan() { throw new UnsupportedOperationException(); }
08  }
```

```
09  Device device = new PrintOnly();
10  device.print("Report");
```

1. Scelta multipla

Che cosa accade eseguendo le righe 9–10? Seleziona una sola risposta.

- **A.** Stampa `Report`.
- **B.** Solleva `UnsupportedOperationException`.
- **C.** Non compila perché `scan` non restituisce una stringa.
- **D.** Non compila perché una classe deve avere le stesse capacità reali dell'interfaccia.

2. Risposta aperta

Devono essere supportati anche scanner privi di stampa e dispositivi che offrono entrambe le capacità. Un servizio di acquisizione deve dipendere solo da ciò che gli serve, senza costringere uno scanner a implementare una stampa fittizia.

Proponi le interfacce necessarie e la firma di un metodo del servizio che riceva il dispositivo appropriato. Indica come un dispositivo multifunzione parteciperà ai contratti. Non implementare dispositivi o servizi.

Lunghezza indicativa: massimo 5 righe di dichiarazioni e una frase.

Mostra la soluzione dell'esercizio 15

1. Scelta multipla: A

La chiamata a `print` stampa `Report`; `scan` non viene invocato.

- **B errata:** l'eccezione appartiene a un altro metodo.
- **C errata:** un metodo che termina sempre con `throw` non deve raggiungere un `return`.
- **D errata:** il compilatore controlla dichiarazioni e corpi, non la capacità fisica di acquisire documenti.

2. Risposta aperta

```
interface Printer { void print(String text); }
interface ScannerDevice { String scan(); }
String acquire(ScannerDevice device);
```

Un dispositivo multifunzione implementa entrambe le interfacce; il servizio di acquisizione dipende solo da `ScannerDevice`. È valida anche un'interfaccia aggregata che estenda le due capacità, senza imporla ai dispositivi monofunzione.

Esercizio 16 — Pulsante di conteggio

Tempo indicativo: 5–7 minuti · Java 25

Un dispositivo conta le pressioni di un pulsante mediante un oggetto `ClickCounter`. Il contatore parte da zero, `click` aggiunge un evento e `reset` azzera il valore. L'operazione undo annulla un evento se presente; quando il valore è zero deve lasciarlo invariato. Il valore del contatore non deve mai diventare negativo.

Si considerano soltanto sequenze brevi, senza overflow e senza concorrenza. Il frammento appartiene a una classe di test, con `ClickCounter` come classe annidata statica; gli import sono omessi. `@Test` identifica un test e `assertEquals(atteso, effettivo)` confronta i valori. I numeri di riga non fanno parte del codice.

```
01  static class ClickCounter {
02      private int value;
03      int value() { return value; }
04      void click() { value++; }
05      void reset() { value = 0; }
06      void undo() {
07          if (value >= 0) {
08              value--;
09          }
10      }
11  }
12
```

```
13  @Test
14  void cancelOneEvent() {
15      ClickCounter counter = new ClickCounter();
16      counter.click();
17      counter.undo();
18      assertEquals(0, counter.value());
19  }
20
21  @Test
22  void clearEvents() {
23      ClickCounter counter = new ClickCounter();
24      counter.click();
25      counter.click();
26      counter.reset();
27      assertEquals(0, counter.value());
28  }
```

1. Scelta multipla

Qual è l'esito dei test mostrati?

- **A.** Passano entrambi.
- **B.** Passa soltanto `cancelOneEvent`.
- **C.** Passa soltanto `clearEvents`.
- **D.** Falliscono entrambi.

2. Risposta aperta

I test forniti bastano a verificare l'invariante per tutte le operazioni ammesse? In **1–3 frasi**, indica un'ulteriore breve sequenza di chiamate, il valore atteso e quello effettivo che motivano la tua valutazione. Non scrivere codice.

Mostra la soluzione dell’esercizio 16

1. Scelta multipla: A

Nel primo test il valore passa da `0` a `1` e poi a `0`. Nel secondo passa da `0` a `2`, poi `reset` lo porta a `0`.

- **B errata:** anche l'azzeramento soddisfa l'asserzione.
- **C errata:** annullare un evento esistente porta correttamente da `1` a `0`.
- **D errata:** nessuno dei due test chiama undo quando il valore è zero.

2. Risposta aperta

No: su un contatore appena creato, una chiamata a undo dovrebbe lasciare `0`, ma produce `-1` perché la condizione accetta anche lo zero. I test mostrano il comportamento di alcune sequenze, ma non verificano la transizione da zero mediante undo.

Alternativa valida: `click`, undo, undo, oppure `click`, `reset`, undo: in entrambi i casi il valore finale atteso è `0`, quello effettivo è `-1`.

Esercizio 17 — Richiesta di prelievo

Tempo indicativo: 5–7 minuti · Java 25

Un conto di pagamento consente prelievi soltanto entro il saldo disponibile. Se l'importo supera il saldo, `withdraw` deve lanciare `IllegalArgumentException` e lasciare invariato il conto. Un prelievo accettato deve invece ridurre il saldo dell'importo richiesto. Il test seguente rappresenta una richiesta rifiutata.

Saldo iniziale e importi sono interi non negativi espressi in centesimi; non vi sono commissioni, overflow o concorrenza. Il frammento appartiene a una classe di test e `Account` è una classe annidata statica. `@Test` identifica un test; `assertThrows(T.class, azione)` passa se l'azione lancia un'eccezione di tipo T o di un suo sottotipo. `assertEquals(atteso, effettivo)` confronta i valori; se un'asserzione fallisce, il test termina. Import omessi; i numeri di riga non fanno parte del codice.

```
01  static class Account {
02      private int balance;
03      Account(int initialBalance) { balance = initialBalance; }
04      int balance() { return balance; }
05      void withdraw(int amount) {
06          balance -= amount;
07          if (balance < 0) {
08              throw new IllegalArgumentException("Insufficient funds");
09          }
10      }
11  }
12
13  @Test
14  void rejectedWithdrawal() {
15      Account account = new Account(100);
16      assertThrows(IllegalArgumentException.class,
17              () -> account.withdraw(150));
18      assertEquals(100, account.balance());
19  }
```

1. Scelta multipla

Quale descrizione dell'esecuzione del test è corretta?

- **A.** Entrambe le asserzioni passano: l'eccezione annulla automaticamente le modifiche.
- **B.** `assertThrows` fallisce perché `withdraw` non dichiara `throws`.
- **C.** `assertThrows` passa; la seconda asserzione non viene eseguita perché l'eccezione esce dal test.
- **D.** `assertThrows` passa; `assertEquals` fallisce perché il saldo è `-50`.

2. Risposta aperta

Un revisore propone di conservare soltanto `assertThrows` nel test e di verificare il saldo solo nei test dei prelievi accettati. Valuta la proposta rispetto al contratto, in **1–3 frasi**, senza correggere il codice.

Mostra la soluzione dell'esercizio 17

1. Scelta multipla: D

Il metodo sottrae prima di verificare il saldo: ottiene `-50` e poi lancia l'eccezione attesa. `assertThrows` la intercetta, quindi l'esecuzione prosegue e il confronto con `100` fallisce.

- **A errata:** un'eccezione non esegue un rollback automatico delle assegnazioni.
- **B errata:** `IllegalArgumentException` è unchecked e non deve essere dichiarata nella firma.
- **C errata:** l'eccezione attesa è gestita da `assertThrows`, non abbandona il metodo di test.

2. Risposta aperta

La proposta elimina la verifica di una parte esplicita del contratto: il conto deve restare invariato dopo un rifiuto. I test dei prelievi accettati non verificano questo percorso; occorre osservare il saldo anche dopo l'operazione che lancia l'eccezione.

Esercizio 18 — Contatti con lo stesso cognome

Tempo indicativo: 5–7 minuti · Java 25

Una rubrica deve restituire tutti i contatti con il cognome richiesto, nell'ordine in cui sono memorizzati. I contatti possono avere lo stesso cognome ma nomi diversi. Una ricerca senza corrispondenze deve restituire una lista vuota. Il team vuole usare i test del contratto anche per una futura implementazione della rubrica basata su array.

Rubrica, contatti e stringhe sono non nulli; il confronto distingue maiuscole e minuscole. Il frammento appartiene a una classe di test, con `Directory` come classe annidata statica. Gli import sono omessi. `@Test` identifica un test; `assertEquals(atteso, effettivo)` confronta le liste per contenuto e ordine, usando `equals` sui contatti. Il record fornisce l'uguaglianza per valori. I numeri di riga non fanno parte del codice.

```
01  record Person(String name, String lastname) {}
02  static class Directory {
03      private final List<Person> people;
04      Directory(List<Person> people) { this.people = people; }
05      List<Person> find(String lastname) {
06          List<Person> found = new ArrayList<>();
07          for (Person person : people) {
08              if (person.lastname().equals(lastname)) {
09                  found.add(person);
10                  break;
11              }
12          }
13          return found;
14      }
15  }
16
17  @Test
18  void searchContact() {
19      Person ada = new Person("Ada", "Rossi");
20      Directory book = new Directory(List.of(ada));
21      assertEquals(List.of(ada), book.find("Rossi"));
22      assertEquals(List.of(), book.find("Verdi"));
23  }
24
25  @Test
26  void searchSharedLastname() {
27      Person ada = new Person("Ada", "Rossi");
28      Person lino = new Person("Lino", "Rossi");
29      Directory book = new Directory(List.of(ada, lino));
30      assertEquals(List.of(ada, lino), book.find("Rossi"));
31  }
```

1. Scelta multipla

Qual è l'esito dei due test?

- **A.** Passano entrambi perché i due contatti hanno lo stesso cognome.
- **B.** Passa soltanto `searchContact`.
- **C.** Passa soltanto `searchSharedLastname`.
- **D.** Falliscono entrambi perché le liste vengono confrontate per identità.

2. Risposta aperta

Per la futura implementazione basata su array, quali parti di questi test devono rimanere indipendenti dalla rappresentazione interna? Rispondi in **1–3 frasi**, distinguendo preparazione della rubrica e comportamento verificato, senza implementare classi o test.

Mostra la soluzione dell'esercizio 18

1. Scelta multipla: B

Il primo test verifica una corrispondenza e un'assenza, entrambe gestite come richiesto. Nel secondo, il `break` interrompe la ricerca dopo ada: la lista effettiva è `[ada]`, non `[ada, lino]`.

- **A errata:** il contratto richiede tutti i contatti, non un rappresentante per cognome.
- **C errata:** la seconda ricerca perde il secondo contatto, mentre il primo test passa.
- **D errata:** `List.equals` confronta contenuto e ordine; non richiede che le liste siano lo stesso oggetto.

2. Risposta aperta

Devono rimanere gli stessi dati logici e le stesse aspettative: tutti i contatti corrispondenti nello stesso ordine, oppure una lista vuota. Può cambiare il modo di costruire e popolare la rubrica, ma le asserzioni devono usare l'operazione pubblica di ricerca, non ispezionare array o liste interne; una comune interfaccia consentirebbe di riutilizzare le verifiche.

E4 - Java Collections

Esercizio 1 — Destinatari di un avviso

Tempo indicativo: 5–7 minuti · Java 25

Un servizio prepara i destinatari di un avviso a partire da una lista di iscrizioni. La stessa persona può comparire più volte perché iscritta a canali diversi. La prima versione copia le iscrizioni e rimuove un destinatario prima dell'invio. La lista originale deve restare disponibile per altre elaborazioni. I nomi sono non nulli e l'uguaglianza è sensibile a maiuscole e minuscole.

Il frammento è eseguito in un metodo con gli import indicati; i numeri di riga non fanno parte del codice.

```
01  import java.util.ArrayList;
02  import java.util.List;
```

```
03  List<String> input = List.of("Ada", "Linus", "Ada", "Grace");
04  List<String> recipients = new ArrayList<>(input);
05  recipients.remove("Ada");
06  IO.println(recipients);
07  IO.println(input.size());
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le due righe nell'ordine.

- **A.** `[Linus, Ada, Grace]`, poi 4.
- **B.** `[Linus, Grace]`, poi 4.
- **C.** `[Linus, Ada, Grace]`, poi 3.
- **D.** Solleva `UnsupportedOperationException` prima di stampare.

2. Risposta aperta

Per una nuova elaborazione, riparti dalla lista `input` originale: ogni destinatario deve comparire una sola volta, nell'ordine della prima comparsa. Il risultato deve essere indipendente da `input`; non sono richieste rimozioni.

Scegli tipo dichiarato e implementazione della collezione risultante e mostra come costruirla. Motiva come la scelta garantisca entrambi i requisiti, senza basarti sul particolare ordine dei quattro nomi dell'esempio.

Lunghezza indicativa: massimo 2 righe di codice e 2 frasi. Puoi omettere gli import aggiuntivi.

Mostra la soluzione dell'esercizio 1

1. Scelta multipla: A

`remove("Ada")` elimina solo la prima occorrenza nella copia: resta `[Linus, Ada, Grace]`. L'originale conserva quattro elementi.

- **B errata:** non vengono eliminate tutte le occorrenze.
- **C errata:** la rimozione non cambia la lista originale.
- **D errata:** `ArrayList` è modificabile anche se costruita da `List.of`.

2. Risposta aperta

```
Set<String> unique = new LinkedHashSet<>(input);
```

Il set elimina duplicati e `LinkedHashSet` conserva l'ordine di prima inserzione. È un contenitore distinto da `input`; le stringhe condivise sono immutabili. È valida anche una `SequencedSet<String>` come tipo dichiarato.

Esercizio 2 — Visite a una pagina

Tempo indicativo: 5–7 minuti · Java 25

Un sito mantiene un riepilogo delle visite per pagina. Una mappa associa il nome di ogni pagina al conteggio corrente. `visit` deve incrementare il conteggio oppure creare una voce pari a 1 per una pagina nuova. Le pagine già presenti non devono essere duplicate come chiavi. Mappa e nomi sono non nulli, i conteggi sono piccoli interi non negativi e mai nulli; il programma è monothread e senza overflow.

Il metodo va dichiarato nel corpo di una classe e le istruzioni in un suo metodo statico, con `java.util.Map` e `java.util.HashMap` importate. `put` restituisce il valore precedente associato alla chiave, oppure `null` se non esisteva. I numeri di riga non fanno parte del codice.

```
01  static void visit(Map<String, Integer> counts, String page) {
02      counts.put(page, counts.get(page) + 1);
03  }
```

```
04  Map<String, Integer> counts = new HashMap<>();
05  IO.println(counts.put("home", 1));
06  IO.println(counts.put("home", 2));
07  IO.println(counts.size());
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le tre righe nell'ordine.

- **A.** 1, poi 2, poi 2.
- **B.** `null`, poi 1, poi 1.
- **C.** `null`, poi 1, poi 2.
- **D.** 1, poi 2, poi 1.

2. Risposta breve

Considera separatamente una mappa inizialmente vuota e due chiamate consecutive `visit(counts, "home")`. Il metodo soddisfa il contratto? Se necessario, correggilo indicando il conteggio che deve risultare dopo entrambe le chiamate.

Lunghezza indicativa: 1–2 frasi e massimo 3 righe per il corpo del metodo.

Mostra la soluzione dell'esercizio 2

1. Scelta multipla: B

Il primo `put` non sostituisce valori e restituisce `null`; il secondo restituisce il precedente 1. La chiave rimane una sola.

- **A errata:** `put` restituisce il valore precedente, non quello inserito.
- **C errata:** sostituire un valore non aggiunge una chiave.
- **D errata:** i primi due risultati non sono i nuovi valori.

2. Risposta breve

La prima chiamata fallisce durante l'unboxing di `null` restituito da `get`. Il corpo può diventare:

```
counts.put(page, counts.getOrDefault(page, 0) + 1);
```

Dopo due chiamate il conteggio è 2. È valido anche un controllo esplicito dell'assenza prima dell'incremento, oppure `merge(page, 1, Integer::sum)`.

Esercizio 3 — Gestione delle richieste

Tempo indicativo: 5–7 minuti · Java 25

Uno sportello riceve richieste identificate da un nome. La prima versione deve servirle nell'ordine di arrivo. La consultazione della prossima richiesta non deve estrarla, mentre il servizio effettivo deve rimuoverla. Il frammento simula arrivi e servizio senza altri componenti esterni. I nomi sono non nulli e tutte le operazioni avvengono nello stesso thread.

Il frammento è eseguito in un metodo con `java.util.Queue` e `java.util.ArrayDeque` importate. I numeri di riga non fanno parte del codice.

```
01  Queue<String> pending = new ArrayDeque<>();
02  pending.offer("A");
03  pending.offer("B");
04  IO.println(pending.poll());
05  pending.offer("C");
06  IO.println(pending.peek());
07  IO.println(pending.size());
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le tre righe nell'ordine.

- **A.** B, poi C, poi 2.
- **B.** A, poi B, poi 1.
- **C.** A, poi C, poi 2.
- **D.** A, poi B, poi 2.

2. Risposta aperta

La nuova versione distingue richieste urgenti e normali. Alla scelta della prossima richiesta devono essere servite prima le urgenti; all'interno di ciascuna categoria va rispettato l'ordine di arrivo. Una richiesta normale può quindi attendere mentre arrivano nuove urgenti.

Scegli una rappresentazione con collezioni standard e descrivi inserimento ed estrazione. Specifica il risultato dell'estrazione quando non esistono richieste. Non serve implementare una classe completa.

Lunghezza indicativa: le dichiarazioni dei campi necessari e 3 frasi, oppure uno schema equivalente di massimo 6 righe.

Mostra la soluzione dell'esercizio 3

1. Scelta multipla: D

`poll` estrae A; peek legge B senza rimuoverlo. Dopo l'inserimento di C restano due richieste.

- **A errata:** una coda FIFO serve prima A.
- **B errata:** peek non riduce la dimensione.
- **C errata:** B precede C nella coda.

2. Risposta aperta

```
Queue<String> urgent = new ArrayDeque<>();
Queue<String> normal = new ArrayDeque<>();
```

Inserisco con `offer` nella coda della categoria. Estraggo con `poll` dalle urgenti se non vuote, altrimenti dalle normali, ottenendo `null` se entrambe sono vuote. Le due code mantengono il FIFO interno e permettono che nuove urgenti precedano normali in attesa.

Alternativa valida: una coda a priorità con categoria e numero progressivo come criteri di ordinamento; la sola categoria non garantisce il FIFO tra pari priorità.

Esercizio 4 — Pulizia di una lista

Tempo indicativo: 5–7 minuti · Java 25

Un catalogo conserva le quantità disponibili in una lista modificabile. Le quantità pari a zero devono essere rimosse prima della stampa. Il chiamante conserva la lista e deve osservare la pulizia sullo stesso oggetto. I valori sono interi non nulli e non negativi. Il programma è monothread.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  List<Integer> quantities = new ArrayList<>(List.of(0, 2, 0, 3));
02  Iterator<Integer> cursor = quantities.iterator();
03  while (cursor.hasNext()) {
04      if (cursor.next() == 0) cursor.remove();
05  }
06  IO.println(quantities);
```

1. Scelta multipla

Qual è l'output? Seleziona una sola risposta.

- **A.** `[0, 2, 0, 3]`.
- **B.** `[2, 0, 3]`.
- **C.** `[2, 3]`.
- **D.** Solleva sempre `ConcurrentModificationException`.

2. Risposta breve

Un revisore propone un ciclo per indici che, quando trova zero, esegue `quantities.remove(i)` e incrementa sempre `i`. Fornisci un input minimo che mostri una quantità zero residua e spiega perché il ciclo con l'iteratore non ha quel difetto.

Lunghezza indicativa: un input e 2 frasi.

Mostra la soluzione dell'esercizio 4

1. Scelta multipla: C

L'iteratore rimuove entrambi gli zeri tramite la propria operazione di rimozione.

- **A errata:** il codice modifica la lista.
- **B errata:** il ciclo visita anche il secondo zero.
- **D errata:** rimuovere tramite lo stesso iteratore è supportato da `ArrayList`.

2. Risposta breve

Con `[0, 0]`, dopo la rimozione all'indice zero, il secondo elemento scivola all'indice zero e viene saltato dall'incremento di `i`. L'iteratore mantiene correttamente la posizione dopo `remove()` e visita il successivo elemento rimasto.

Esercizio 5 — Elenco pubblico dei turni

Tempo indicativo: 5–7 minuti · Java 25

Un servizio conserva i turni in una lista interna. Vuole esporre i nomi senza permettere ai lettori di modificarli. La versione mostrata costruisce una vista non modificabile e una copia. Dopo la pubblicazione il servizio può aggiungere nuovi turni alla lista interna. Tutte le operazioni sono nello stesso thread e i nomi sono stringhe non nulle.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  List<String> internal = new ArrayList<>(List.of("Ada"));
02  List<String> view = Collections.unmodifiableList(internal);
03  List<String> snapshot = List.copyOf(internal);
04  internal.add("Linus");
05  IO.println(view.size());
06  IO.println(snapshot.size());
```

1. Scelta multipla

Seleziona le due righe di output nell'ordine.

- **A.** 1, poi 1.
- **B.** 2, poi 1.
- **C.** 2, poi 2.
- **D.** Eccezione alla riga 4.

2. Risposta aperta

L'API deve pubblicare una fotografia non modificabile dei turni al momento della chiamata, non una vista aggiornata. Proponi il corpo di `List<String> turns()` assumendo che `internal` sia il campo della classe e motiva perché basta per questi elementi.

Lunghezza indicativa: una riga di codice e 2 frasi.

Mostra la soluzione dell'esercizio 5

1. Scelta multipla: B

La vista segue la lista interna, mentre la copia mantiene il contenuto iniziale.

- **A errata:** la vista riflette l'aggiunta.
- **C errata:** la copia non è collegata alle modifiche successive.
- **D errata:** la lista interna resta modificabile.

2. Risposta aperta

```
return List.copyOf(internal);
```

Il risultato non permette modifiche e non riflette i successivi aggiornamenti di `internal`. La condivisione dei riferimenti agli elementi non espone stato modificabile perché `String` è immutabile. È valida una vista non modificabile su una nuova `ArrayList` copiata, non direttamente sul campo.

Esercizio 6 — Graduatoria dei risultati

Tempo indicativo: 5–7 minuti · Java 25

Una gara mantiene una graduatoria con i punteggi degli iscritti. Due iscritti possono avere lo stesso punteggio. Il componente mostrato usa un insieme ordinato per memorizzare i risultati. La stampa serve a controllare ordinamento e numero di elementi. I punteggi sono interi e non ci sono operazioni concorrenti.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  Set<Integer> scores = new TreeSet<>();
02  scores.add(8);
03  scores.add(5);
04  scores.add(8);
05  IO.println(scores);
06  IO.println(scores.size());
```

1. Scelta multipla

Seleziona l'output corretto.

- **A.** [8, 5, 8], poi 3.
- **B.** [8, 5], poi 2.
- **C.** [5, 8, 8], poi 3.
- **D.** [5, 8], poi 2.

2. Risposta aperta

La graduatoria deve conservare una voce per ogni iscritto, anche a parità di punteggio, ordinandole per punteggio decrescente. Scegli una collezione e descrivi come ordinarla; non occorre aggiungere nomi o una nuova classe.

Lunghezza indicativa: massimo 2 righe di codice e 1–2 frasi.

Mostra la soluzione dell'esercizio 6

1. Scelta multipla: D

`TreeSet` ordina i valori e conserva una sola occorrenza per punteggio.

- **A errata:** un set non conserva questi duplicati.
- **B errata:** l'ordine naturale degli interi è crescente, non di inserimento.
- **C errata:** il secondo 8 non viene aggiunto.

2. Risposta aperta

```
List<Integer> ranking = new ArrayList<>(List.of(8, 5, 8));
ranking.sort(Comparator.reverseOrder());
```

La lista mantiene le due occorrenze di 8; il comparatore dà l'ordine decrescente. Sono valide altre liste modificabili ordinabili; un set dei soli punteggi non soddisfa il requisito.

Esercizio 7 — Prossima attività

Tempo indicativo: 5–7 minuti · Java 25

Un pianificatore associa a ogni attività una priorità numerica. In questo esempio le attività sono rappresentate soltanto dalle priorità e il valore minore deve essere servito prima. La coda permette di estrarre ripetutamente la prossima priorità. Possono esserci duplicati e la coda può diventare vuota. Tutte le operazioni avvengono nello stesso thread.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  Queue<Integer> queue = new PriorityQueue<>();
02  queue.offer(4);
03  queue.offer(1);
04  queue.offer(3);
05  IO.println(queue.poll());
06  IO.println(queue.poll());
07  IO.println(queue.peek());
```

1. Scelta multipla

Seleziona l'output corretto.

- **A.** 1, 3, 4.
- **B.** 4, 1, 3.
- **C.** 1, 4, 3.
- **D.** L'ordine delle tre stampe non è garantito.

2. Risposta breve

Un client sostituisce le estrazioni con un ciclo `for` sulla coda e presume di ottenere tutte le priorità in ordine crescente. La garanzia è corretta? Indica come produrre la sequenza ordinata senza consumare la coda originale.

Lunghezza indicativa: 2–3 frasi, senza codice obbligatorio.

Mostra la soluzione dell'esercizio 7

1. Scelta multipla: A

Le estrazioni prendono il minimo: prima 1, poi 3; resta 4 in testa.

- **B errata:** la coda non usa l'ordine di arrivo.
- **C errata:** dopo 1, il minimo residuo è 3.
- **D errata:** l'ordine di estrazione per queste priorità è garantito.

2. Risposta breve

L'iteratore di `PriorityQueue` non garantisce l'ordine crescente. Si può copiare la coda in una nuova `PriorityQueue` ed estrarre con `poll` fino allo svuotamento della copia. In alternativa, si copiano gli elementi in una lista e la si ordina.

Esercizio 8 — Ordine dei contatti

Tempo indicativo: 5–7 minuti · Java 25

Una rubrica associa ogni nome a un numero interno. L'ordine richiesto è quello del primo inserimento del nome. Un aggiornamento del numero non deve spostare il contatto. La rubrica mostrata è monothread e non usa la modalità di ordine per accesso. Chiavi e valori sono stringhe non nulle.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  Map<String, String> contacts = new LinkedHashMap<>();
02  contacts.put("Ada", "10");
03  contacts.put("Linus", "20");
04  contacts.put("Ada", "30");
05  IO.println(contacts.keySet());
06  IO.println(contacts.get("Ada"));
```

1. Scelta multipla

Seleziona l'output corretto.

- **A.** `[Linus, Ada]`, poi `30`.
- **B.** `[Ada, Linus, Ada]`, poi `10`.
- **C.** `[Ada, Linus]`, poi `30`.
- **D.** `[Ada, Linus]`, poi `10`.

2. Risposta breve

Il client riceve `Set<String> names = contacts.keySet();` e chiama `names.remove("Ada")`. Descrivi l'effetto sulla rubrica. Se il requisito è dare una copia modificabile dei soli nomi senza permettere modifiche alla rubrica, proponi una costruzione che conservi l'ordine.

Lunghezza indicativa: 1–2 frasi e una riga di codice.

Mostra la soluzione dell'esercizio 8

1. Scelta multipla: C

L'aggiornamento mantiene la posizione della chiave ma sostituisce il suo valore con `30`.

- **A errata:** l'aggiornamento non reinserisce la chiave in fondo.
- **B errata:** una mappa non duplica una chiave uguale.
- **D errata:** il valore precedente è stato sostituito.

2. Risposta breve

`keySet()` è una vista collegata alla mappa: rimuovere la chiave elimina anche l'associazione. Una copia indipendente e modificabile è:

```
Set<String> names = new LinkedHashSet<>(contacts.keySet());
```

Esercizio 9 — Ricerca nel catalogo

Tempo indicativo: 5–7 minuti · Java 25

Un catalogo conserva codici unici in ordine crescente. Un client cerca un codice mediante `Collections.binarySearch`. Se non lo trova, deve inserirlo mantenendo l'ordinamento. Il confronto usa l'ordine naturale degli interi. Per una ricerca senza successo, l'API restituisce `-(puntoDiInserimento) - 1`.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```java
List<Integer> codes = new ArrayList<>(List.of(10, 30, 50));
int position = Collections.binarySearch(codes, 40);
IO.println(position);
IO.println(codes.size());
```

1. Scelta multipla

Seleziona l'output corretto.

- **A.** 2, poi 4.
- **B.** -3, poi 3.
- **C.** -1, poi 3.
- **D.** 3, poi 3.

2. Risposta breve

Mostra come usare `position` per inserire `40` soltanto se assente. Spiega perché applicare la stessa ricerca a una lista non ordinata non soddisferebbe il requisito generale.

Lunghezza indicativa: una riga di codice e 1–2 frasi.

Mostra la soluzione dell'esercizio 9

1. Scelta multipla: B

Il punto di inserimento è 2, quindi il risultato è `-3`. La ricerca non modifica la dimensione.

- **A errata:** la ricerca non inserisce e restituisce una codifica negativa quando manca il codice.
- **C errata:** `-1` codificherebbe il punto zero.
- **D errata:** non è un indice trovato.

2. Risposta breve

```java
if (position < 0) codes.add(-position - 1, 40);
```

La ricerca binaria richiede l'ordinamento con lo stesso criterio di confronto. Su una lista non ordinata non garantisce né una ricerca corretta né un punto di inserimento utile.

Esercizio 10 — Iscrizioni ai laboratori

Tempo indicativo: 5–7 minuti · Java 25

Un'organizzazione conserva gli iscritti a due laboratori. Ogni laboratorio usa un set di nomi, quindi un nome non si ripete al suo interno. Serve calcolare chi frequenta entrambi senza modificare le iscrizioni originali. La stampa verifica soltanto appartenenza e dimensione, non l'ordine dei set hash. I nomi sono stringhe non nulle e i set sono modificabili.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  Set<String> first = new HashSet<>(Set.of("Ada", "Linus"));
02  Set<String> second = new HashSet<>(Set.of("Ada", "Grace"));
03  Set<String> common = new HashSet<>(first);
04  common.retainAll(second);
05  IO.println(common.contains("Ada"));
06  IO.println(first.size());
```

1. Scelta multipla

Seleziona l'output corretto.

- **A.** `true`, poi 1.
- **B.** `false`, poi 2.
- **C.** `false`, poi 1.
- **D.** `true`, poi 2.

2. Risposta aperta

Una nuova funzione deve invece restituire tutti gli iscritti, senza duplicati, ordinati alfabeticamente e indipendenti dai set ricevuti. Proponi rappresentazione e costruzione del risultato senza modificare gli input.

Lunghezza indicativa: massimo 2 righe di codice e una frase.

Mostra la soluzione dell'esercizio 10

1. Scelta multipla: D

L'intersezione della copia contiene Ada; `first` resta invariato con due elementi.

- **A errata:** viene modificato solo `common`.
- **B errata:** Ada appartiene a entrambi i set.
- **C errata:** sono errate sia l'appartenenza sia la dimensione originale.

2. Risposta aperta

```
Set<String> all = new TreeSet<>(first);
all.addAll(second);
```

TreeSet elimina duplicati e ordina alfabeticamente secondo l'ordine naturale delle stringhe; i contenitori originali non vengono modificati. È valida anche una lista ordinata costruita da un'unione deduplicata.

Esercizio 11 — Indice dei nomi

Tempo indicativo: 5–7 minuti · Java 25

Un servizio inverte una mappa che associa codici numerici a nomi. Il risultato deve contenere ogni nome distinto come chiave, associato a uno dei codici che avevano quel nome nella mappa originale. Quando più codici hanno lo stesso nome, il contratto consente di sceglierne uno qualsiasi. Non è richiesto un particolare ordine e la mappa originale non deve essere modificata.

Chiavi e valori sono non nulli. `putIfAbsent(k, v)` inserisce l'associazione soltanto se la chiave non è già presente. La `LinkedHashMap` del test viene attraversata nell'ordine di inserimento. Il frammento appartiene a una classe di test; import omessi. `@Test` identifica un test e `assertEquals(atteso, effettivo)` confronta le mappe per associazioni, non per ordine. I numeri di riga non fanno parte del codice.

```
01  static Map<String, Integer> invert(Map<Integer, String> source) {
02      Map<String, Integer> result = new HashMap<>();
03      for (Map.Entry<Integer, String> entry : source.entrySet()) {
04          result.putIfAbsent(entry.getValue(), entry.getKey());
05      }
06      return result;
07  }
08
09  @Test
10  void distinctNames() {
11      Map<Integer, String> source = Map.of(1, "Ada", 2, "Lino");
12      assertEquals(Map.of("Ada", 1, "Lino", 2), invert(source));
13  }
14
15  @Test
16  void sharedName() {
17      Map<Integer, String> source = new LinkedHashMap<>();
18      source.put(1, "Ada");
19      source.put(2, "Ada");
20      assertEquals(Map.of("Ada", 2), invert(source));
21  }
```

1. Scelta multipla

Quale valutazione dei test e dell'implementazione è corretta?

- **A.** Entrambi i test passano e verificano anche l'ordine delle mappe.
- **B.** `sharedName` fallisce, dimostrando che il metodo viola il contratto sui duplicati.
- **C.** Entrambi falliscono perché una `HashMap` non è confrontabile con `Map.of`.
- **D.** `distinctNames` passa; `sharedName` fallisce anche se il risultato prodotto è ammesso dal contratto.

2. Risposta aperta

Descrivi in **1–3 frasi** quali proprietà dovrebbe verificare un test del caso con nomi ripetuti, per accettare tutte e sole le associazioni consentite dal contratto. Non scrivere codice.

Mostra la soluzione dell'esercizio 11

1. Scelta multipla: D

Nel primo test ogni nome ha un solo codice e le associazioni coincidono. Nel secondo, `putIfAbsent` conserva 1, mentre il test pretende 2: l'asserzione fallisce, ma entrambe le scelte sono ammesse.

- **A errata:** il secondo test fallisce; inoltre l'uguaglianza delle mappe non confronta l'ordine.
- **B errata:** il contratto non richiede di conservare l'ultimo codice incontrato.
- **C errata:** mappe di implementazioni diverse possono essere uguali se contengono le stesse associazioni.

2. Risposta aperta

Le chiavi del risultato devono essere esattamente i nomi distinti dell'input; per ogni coppia `(nome, codice)` restituita, l'input deve associare proprio quel codice a quel nome. Nel caso mostrato, occorre accettare sia `{"Ada": 1}` sia `{"Ada": 2}`, ma non una mappa vuota, un codice estraneo o nomi aggiuntivi; va inoltre verificato separatamente che l'input resti invariato.

Esercizio 12 — Messaggi degli utenti

Tempo indicativo: 5–7 minuti · Java 25

Un servizio combina gli utenti con i loro messaggi mediante l'identificativo dell'autore. Ogni utente ha un identificativo univoco e può avere zero, uno o più messaggi. Il risultato deve contenere una voce per ogni messaggio di un utente noto, con il nome dell'utente e il testo;

i messaggi di utenti sconosciuti vanno ignorati. L'ordine richiesto è quello degli utenti e, per ciascun utente, quello dei suoi messaggi nell'input.

Liste, elementi e campi sono non nulli; gli input non devono essere modificati. Il frammento appartiene a una classe di test. I record confrontano i propri campi per valore; `Map.put` sostituisce il valore precedente se la chiave esiste già. `@Test` identifica un test e `assertEquals(atteso, effettivo)` confronta le liste per contenuto e ordine. Import omessi; i numeri di riga non fanno parte del codice.

```
01  record User(long id, String name) {}
02  record Post(long userId, String text) {}
03  record UserPost(String name, String text) {}
04
05  static List<UserPost> combine(List<User> users, List<Post> posts) {
06      Map<Long, Post> byUser = new HashMap<>();
07      for (Post post : posts) {
08          byUser.put(post.userId(), post);
09      }
10      List<UserPost> result = new ArrayList<>();
11      for (User user : users) {
12          Post post = byUser.get(user.id());
13          if (post != null) {
14              result.add(new UserPost(user.name(), post.text()));
15          }
16      }
17      return result;
18  }
19
20  @Test
21  void messagesFromKnownUsers() {
22      List<User> users = List.of(new User(1, "Ada"), new User(2, "Lino"));
23      List<Post> posts = List.of(new Post(2, "B"), new Post(1, "A"),
24              new Post(9, "X"));
25      assertEquals(List.of(new UserPost("Ada", "A"),
26              new UserPost("Lino", "B")), combine(users, posts));
27  }
```

1. Scelta multipla

Come termina il test mostrato?

- **A.** Passa: il risultato è `[("Ada", "A"), ("Lino", "B")]`.
- **B.** Fallisce: il risultato segue l'ordine dei messaggi, quindi comincia con Lino.
- **C.** Fallisce: contiene anche il messaggio X dell'utente sconosciuto.
- **D.** Lancia `NullPointerException` quando incontra il messaggio dell'utente 9.

2. Risposta aperta

Valuta se il test è sufficiente rispetto alla molteplicità dei messaggi ammessa dal contratto. In **1–3 frasi**, proponi un piccolo input aggiuntivo e indica risultato atteso e risultato effettivo,

senza scrivere codice.

Mostra la soluzione dell’esercizio 12

1. Scelta multipla: A

La mappa contiene i messaggi indicizzati per autore; il ciclo finale segue la lista degli utenti, prima Ada e poi Lino. L'utente 9 non viene attraversato e il suo messaggio non entra nel risultato.

- **B errata:** l'ordine finale è determinato dal ciclo sugli utenti, non dall'ordine di inserimento nella mappa.
- **C errata:** il risultato viene costruito soltanto per gli utenti noti.
- **D errata:** non viene cercato un utente per ogni messaggio e non si dereferenzia un utente mancante.

2. Risposta aperta

Il test non verifica più messaggi dello stesso autore: con utenti `[(1, "Ada")]` e messaggi `[(1, "A"), (1, "B")]`, il risultato atteso è `[("Ada", "A"), ("Ada", "B")]`, quello effettivo è soltanto `[("Ada", "B")]`. La mappa conserva un solo `Post` per identificativo e il secondo inserimento sostituisce il primo.

Alternativa valida: qualsiasi input con più messaggi di uno stesso utente noto, accompagnato dai risultati corretti e dalla spiegazione della sostituzione.

E5 - Java Exceptions

Esercizio 1 — Importazione di quantità

Tempo indicativo: 5–7 minuti · Java 25

Un magazzino importa quantità da un file già letto in memoria. Ogni riga rappresenta un intero decimale non negativo da aggiungere al totale. Una riga non numerica deve produrre una segnalazione ed essere ignorata. L'errore su una riga non deve impedire l'elaborazione delle successive. Le righe sono non nulle; non considerare numeri negativi o overflow.

Il frammento è eseguito in un metodo. I numeri di riga non fanno parte del codice.

```
01  String[] rows = {"2", "3"};
02  int total = 0;
03  try {
04      for (String row : rows) {
05          total += Integer.parseInt(row);
06      }
07  } catch (NumberFormatException exception) {
08      IO.println("Invalid row");
09  }
10  IO.println(total);
```

1. Scelta multipla

Qual è l'output con l'input mostrato? Seleziona una sola risposta.

- **A.** 23.
- **B.** `Invalid row`, poi `0`.
- **C.** 5.
- **D.** 3.

2. Risposta breve

Valuta il comportamento con `rows = new String[]{"2", "x", "3"}` rispetto al contratto. Se necessario, riscrivi soltanto il ciclo e la gestione dell'eccezione affinché il requisito sia rispettato per ogni riga.

Lunghezza indicativa: una frase e massimo 7 righe di codice.

Mostra la soluzione dell'esercizio 1

1. Scelta multipla: C

Le due conversioni riescono e la somma stampata è 5.

- **A errata:** si sommano interi, non si concatenano stringhe.
- **B errata:** nessuna riga provoca un errore.
- **D errata:** entrambe le quantità vengono sommate.

2. Risposta breve

Con l'input modificato si stampa `Invalid row`, poi 2: il `catch` esterno termina l'elaborazione al primo errore. Per continuare sulle righe successive:

```
for (String row : rows) {
    try {
        total += Integer.parseInt(row);
    } catch (NumberFormatException exception) {
        IO.println("Invalid row");
    }
}
```

Il totale finale diventa 5.

Esercizio 2 — Scrittura di un resoconto

Tempo indicativo: 5–7 minuti · Java 25

Un componente prepara un resoconto usando due risorse chiudibili. Deve rilasciare entrambe anche se il lavoro fallisce. Per osservare il rilascio senza dipendere da file reali, `Trace` stampa il proprio nome quando viene chiusa. Il chiamante gestisce l'errore del lavoro dopo il rilascio delle risorse. Costruzione e chiusura delle due risorse non sollevano eccezioni.

Le istruzioni dopo la classe sono eseguite in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Trace implements AutoCloseable {
02      private final String name;
03      Trace(String name) { this.name = name; }
04      public void close() { IO.println("Close " + name); }
05  }
```

```
06  try (Trace first = new Trace("A"); Trace second = new Trace("B")) {
07      IO.println("Work");
08      throw new IllegalArgumentException();
09  } catch (IllegalArgumentException exception) {
10      IO.println("Handled");
11  }
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le righe nell'ordine.

- **A.** `Work`, `Handled`, `Close A`, `Close B`.
- **B.** `Work`, `Close A`, `Close B`, `Handled`.
- **C.** `Work`, `Handled`, senza altre stampe.
- **D.** `Work`, `Close B`, `Close A`, `Handled`.

2. Risposta breve

Separatamente, un helper riceve un `java.io.Writer` aperto dal chiamante. Il contratto richiede che lo lasci aperto, anche se la scrittura fallisce, e che propaghi eventuali `java.io.IOException`.

```
12  static void writeReport(java.io.Writer output) throws java.io.IOException {
13      try (output) {
14          output.write("Report");
15      }
16  }
```

Valuta l'helper rispetto al contratto e, se necessario, correggine il corpo. Il parametro è non nullo; non occorre gestire il flush.

Lunghezza indicativa: una riga di codice e 1–2 frasi.

Mostra la soluzione dell'esercizio 2

1. Scelta multipla: D

Il corpo stampa `Work`, poi le risorse vengono chiuse in ordine inverso di dichiarazione: `Close B`, `Close A`. Infine il `catch` stampa `Handled`.

- **A errata:** il rilascio precede l'ingresso nel `catch` esterno.
- **B errata:** l'ordine di chiusura è invertito rispetto alla costruzione.
- **C errata:** l'eccezione non salta il rilascio automatico.

2. Risposta breve

Il try-with-resources chiude anche una risorsa ricevuta come parametro e viola il contratto di prestito. Il corpo deve essere:

```
output.write("Report");
```

La firma con `throws IOException` propaga il fallimento; la chiusura resta responsabilità del chiamante.

Esercizio 3 — Quantità richiesta

Tempo indicativo: 5–7 minuti · Java 25

Un componente riceve quantità sotto forma di testo. Il risultato numerico viene usato per pianificare un'elaborazione, nella quale anche zero è un valore significativo. La versione corrente intercetta gli errori di conversione e registra il termine dell'operazione con una stampa. Il metodo attuale è mostrato integralmente. Il testo è non nullo e non sono richiesti vincoli di intervallo ulteriori rispetto al tipo `int`.

Il metodo va dichiarato nel corpo di una classe e la chiamata in un suo metodo statico. I numeri di riga non fanno parte del codice.

```
01  static int count(String text) {
02      try {
03          return Integer.parseInt(text);
04      } catch (NumberFormatException exception) {
05          return 0;
06      } finally {
07          IO.println("Done");
08      }
09  }
```

```
10  IO.println(count("abc"));
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le righe nell'ordine.

- **A.** `0`, poi Done.
- **B.** Done, poi `0`.
- **C.** Solleva `NumberFormatException`, senza altre stampe.
- **D.** Stampa soltanto `0`.

2. Risposta aperta

Una nuova API deve distinguere un conteggio valido pari a zero da un errore di conversione. L'errore deve includere un messaggio di contesto e conservare la causa originaria, senza stampare né restituire valori sostitutivi.

Proponi un contratto di errore basato su un'eccezione di dominio: scegli checked o unchecked e motiva la scelta. Indica la relazione di ereditarietà, il costruttore necessario, l'eventuale modifica alla firma di `count` e il comportamento del suo `catch`. Non riscrivere il parsing.

Lunghezza indicativa: uno schema di massimo 6 righe e 1–2 frasi; indica anche che cosa accade al blocco `finally` attuale.

Mostra la soluzione dell'esercizio 3

1. Scelta multipla: B

Il parsing fallisce e il `catch` prepara il risultato zero; prima del ritorno viene eseguito il `finally`, che stampa Done.

- **A errata:** il `finally` precede il ritorno al chiamante.
- **C errata:** `NumberFormatException` viene intercettata.
- **D errata:** il `finally` viene eseguito anche dopo un `return` nel `catch`.

2. Risposta aperta

```
class InvalidCountException extends IllegalArgumentException {
    InvalidCountException(String message, Throwable cause) { super(message, cause); }
}
// Nel catch di count:
throw new InvalidCountException("Quantità non valida: " + text, exception);
```

Scelgo unchecked per un argomento non valido: `count` non deve aggiungere `throws` obbligatori. Elimino il `finally` che stampa; il chiamante distingue lo zero valido dall'eccezione e può consultarne la causa.

Alternativa valida: eccezione checked derivata da `Exception`, con dichiarazione `throws InvalidCountException`, se si vuole imporre ai chiamanti una gestione o propagazione esplicita.

Esercizio 4 — Lettura di un valore remoto

Tempo indicativo: 5–7 minuti · Java 25

Un servizio simula il recupero di un valore da una sorgente esterna. Il contratto dichiara che la lettura può fallire con `IOException`. Il client vuole stampare il valore quando disponibile oppure segnalare il fallimento. Per questo esercizio la sorgente fallisce sempre, senza effetti esterni. Tutti i metodi sono mostrati e non esistono gestori esterni delle eccezioni.

I metodi sono nella stessa classe, con `java.io.IOException` importata. Il secondo è il metodo chiamato dal programma. I numeri di riga non fanno parte del codice.

```
01  static int read() throws IOException {
02      throw new IOException("Offline");
03  }
04  static void printValue() {
05      IO.println(read());
06  }
```

1. Scelta multipla

La classe compila? Seleziona una sola risposta.

- **A.** Sì, e stampa `0`.
- **B.** Sì, perché `println` gestisce `IOException`.
- **C.** No: `printValue` non gestisce né dichiara l'eccezione checked.
- **D.** No: un metodo con risultato `int` non può terminare con `throw`.

2. Risposta breve

Il requisito di `printValue` è stampare `Unavailable` in caso di `IOException`, senza propagarla. Indica la modifica necessaria e spiega se sostituire l'eccezione con `RuntimeException` risolverebbe da solo tale requisito.

Lunghezza indicativa: massimo 5 righe di codice e una frase.

Mostra la soluzione dell'esercizio 4

1. Scelta multipla: C

`IOException` è checked e il chiamante deve gestirla o dichiararla.

- **A errata:** non esiste un ritorno automatico di zero.
- **B errata:** il calcolo dell'argomento avviene prima della stampa.
- **D errata:** un percorso che termina con `throw` non deve restituire un valore.

2. Risposta breve

```
try {
    IO.println(read());
} catch (IOException exception) {
    IO.println("Unavailable");
}
```

Rendere l'errore unchecked elimina un obbligo del compilatore, ma non produce da solo la segnalazione richiesta.

Esercizio 5 — Segnalazioni di importazione

Tempo indicativo: 5–7 minuti · Java 25

Un importatore usa un handler per classificare gli input non validi. Il parsing può sollevare `NumberFormatException`, che estende `IllegalArgumentException`. Il chiamante vuole distinguere il testo non numerico dagli altri argomenti non validi. Le righe sono non nulle e non servono risorse esterne. Il frammento contiene tutti i gestori rilevanti.

Frammento in un metodo. I numeri di riga non fanno parte del codice.

```
01  try {
02      IO.println(Integer.parseInt("x"));
03  } catch (IllegalArgumentException exception) {
04      IO.println("Argument");
05  } catch (NumberFormatException exception) {
06      IO.println("Number");
07  }
```

1. Scelta multipla

Che cosa accade? Seleziona una sola risposta.

- **A.** Non compila: il secondo `catch` è già coperto dal primo.
- **B.** Stampa `Argument`, poi `Number`.
- **C.** Stampa solo `Number`.
- **D.** Stampa solo `Argument`.

2. Risposta breve

Proponi una modifica minima che conservi entrambi i gestori e dia priorità al messaggio specifico. Indica l'output dopo la modifica e quanti `catch` vengono eseguiti per questa singola eccezione.

Lunghezza indicativa: 2–3 frasi; non riscrivere il frammento.

Mostra la soluzione dell'esercizio 5

1. Scelta multipla: A

Il gestore di `IllegalArgumentException` include già la sua sottoclasse, rendendo irraggiungibile il gestore successivo.

- **B errata:** per una singola eccezione non si attraversano entrambi i gestori.
- **C errata:** il codice originale non compila.
- **D errata:** non è solo una questione di scelta a runtime: l'ordine è rifiutato dal compilatore.

2. Risposta breve

Scambio i due `catch`, mettendo prima `NumberFormatException`. L'input mostrato produce `Number`; viene eseguito soltanto il primo gestore compatibile. Il gestore più generale resta disponibile per altre `IllegalArgumentException`.

Esercizio 6 — Esito di una verifica

Tempo indicativo: 5–7 minuti · Java 25

Un componente verifica un dato e restituisce un codice di esito. Il chiamante deve poter distinguere il risultato normale da un input non valido. Un blocco di pulizia viene eseguito prima del ritorno. Nel frammento la pulizia è simulata senza risorse esterne. Il parametro è un booleano e l'eccezione di validazione deve raggiungere il chiamante.

Metodo in una classe; la chiamata è in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  static int check(boolean valid) {
02      try {
03          if (!valid) throw new IllegalArgumentException("Invalid");
04          return 1;
05      } finally {
06          return 2;
07      }
08  }
09  // Chiamata in un altro metodo:
10  IO.println(check(false));
```

1. Scelta multipla

Che cosa accade? Seleziona una sola risposta.

- **A.** Non compila perché `finally` non può contenere `return`.
- **B.** Propaga `IllegalArgumentException`.
- **C.** Stampa 1.
- **D.** Stampa 2.

2. Risposta breve

La pulizia deve stampare solo `Cleanup`, senza alterare il risultato o nascondere l'errore. Indica il nuovo corpo di `finally` e i comportamenti attesi con `true` e `false`.

Lunghezza indicativa: una riga di codice e 2 frasi.

Mostra la soluzione dell'esercizio 6

1. Scelta multipla: D

Il `return` nel `finally` sostituisce l'uscita eccezionale del `try` con il valore 2.

- **A errata:** è sintassi ammessa, anche se pericolosa.
- **B errata:** il `finally` maschera l'eccezione.
- **C errata:** il parametro è `false` e comunque il `finally` sostituirebbe anche quel risultato.

2. Risposta breve

```
IO.println("Cleanup");
```

Con `true`, dopo `Cleanup` il metodo restituisce 1. Con `false`, dopo la stessa stampa propaga `IllegalArgumentException`, senza un valore di ritorno normale.

Esercizio 7 — Rilascio di una connessione

Tempo indicativo: 5–7 minuti · Java 25

Un componente acquisisce una risorsa e la usa per un'operazione. Tanto l'operazione quanto la chiusura possono fallire. Per riconoscere entrambi i problemi, il chiamante esamina l'eccezione principale e quelle soppresse. La risorsa qui è simulata e la costruzione riesce sempre. `getSuppressed()` restituisce le eccezioni di chiusura conservate dal try-with-resources quando il corpo è già fallito.

Classe e istruzioni in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Connection implements AutoCloseable {
02      public void close() { throw new IllegalStateException("Close"); }
03  }
04  // In un altro metodo:
05  try (Connection connection = new Connection()) {
06      throw new IllegalArgumentException("Work");
07  } catch (RuntimeException exception) {
08      IO.println(exception.getMessage());
09      IO.println(exception.getSuppressed().length);
10  }
```

1. Scelta multipla

Seleziona l'output corretto.

- **A.** `Close`, poi `0`.
- **B.** `Work`, poi `1`.
- **C.** `Work`, poi `0`.
- **D.** `Close`, poi `1`.

2. Risposta breve

Considera la versione in cui il corpo del `try` termina normalmente e solo `close()` fallisce. Quale eccezione raggiunge il `catch` e quante eccezioni soppresse contiene in questo esempio? Spiega la differenza rispetto al codice mostrato.

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 7

1. Scelta multipla: B

Il fallimento del corpo resta principale con messaggio `Work`; quello della chiusura è conservato come unica eccezione soppressa.

- **A errata:** la chiusura non sostituisce il fallimento già avvenuto.
- **C errata:** il fallimento di chiusura viene conservato.
- **D errata:** il principale non ha messaggio `Close`.

2. Risposta breve

Arriva `IllegalStateException("Close")`, senza eccezioni soppresse. In assenza di un fallimento del corpo, la prima eccezione di chiusura è essa stessa il fallimento principale.

Esercizio 8 — Apertura di due risorse

Tempo indicativo: 5–7 minuti · Java 25

Un'elaborazione deve acquisire due risorse prima di iniziare il lavoro. L'acquisizione della seconda può fallire. La prima è già stata aperta e deve essere rilasciata anche in quel caso. La simulazione stampa le chiusure e segnala il fallimento dell'apertura, senza dipendere dal filesystem. Soltanto il costruttore relativo al nome `B` fallisce; `close()` termina normalmente.

Classe e istruzioni in un metodo statico separato. I numeri di riga non fanno parte del codice.

```java
class Resource implements AutoCloseable {
    private final String name;
    Resource(String name) {
        this.name = name;
        if (name.equals("B")) throw new IllegalStateException("Open B");
    }
    public void close() { IO.println("Close " + name); }
}
// In un altro metodo:
try (Resource first = new Resource("A"); Resource second = new Resource("B")) {
    IO.println("Work");
} catch (IllegalStateException exception) {
    IO.println(exception.getMessage());
}
```

1. Scelta multipla

Seleziona l'output corretto.

- **A.** `Work`, `Close B`, `Close A`, `Open B`.
- **B.** Soltanto `Open B`.
- **C.** `Close A`, poi `Open B`.
- **D.** `Close B`, `Close A`, `Open B`.

2. Risposta aperta

Un nuovo helper riceve una risorsa già aperta dal chiamante e ne apre una seconda di propria iniziativa. Definisci un contratto di proprietà che chiarisca chi chiude ciascuna risorsa, anche in caso di fallimento, e quale delle due va dichiarata nel try-with-resources dell'helper.

Lunghezza indicativa: 3 frasi; non serve codice.

Mostra la soluzione dell'esercizio 8

1. Scelta multipla: C

Fallisce l'acquisizione di `B`, quindi il corpo non parte e viene chiusa soltanto la risorsa `A`, già acquisita.

- **A errata:** il corpo non viene eseguito.
- **B errata:** la prima risorsa viene rilasciata.
- **D errata:** la risorsa la cui costruzione fallisce non viene registrata per la chiusura automatica.

2. Risposta aperta

La prima risorsa è in prestito: la chiude il chiamante, anche quando l'helper fallisce. La seconda è posseduta dall'helper, che deve garantirne il rilascio. Solo la seconda viene dichiarata nel try-with-resources dell'helper; eventuali eccezioni sono propagate o tradotte conservandone la causa.

Esercizio 9 — Ricerca di una prenotazione

Tempo indicativo: 5–7 minuti · Java 25

Un servizio cerca una prenotazione mediante un codice. Una prenotazione assente è un esito normale, diverso dall'impossibilità di interrogare l'archivio. La versione mostrata usa un valore sostitutivo per la sola assenza. Il nuovo client deve poter distinguere i due esiti senza analizzare messaggi testuali. Codici e archivio sono non nulli e l'archivio in memoria non può fallire durante il frammento.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  Map<String, String> bookings = Map.of("A", "Ada");
02  String name = bookings.get("B");
03  if (name == null) {
04      IO.println("Missing");
05  } else {
06      IO.println(name);
07  }
```

1. Scelta multipla

Che cosa accade? Seleziona una sola risposta.

- **A.** Stampa `Missing`.
- **B.** Solleva `NullPointerException`.
- **C.** Stampa `Ada`.
- **D.** Non compila perché una mappa non può restituire `null`.

2. Risposta aperta

Proponi il contratto di `find` per un archivio che può fallire con `IOException`: rappresenta l'assenza senza eccezioni e lascia distinguibile il fallimento tecnico. Fornisci la firma e descrivi i tre esiti, senza implementare l'accesso all'archivio.

Lunghezza indicativa: una firma e 2–3 frasi. Puoi usare `Optional` specificando che rappresenta presenza o assenza.

Mostra la soluzione dell'esercizio 9

1. Scelta multipla: A

La chiave B manca e `get` restituisce `null`, che viene controllato prima della stampa.

- **B errata:** confrontare un riferimento con `null` non lo dereferenzia.
- **C errata:** il valore di A non viene usato come ripiego.
- **D errata:** anche una mappa che vieta valori nulli restituisce `null` per una chiave assente.

2. Risposta aperta

`Optional<String> find(String code) throws IOException`: valore presente per una prenotazione trovata, `Optional.empty()` per l'assenza, eccezione per il fallimento tecnico. Non si deve trasformare un'`IOException` in un risultato vuoto. È valido anche un contratto documentato con `null` per l'assenza e la stessa eccezione per l'errore, se la distinzione è esplicita.

Esercizio 10 — Ripresa di un'operazione

Tempo indicativo: 5–7 minuti · Java 25

Un componente può fallire temporaneamente prima di completare un'operazione. Il chiamante consente al massimo due tentativi. La simulazione fallisce alla prima invocazione e riesce alla seconda. Il contatore parte da zero e il programma è monothread. Non sono richiesti attese, rete o concorrenza.

Metodi nella stessa classe; istruzioni finali in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  static int calls;
02  static int attempt() {
03      if (++calls == 1) throw new IllegalStateException("Retry");
04      return 7;
05  }
06  // In un altro metodo:
07  for (int i = 0; i < 2; i++) {
08      try {
09          IO.println(attempt());
10          break;
11      } catch (IllegalStateException exception) {
12          IO.println("Failed");
13      }
14  }
```

1. Scelta multipla

Seleziona l'output corretto.

- **A.** `Failed`, poi `Failed`.
- **B.** Soltanto 7.
- **C.** `Failed`, poi 7, poi 7.
- **D.** `Failed`, poi 7.

2. Risposta breve

Un revisore sposta il `try/catch` all'esterno dell'intero ciclo. Con lo stesso stato iniziale, il secondo tentativo viene ancora eseguito dopo il primo fallimento? Spiega il percorso di controllo e indica quale collocazione soddisfa la politica di riprova.

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 10

1. Scelta multipla: D

La prima chiamata fallisce e stampa `Failed`; la seconda restituisce 7 e il `break` termina il ciclo.

- **A errata:** solo la prima chiamata fallisce.
- **B errata:** il primo fallimento viene stampato.
- **C errata:** il limite e il `break` impediscono ulteriori stampe.

2. Risposta breve

Con un unico `try` esterno, l'eccezione abbandona tutto il ciclo e il secondo tentativo non parte. Il `catch` interno all'iterazione consente invece di proseguire alla successiva. La riprova richiede questa gestione per tentativo, non il rientro automatico nel punto che ha fallito.

Esercizio 11 — Confronto delle date

Tempo indicativo: 5–7 minuti · Java 25

Un servizio verifica che la data iniziale di una prenotazione sia strettamente precedente a quella finale. Le date arrivano come stringhe nel formato `yyyy-MM-dd`. Il contratto di `before` richiede di propagare al chiamante gli errori di interpretazione, per distinguerli da un intervallo non valido. Un altro componente dell'applicazione usa invece una politica diversa, restituendo `false` anche per date malformate: i due contratti non vanno confusi.

Le stringhe sono non nulle. `LocalDate.parse` interpreta una data e lancia `DateTimeParseException` se non è valida; `isBefore` è falso anche quando le date sono uguali. Il frammento

appartiene a una classe di test. `@Test` identifica un test; `assertTrue` richiede una condizione vera e `assertThrows(T.class, azione)` richiede che l'azione lanci un'eccezione di tipo T o di un suo sottotipo, altrimenti fallisce. Import omessi; i numeri di riga non fanno parte del codice.

```
01  static boolean before(String begin, String end) {
02      try {
03          return LocalDate.parse(begin).isBefore(LocalDate.parse(end));
04      } catch (DateTimeParseException e) {
05          return false;
06      }
07  }
08
09  @Test
10  void validInterval() {
11      assertTrue(before("2026-05-01", "2026-05-03"));
12  }
13
14  @Test
15  void malformedDate() {
16      assertThrows(DateTimeParseException.class,
17              () -> before("not-a-date", "2026-05-03"));
18  }
```

1. Scelta multipla

Qual è l'esito dei test rispetto al metodo mostrato?

- **A.** Passano entrambi perché `parse` lancia l'eccezione richiesta.
- **B.** Passa soltanto `malformedDate`.
- **C.** Passa soltanto `validInterval`.
- **D.** Il metodo non compila perché `DateTimeParseException` deve comparire in una clausola `throws`.

2. Risposta aperta

Indica, in **1–3 frasi**, un caso che permetta di distinguere il requisito «strettamente precedente» da «precedente o uguale», con l'esito atteso. Spiega anche perché il solo caso di un intervallo crescente non distingue queste due interpretazioni. Non scrivere codice.

Mostra la soluzione dell’esercizio 11

1. Scelta multipla: C

Il primo intervallo è crescente e restituisce `true`. Nel secondo test `parse` lancia l'eccezione, ma il metodo la intercetta e restituisce `false`: `assertThrows` fallisce perché non riceve alcuna eccezione.

- **A errata:** conta l'eccezione che esce dall'azione, non una che viene lanciata e gestita al suo interno.
- **B errata:** è il test sull'input malformato a fallire; quello sull'intervallo valido passa.
- **D errata:** `DateTimeParseException` è unchecked e non richiede una dichiarazione `throws`.

2. Risposta aperta

Con `begin` ed `end` entrambi uguali a `"2026-05-01"`, l'esito atteso è `false`; un confronto che ammette anche l'uguaglianza restituirebbe `true`. Un intervallo crescente soddisfa entrambe le interpretazioni e non permette di distinguerle; questa verifica del confine è distinta da quella sulla propagazione degli errori.

Esercizio 12 — Invio di una notifica

Tempo indicativo: 5–7 minuti · Java 25

Un servizio prova a inviare una notifica e ripete l'operazione se riceve `IOException`. Il parametro `maxAttempts` è il numero massimo di chiamate totali, inclusa la prima, ed è almeno uno. Al primo successo il servizio deve terminare; se tutti i tentativi consentiti falliscono deve propagare l'ultima eccezione. Nei test, `Writer` simula un numero prestabilito di fallimenti iniziali e conta le chiamate.

Non vi sono concorrenza o altri tipi di errore. Il frammento appartiene a una classe di test e `Writer` è una classe annidata statica. `@Test` identifica un test; `assertEquals(atteso, effettivo)` confronta i valori e `assertThrows(T.class, azione)` intercetta un'eccezione del tipo richiesto o di un suo sottotipo, fallendo se non viene lanciata. Import omessi; i numeri di riga non fanno parte del codice.

```
01  static class Writer {
02      final int failures;
03      int calls;
04      Writer(int failures) { this.failures = failures; }
05      void write() throws IOException {
06          calls++;
07          if (calls <= failures) {
08              throw new IOException("Unavailable");
09          }
10      }
11  }
12  static void send(Writer writer, int maxAttempts) throws IOException {
13      for (int attempt = 0; attempt <= maxAttempts; attempt++) {
14          try {
15              writer.write();
16              return;
17          } catch (IOException e) {
```

```
18              if (attempt == maxAttempts) {
19                  throw e;
20              }
21          }
22      }
23  }
24  @Test
25  void recovery() throws IOException {
26      Writer writer = new Writer(1);
27      send(writer, 2);
28      assertEquals(2, writer.calls);
29  }
30  @Test
31  void unavailable() {
32      Writer writer = new Writer(5);
33      assertThrows(IOException.class, () -> send(writer, 2));
34      assertEquals(2, writer.calls);
35  }
```

1. Scelta multipla

Qual è l'esito dei test?

- **A.** Passano entrambi: il limite rappresenta due ritentativi dopo il primo invio.
- **B.** Passa soltanto `recovery`; in `unavailable` l'eccezione è intercettata dall'asserzione, ma il contatore vale 3.
- **C.** Passa soltanto `unavailable`; il metodo continua a chiamare `write` anche dopo un successo.
- **D.** Falliscono entrambi perché il primo `IOException` esce immediatamente da `send`.

2. Risposta aperta

Un collega propone di sostituire `Writer` nei test con un oggetto che fallisce casualmente a ogni chiamata. Valuta in **1–3 frasi** l'effetto di questa scelta sulla riproducibilità dei test e sulla verifica del limite di tentativi. Non scrivere codice.

Mostra la soluzione dell’esercizio 12

1. Scelta multipla: B

In `recovery` il primo invio fallisce e il secondo riesce: il `return` termina il metodo con due chiamate. In `unavailable` il ciclo usa gli indici `0`, `1`, `2`, effettuando tre chiamate: `assertThrows` passa, poi il confronto tra 2 e 3 fallisce.

- **A errata:** il contratto definisce due tentativi totali, non due ritentativi oltre al primo.
- **C errata:** il `return` impedisce chiamate successive al successo.
- **D errata:** il `catch` gestisce i fallimenti intermedi e propaga soltanto quello dell'ultima iterazione.

2. Risposta aperta

Con esiti casuali il numero di chiamate e l'eventuale eccezione cambierebbero tra esecuzioni: un test sul limite potrebbe incontrare un successo anticipato e non verificare il percorso desiderato. Gli esiti prestabiliti consentono invece di verificare separatamente recupero dopo un fallimento e fallimento di tutti i tentativi, con aspettative riproducibili.

E6 - Java Generics

Esercizio 1 — Primo elemento

Tempo indicativo: 5–7 minuti · Java 25

Un servizio consulta il primo elemento di liste provenienti da archivi diversi. Il metodo `first` non deve modificare le liste. I client possono lavorare sia con nomi sia con conteggi e vogliono conservare l'informazione sul tipo degli elementi. Le liste sono non nulle e non vuote. I due frammenti di chiamata vanno considerati separatamente, non come parti dello stesso programma.

I metodi e le istruzioni si trovano rispettivamente nel corpo di una classe e in un suo metodo statico, con `java.util.List` importata. I numeri di riga non fanno parte del codice.

```
01  static Object first(List<?> values) {
02      return values.get(0);
03  }
```

```
04  List<Integer> values = List.of(4, 7);
05  List<Number> numbers = values;
06  IO.println(numbers.get(0));
```

1. Scelta multipla

Che cosa accade eseguendo le righe 4–6? Seleziona una sola risposta.

- **A.** Stampa 4, perché `Integer` è un sottotipo di `Number`.
- **B.** Non compila: `List<Integer>` non è assegnabile a `List<Number>`.
- **C.** Compila, ma solleva `ClassCastException` alla riga 6.
- **D.** Non compila perché `List.of` non può contenere wrapper numerici.

2. Risposta aperta

Si vuole usare lo stesso metodo `first`, senza cast nel metodo o nei chiamanti, in entrambi questi casi:

```
07  String name = first(List.of("Ada", "Linus"));
08  Integer count = first(List.of(4, 7));
```

Proponi il contratto di tipo e il corpo necessari, senza tipi raw né overload dedicati ai singoli tipi. Spiega quale informazione il contratto attuale non conserva per il chiamante.

Lunghezza indicativa: massimo 3 righe di codice e 1–2 frasi.

Mostra la soluzione dell'esercizio 1

1. Scelta multipla: B

I tipi generici sono invarianti: `List<Integer>` non è un sottotipo di `List<Number>`.

- **A errata:** la relazione tra i tipi degli elementi non si trasferisce alle liste.
- **C errata:** il problema è rilevato a compilazione, non durante la lettura.
- **D errata:** `List.of` accetta wrapper numerici.

2. Risposta aperta

```java
static <T> T first(List<T> values) {
    return values.get(0);
}
```

Il parametro di tipo collega il risultato al tipo degli elementi, mentre `Object` non conserva questo legame per il chiamante. È valida anche `List<? extends T>` come parametro, lasciando invariato il risultato T.

Esercizio 2 — Operazioni su misure numeriche

Tempo indicativo: 5–7 minuti · Java 25

Un laboratorio riceve misure numeriche da strumenti diversi. Alcuni producono interi e altri valori decimali. Il componente deve poter leggere entrambe le rappresentazioni tramite le operazioni di `Number`. Le misure sono non nulle e di piccola grandezza: non considerare overflow o problemi di precisione. Le liste sono non nulle, anche eventualmente vuote.

I metodi vanno dichiarati nel corpo di una classe e le istruzioni in un suo metodo statico, con `java.util.List` importata. I numeri di riga non fanno parte del codice.

```java
01  static <T extends Number> double twice(T value) {
02      return 2 * value.doubleValue();
03  }
04  static double sum(List<Number> values) {
05      double result = 0;
06      for (Number value : values) result += value.doubleValue();
07      return result;
08  }
```

```
09  IO.println(twice("3"));
```

1. Scelta multipla

Che cosa accade alla riga 9? Seleziona una sola risposta.

- **A.** Stampa `6.0` dopo aver convertito la stringa in un numero.
- **B.** Compila, ma solleva `ClassCastException` dentro `twice`.
- **C.** Non compila: `String` non soddisfa il vincolo sul parametro di tipo.
- **D.** Stampa 33, applicando la concatenazione.

2. Risposta aperta

Il metodo `sum` deve essere chiamabile sia con una variabile `List<Integer>` sia con una `List<Double>` o `List<Number>`. Deve soltanto leggere gli elementi, senza scrivere, copiare la lista o introdurre cast.

Proponi una firma che permetta questo uso lasciando invariato il corpo. Spiega quali operazioni sugli elementi sono garantite dal tipo scelto e se la nuova firma renda immutabile la lista originale.

Lunghezza indicativa: una firma e 2–3 frasi.

Mostra la soluzione dell'esercizio 2

1. Scelta multipla: C

Il vincolo `T extends Number` esclude `String`: non viene eseguita una conversione dal testo.

- **A errata:** il metodo non effettua parsing.
- **B errata:** il compilatore rifiuta già la chiamata.
- **D errata:** la moltiplicazione usa il valore numerico di `Number`, non concatenazioni.

2. Risposta aperta

`static double sum(List<? extends Number> values)` consente di leggere ogni elemento come `Number` e invocare `doubleValue`. Il tipo preciso non è noto, quindi non si possono aggiungere arbitrariamente numeri non nulli. La firma non rende immutabile l'oggetto lista: limita le operazioni disponibili attraverso quel riferimento, senza impedire modifiche tramite altri alias.

Alternativa valida: `static <T extends Number> double sum(List<T> values)`.

Esercizio 3 — Riepilogo di una lista

Tempo indicativo: 5–7 minuti · Java 25

Un componente espone il numero di elementi di liste provenienti da due archivi. Il primo archivio contiene testi e il secondo conteggi interi. Il programmatore propone due overload con lo stesso nome per servire entrambi i client. Nessun metodo deve modificare la lista ricevuta. Le liste sono non nulle e possono essere vuote.

I metodi vanno dichiarati nella stessa classe, con `java.util.List` importata. I numeri di riga non fanno parte del codice.

```
01  static int size(List<String> values) {
02      return values.size();
03  }
04  static int size(List<Integer> values) {
05      return values.size();
06  }
```

1. Scelta multipla

La classe può essere compilata con entrambe le dichiarazioni? Seleziona una sola risposta.

- **A.** No: i due metodi hanno la stessa firma dopo l'erasure.
- **B.** Sì: il tipo degli elementi rende distinte le firme a runtime.
- **C.** Sì, purché nessun chiamante passi una lista vuota.
- **D.** No: i metodi statici non possono ricevere liste parametrizzate.

2. Risposta breve

Separatamente dai due metodi, serve verificare un oggetto ricevuto come `Object`: il controllo deve restituire `true` soltanto se si tratta di una lista i cui elementi sono tutti stringhe non nulle. Una lista vuota deve essere accettata; `null`, gli altri oggetti e le liste con elementi di altro tipo devono essere rifiutati.

Descrivi i controlli da effettuare senza tipi raw o cast non verificati. Precisa che cosa puoi controllare sul contenitore e che cosa richiede l'esame degli elementi.

Lunghezza indicativa: 3 frasi oppure massimo 6 righe di pseudocodice.

Mostra la soluzione dell'esercizio 3

1. Scelta multipla: A

Entrambi i parametri diventano `List` dopo l'erasure; le dichiarazioni hanno lo stesso nome e la stessa firma erasa.

- **B errata:** gli argomenti di tipo non distinguono questi overload a runtime.
- **C errata:** i contenuti delle liste non incidono sulla validità delle dichiarazioni.

- **D errata:** un metodo statico può ricevere tipi parametrizzati.

2. Risposta breve

Controllo `value instanceof List<?> list`; se fallisce, restituisco `false`, anche per `null`. Visito poi ogni elemento e rifiuto quelli per cui `element instanceof String` è falso, compreso `null`. Se la visita termina, restituisco `true`: il contenitore è riconoscibile a runtime, ma il tipo di tutti i suoi elementi richiede la visita; quella vuota viene accettata.

Esercizio 4 — Deposito di un elemento

Tempo indicativo: 5–7 minuti · Java 25

Un componente conserva un singolo dato ricevuto dal chiamante. Lo stesso contenitore deve essere riutilizzabile per tipi diversi senza cast nei client. La versione mostrata conserva il tipo tramite un parametro generico. Due depositi distinti non condividono stato. I dati sono non nulli e non sono richieste operazioni concorrenti.

Classe e istruzioni in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Slot<T> {
02      private T value;
03      Slot(T value) { this.value = value; }
04      T get() { return value; }
05      void set(T value) { this.value = value; }
06  }
07  // In un altro metodo:
08  Slot<String> slot = new Slot<>("Ada");
09  slot.set("Linus");
10  IO.println(slot.get().length());
```

1. Scelta multipla

Che cosa accade? Seleziona una sola risposta.

- **A.** Stampa 3.
- **B.** Non compila perché l'erasure impedisce di chiamare `length`.
- **C.** Stampa 5.
- **D.** Solleva `ClassCastException`.

2. Risposta aperta

Un client deve poter leggere qualunque `Slot`, senza conoscere il tipo del dato e senza sostituirlo. Proponi il tipo del parametro di `Object inspect(...)` e il suo corpo minimo; spiega perché non serve un tipo raw.

Lunghezza indicativa: massimo 3 righe di codice e una frase.

Mostra la soluzione dell'esercizio 4

1. Scelta multipla: C

Il dato è stato sostituito con `Linus`, che ha lunghezza cinque. Il compilatore conserva l'informazione di tipo necessaria al client.

- **A errata:** il vecchio testo non è più il valore del deposito.
- **B errata:** l'erasure non annulla il controllo statico della chiamata.
- **D errata:** il dato letto ha effettivamente tipo `String`.

2. Risposta aperta

```java
static Object inspect(Slot<?> slot) {
    return slot.get();
}
```

La wildcard rappresenta un tipo sconosciuto e consente la lettura come `Object`, senza rinunciare ai controlli sui generics. Non rende immutabile il deposito, ma impedisce a questo client di assegnargli un arbitrario valore non nullo.

Esercizio 5 — Destinazione dei conteggi

Tempo indicativo: 5–7 minuti · Java 25

Un servizio trasferisce conteggi interi verso una raccolta scelta dal chiamante. La destinazione può essere una lista di interi o di un loro supertipo. La sorgente non viene modificata. I valori sono non nulli e le raccolte sono modificabili e non nulle. Il metodo aggiunge un solo conteggio per chiamata.

Metodo e istruzioni in un metodo statico separato, con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```java
static void append(List<? super Integer> target, int count) {
    target.add(count);
}
// In un altro metodo:
List<Number> target = new ArrayList<>();
append(target, 4);
IO.println(target.get(0));
```

1. Scelta multipla

Che cosa accade? Seleziona una sola risposta.

- **A.** Stampa 4.
- **B.** Non compila perché `Number` non è esattamente `Integer`.
- **C.** Compila, ma add solleva sempre un'eccezione.
- **D.** Stampa `null`.

2. Risposta breve

Nel corpo di append, si può aggiungere `2.5` oppure leggere un elemento direttamente in una variabile `Integer`? Valuta separatamente le due operazioni sulla base del tipo del parametro, senza cast.

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 5

1. Scelta multipla: A

`List<Number>` è una destinazione ammessa per `? super Integer`; il conteggio viene inserito e letto.

- **B errata:** la wildcard ammette i supertipi di `Integer`.
- **C errata:** la destinazione concreta è un'`ArrayList` modificabile.
- **D errata:** viene aggiunto il valore 4.

2. Risposta breve

Non si può aggiungere `2.5`, perché la destinazione potrebbe essere una `List<Integer>`. Neppure la lettura diretta come `Integer` è garantita: potrebbe essere una `List<Object>` contenente altri oggetti. La lettura generale è come `Object`, mentre la scrittura di un `Integer` è sicura.

Esercizio 6 — Trasferimento tra archivi

Tempo indicativo: 5–7 minuti · Java 25

Due archivi devono trasferire dati senza conversioni numeriche. La sorgente produce elementi da leggere e la destinazione li riceve. Un primo metodo usa lo stesso tipo preciso di lista per entrambi. I client devono poter trasferire interi in raccolte di numeri o oggetti, senza tipi raw. Le liste sono non nulle, distinte e la destinazione è modificabile.

Metodo e chiamata separata nella stessa classe, con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  static <T> void transfer(List<T> source, List<T> target) {
02      for (T value : source) {
03          target.add(value);
04      }
05  }
06  // In un altro metodo:
07  List<Integer> source = List.of(2, 3);
08  List<Number> target = new ArrayList<>();
09  transfer(source, target);
```

1. Scelta multipla

La chiamata compila? Seleziona una sola risposta.

- **A.** Sì, con `T = Object`.
- **B.** Sì, e converte ogni `Integer` in `Double`.
- **C.** Sì, perché entrambe le liste hanno elementi numerici.
- **D.** No: i due parametri `List<T>` impongono tipi incompatibili per questa chiamata.

2. Risposta aperta

Proponi una firma generica che consenta il trasferimento richiesto lasciando invariato il corpo. Descrivi il ruolo dei due limiti e indica se consente anche il percorso inverso da `List<Number>` a `List<Integer>`.

Lunghezza indicativa: una firma e 2 frasi.

Mostra la soluzione dell’esercizio 6

1. Scelta multipla: D

L’invarianza impedisce di scegliere un unico `T` che renda assegnabili entrambe le liste ai parametri dichiarati.

- **A errata:** `List<Integer>` non diventa `List<Object>`.
- **B errata:** i generics non convertono gli elementi tra wrapper.
- **C errata:** una comune superclasse degli elementi non rende covarianti le liste.

2. Risposta aperta

`static <T> void transfer(List<? extends T> source, List<? super T> target)` legge elementi compatibili con T e li scrive in un contenitore che può riceverli. Con sorgente `Integer` e destinazione `Number` si può scegliere `T = Integer`; il percorso inverso non è ammesso perché un `Number` potrebbe essere un `Double`.

Esercizio 7 — Contenuto di un pacco

Tempo indicativo: 5–7 minuti · Java 25

Un servizio riceve oggetti da componenti con informazioni di tipo diverse. Deve riconoscere le liste e consultarne la dimensione senza assumere il tipo degli elementi. Il controllo non deve modificare il dato. `null` e gli oggetti che non sono liste devono essere rifiutati. La lista dell'esempio contiene stringhe non nulle.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  Object payload = List.of("Ada", "Linus");
02  if (payload instanceof List<?> values) {
03      IO.println(values.size());
04      IO.println(values.get(0));
05  }
```

1. Scelta multipla

Che cosa accade? Seleziona una sola risposta.

- **A.** Non compila perché nessun `instanceof` può usare generics.
- **B.** Stampa 2, poi Ada.
- **C.** Solleva `ClassCastException` leggendo il primo elemento.
- **D.** Stampa `0`, poi `null`.

2. Risposta breve

Un revisore vuole sostituire il controllo con `payload instanceof List<String>`. Perché non è ammesso qui? Spiega anche perché il controllo originale non dimostri che tutti gli elementi siano stringhe.

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 7

1. Scelta multipla: B

`List<?>` può essere verificato con questo `instanceof`; la dimensione è due e l'elemento letto come `Object` viene stampato come Ada.

- **A errata:** una wildcard non limitata è consentita in questo controllo.
- **C errata:** non si effettua un cast a un tipo specifico di elemento.
- **D errata:** il controllo non svuota la lista.

2. Risposta breve

Da un `Object` non si può verificare a runtime l'argomento `String` di una lista: l'erasure non lo conserva per quel controllo. `List<?>` dimostra soltanto che il contenitore è una lista. Per verificare il contenuto corrente bisogna esaminare i singoli elementi, senza dedurne un vincolo permanente su tutte le possibili modifiche future.

Esercizio 8 — Dati di un componente precedente

Tempo indicativo: 5–7 minuti · Java 25

Un vecchio componente usa una lista senza argomenti di tipo. Un nuovo client dichiara invece di lavorare soltanto con stringhe. Le due parti condividono lo stesso oggetto lista. La migrazione deve rendere rilevabili gli inserimenti di tipo errato prima dell'esecuzione. Il frammento ammette warning del compilatore, che non sono trattati come errori.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  List<String> names = new ArrayList<>();
02  List legacy = names;
03  legacy.add(42);
04  String first = names.get(0);
05  IO.println(first);
```

1. Scelta multipla

Che cosa accade? Seleziona una sola risposta.

- **A.** Stampa 42 come stringa.
- **B.** Non compila neppure se si ammettono i warning.
- **C.** Solleva `ClassCastException` alla riga 4.
- **D.** Solleva `ClassCastException` alla riga 3.

2. Risposta breve

Per la nuova versione tutto il codice è modificabile e non deve restare alcun tipo raw. Quale dichiarazione useresti per `legacy` e quale riga diventerebbe errata a compilazione? Spiega perché aggiungere un cast alla lettura non risolve il problema di progetto.

Lunghezza indicativa: una dichiarazione e 2 frasi.

Mostra la soluzione dell'esercizio 8

1. Scelta multipla: C

L'inserimento raw produce un warning ma entra nella lista. La lettura tramite `List<String>` richiede un cast a `String`, che fallisce sull'`Integer`.

- **A errata:** non esiste una conversione automatica a testo.
- **B errata:** il codice può compilare con warning unchecked.
- **D errata:** la lista erasa non rifiuta quel valore al momento dell'inserimento.

2. Risposta breve

Uso `List<String> legacy = names;`: `legacy.add(42)` diventa un errore di tipo. Un cast alla lettura non cambia il contenuto incompatibile e può solo posticipare o ripetere il fallimento a runtime.

Esercizio 9 — Massimo tra valori

Tempo indicativo: 5–7 minuti · Java 25

Un servizio deve scegliere il maggiore tra due valori dello stesso tipo. Il criterio è l'ordine naturale del tipo scelto dal chiamante. Il servizio non modifica gli oggetti e non deve convertire tutto in stringhe. Gli argomenti sono non nulli e la relazione d'ordine è quella di `compareTo`. Un risultato negativo di `compareTo` significa che il ricevente precede l'argomento.

Metodo e chiamata in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  static <T extends Comparable<T>> T larger(T first, T second) {
02      if (first.compareTo(second) >= 0) return first;
03      return second;
04  }
05  // In un altro metodo:
06  IO.println(larger("Ada", "Linus"));
```

1. Scelta multipla

Che cosa accade? Seleziona una sola risposta.

- **A.** Stampa `Linus`.
- **B.** Stampa `Ada` perché è il primo argomento.
- **C.** Non compila: i generics non ammettono vincoli su interfacce.
- **D.** Stampa il numero restituito da `compareTo`.

2. Risposta aperta

Una variante deve ricevere l'ordinamento dal chiamante invece di richiedere `Comparable`. Proponi la firma con un `Comparator` e descrivi la sola modifica al confronto. Non implementare un comparatore concreto.

Lunghezza indicativa: una firma e 1–2 frasi. Puoi omettere l'import di `Comparator`.

Mostra la soluzione dell'esercizio 9

1. Scelta multipla: A

Nell'ordine naturale delle stringhe `Ada` precede `Linus`, quindi il metodo restituisce il secondo valore.

- **B errata:** il primo valore viene restituito solo se non minore.
- **C errata:** `extends` può esprimere un vincolo su un'interfaccia.
- **D errata:** il risultato del metodo è un oggetto di tipo `T`, non l'esito numerico del confronto.

2. Risposta aperta

`static <T> T larger(T first, T second, Comparator<? super T> order)` elimina il vincolo `Comparable`. Il confronto diventa `order.compare(first, second) >= 0`; resta invariata la scelta dell'oggetto da restituire. È valida anche una firma con `Comparator<T>` se basta l'uso con il tipo preciso.

Esercizio 10 — Costruzione di una raccolta

Tempo indicativo: 5–7 minuti · Java 25

Un componente generico conserva elementi e vuole creare una nuova raccolta vuota dello stesso tipo di elemento. La versione mostrata tenta di usare un array. Il componente deve restare indipendente dalla classe concreta degli elementi. Non occorre riempire o leggere la raccolta in questo esercizio. Le dimensioni ricevute sono non negative.

Metodo nel corpo di una classe. I numeri di riga non fanno parte del codice.

```
01  static <T> T[] empty(int size) {
02      return new T[size];
03  }
```

1. Scelta multipla

Il metodo compila? Seleziona una sola risposta.

- **A.** Sì, crea un array del tipo inferito dal chiamante.
- **B.** Sì, se `size` è zero.
- **C.** Sì, ma fallisce sempre con `ClassCastException`.
- **D.** No: non è consentita questa creazione di un array di tipo T.

2. Risposta aperta

Il requisito può cambiare: il risultato deve essere una raccolta vuota modificabile di elementi T, non necessariamente un array e senza una capacità pubblica prestabilita. Proponi una firma senza il parametro dimensione e il corpo minimo, evitando cast e reflection.

Lunghezza indicativa: massimo 3 righe di codice e una frase. Puoi usare `java.util.List` e `ArrayList`.

Mostra la soluzione dell'esercizio 10

1. Scelta multipla: D

Il parametro di tipo non fornisce il tipo runtime necessario a creare quell'array.

- **A errata:** l'inferenza non rende disponibile una classe runtime per new `T[]`.
- **B errata:** anche con lunghezza zero la dichiarazione resta vietata.
- **C errata:** il problema è di compilazione, non un fallimento inevitabile a runtime.

2. Risposta aperta

```
static <T> List<T> empty() {
    return new ArrayList<>();
}
```

La lista fornisce una raccolta modificabile con controllo statico del tipo, senza creare array generici. `List.of()` non soddisferebbe il requisito di modificabilità.

Esercizio 11 — Selezione secondo un criterio

Tempo indicativo: 5–7 minuti · Java 25

Un'utilità generica seleziona il massimo di una lista secondo un criterio fornito dal chiamante. Il contratto richiede di usare il `Comparator` ricevuto, anche quando è diverso dall'ordine naturale degli elementi. A parità di confronto può essere restituito uno qualsiasi degli elementi massimi. I test esercitano lo stesso metodo con due criteri.

Liste, elementi e comparatori sono non nulli; le liste non sono vuote e non devono essere modificate. `compare(a, b)` positivo significa che a è maggiore di b secondo il comparatore; `compareTo` usa invece l'ordine naturale. `Comparator.naturalOrder()` usa l'ordine naturale e `reverseOrder()` lo inverte. Il frammento appartiene a una classe di test; `@Test` identifica un test e `assertEquals(atteso, effettivo)` confronta i valori. Import omessi; i numeri di riga non fanno parte del codice.

```
01  static <T extends Comparable<? super T>> T max(
02          List<T> values, Comparator<? super T> order) {
03      T best = values.getFirst();
04      for (T candidate : values) {
05          if (candidate.compareTo(best) > 0) {
06              best = candidate;
07          }
08      }
09      return best;
10  }
11
12  @Test
13  void naturalCriterion() {
14      Integer result = max(List.of(2, 7, 3), Comparator.naturalOrder());
15      assertEquals(Integer.valueOf(7), result);
16  }
17
18  @Test
19  void reversedCriterion() {
20      Integer result = max(List.of(2, 7, 3), Comparator.reverseOrder());
21      assertEquals(Integer.valueOf(2), result);
22  }
```

1. Scelta multipla

Qual è l'esito dei test?

- **A.** Passano entrambi perché `max` adatta automaticamente `compareTo` al comparatore.
- **B.** Passa soltanto `reversedCriterion`.
- **C.** Il metodo non compila perché `Comparator<? super T>` non ammette comparatori di interi.
- **D.** Passa soltanto `naturalCriterion`; nell'altro test il risultato effettivo è 7.

2. Risposta aperta

Proponi, in **1–3 frasi**, un test discriminante su stringhe con un criterio diverso dall'ordine naturale: indica elementi, criterio, massimo atteso e risultato effettivo del metodo. Non scrivere codice; puoi assumere disponibile un comparatore che ordina per lunghezza crescente.

Mostra la soluzione dell’esercizio 11

1. Scelta multipla: D

Il metodo sceglie sempre secondo `compareTo` e non usa `order`. Restituisce quindi 7 in entrambi i test: corretto per l'ordine naturale, ma non per quello inverso, il cui massimo è 2.

- **A errata:** passare un comparatore non modifica il comportamento di `compareTo`; il metodo deve usarlo esplicitamente.
- **B errata:** il test naturale passa e quello inverso fallisce.
- **C errata:** il vincolo e il wildcard sono compatibili con `Integer` e con entrambi i comparatori mostrati.

2. Risposta aperta

Con la lista `["z", "aa"]` e un comparatore per lunghezza crescente, il massimo atteso è `"aa"`, ma il metodo restituisce `"z"` secondo l'ordine naturale delle stringhe. Questo test distingue il rispetto del criterio passato dall'uso esclusivo di `compareTo`; test con il solo ordine naturale non lo fanno.

Alternativa valida: altri elementi e criteri esplicitati per cui il massimo naturale differisce da quello richiesto, senza ambiguità dovute a parità.

E7 - Java Functional Programming

Esercizio 1 — Controllo di una misura

Tempo indicativo: 5–7 minuti · Java 25

Un laboratorio costruisce regole riutilizzabili per validare le misure. Una misura è ammessa se è non negativa e minore di una soglia. Ogni regola viene creata con una soglia fissa e può essere usata più volte. Le misure sono interi non nulli e le soglie sono piccoli interi. `Predicate.and` produce la congiunzione dei due predicati, senza modificare quelli originali.

Il metodo va dichiarato nel corpo di una classe, le istruzioni in un suo metodo statico; è importata `java.util.function.Predicate`. I numeri di riga non fanno parte del codice.

```
01  static Predicate<Integer> below(int limit) {
02      return value -> value < limit;
03  }
```

```
04  Predicate<Integer> nonNegative = value -> value >= 0;
05  Predicate<Integer> allowed = nonNegative.and(below(5));
06  IO.println(allowed.test(-1));
07  IO.println(allowed.test(4));
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le due righe nell'ordine.

- **A.** `true`, poi `true`.
- **B.** `false`, poi `false`.
- **C.** `true`, poi `false`.
- **D.** `false`, poi `true`.

2. Risposta breve

Un revisore propone questo corpo alternativo per `below`:

```
08  Predicate<Integer> rule = value -> value < limit;
09  limit++;
10  return rule;
```

Valuta se sia compilabile e rispetti il contratto. Il revisore sostiene che la lambda conserverebbe automaticamente il valore di `limit` precedente all'incremento: giustifica o confuta questa spiegazione.

Lunghezza indicativa: 2–3 frasi; non occorre riscrivere il metodo.

Mostra la soluzione dell'esercizio 1

1. Scelta multipla: D

`-1` non soddisfa la non negatività; 4 soddisfa entrambe le condizioni.

- **A errata:** la prima condizione rifiuta `-1`.
- **B errata:** 4 è minore di 5 e non negativo.
- **C errata:** inverte entrambi i risultati.

2. Risposta breve

La variante non compila: `limit` è un parametro locale catturato dalla lambda e non è più effettivamente finale perché viene incrementato. Non viene automaticamente creata una fotografia che consenta di ignorare la riassegnazione. La versione originale, senza incremento, compila e conserva la soglia richiesta.

Esercizio 2 — Nome da visualizzare

Tempo indicativo: 5–7 minuti · Java 25

Una schermata riceve un nome eventualmente assente. Il metodo `display` deve restituire il valore presente senza calcolare un sostituto. Soltanto in sua assenza deve invocare una volta il fornitore del nome di riserva. Il fornitore può svolgere un calcolo costoso o avere effetti osservabili. L'`Optional`, il fornitore e le stringhe prodotte sono non nulli.

Il metodo va dichiarato nel corpo di una classe e le istruzioni in un suo metodo statico, con `java.util.Optional` e `java.util.function.Supplier` importate. I numeri di riga non fanno parte del codice.

```
01  static String display(Optional<String> name, Supplier<String> fallback) {
02      return name.orElse(fallback.get());
03  }
```

```
04  Optional<String> name = Optional.of(" Ada ");
05  Optional<Integer> length = name.map(String::strip).map(String::length);
06  IO.println(length.orElse(-1));
07  IO.println("[" + name.get() + "]");
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le due righe nell'ordine.

- **A.** 5, poi `[ Ada ]`.
- **B.** 3, poi `[Ada]`.
- **C.** `-1`, poi `[ Ada ]`.
- **D.** 3, poi `[ Ada ]`.

2. Risposta breve

Valuta separatamente `display(Optional.of("Ada"), fallback)` e `display(Optional.empty(), fallback)` rispetto al contratto. Se necessario, correggi il corpo del metodo indicando quando verrà invocato il fornitore.

Lunghezza indicativa: una riga di codice e 1–2 frasi.

Mostra la soluzione dell'esercizio 2

1. Scelta multipla: D

I due `map` producono un nuovo `Optional` contenente 3; `name` conserva il testo originale con spazi.

- **A errata:** la lunghezza è calcolata dopo `strip`.
- **B errata:** le trasformazioni non cambiano il valore nell'`Optional` originale.
- **C errata:** il valore trasformato è presente, quindi non si usa `-1`.

2. Risposta breve

```
return name.orElseGet(fallback);
```

La versione originale invoca `fallback.get()` prima della chiamata a `orElse`, anche con nome presente, violando il contratto. `orElseGet` invoca il fornitore soltanto per l'assenza, una volta per questa chiamata.

Esercizio 3 — Trasformazioni di un’etichetta

Tempo indicativo: 5–7 minuti · Java 25

Un componente prepara i dati prima della visualizzazione. I client possono combinare trasformazioni elementari in un ordine scelto. La versione attuale del metodo testuale applica sempre la stessa pulizia. Un frammento numerico separato mostra l’effetto dell’ordine di composizione. I valori sono piccoli e senza overflow; testi, trasformazioni e risultati testuali sono non nulli.

Il metodo va dichiarato nel corpo di una classe e le istruzioni in un suo metodo statico, con `java.util.function.Function` importata. `f.andThen(g)` applica prima `f` e poi `g`; `f.compose(g)` applica prima `g` e poi `f`. I numeri di riga non fanno parte del codice.

```
01  static String format(String text) {
02      return text.strip();
03  }
```

```
04  Function<Integer, Integer> increment = value -> value + 1;
05  Function<Integer, Integer> twice = value -> 2 * value;
06  IO.println(increment.andThen(twice).apply(3));
07  IO.println(increment.compose(twice).apply(3));
```

1. Scelta multipla

Seleziona l’unico output corretto, indicando le due righe nell’ordine.

- **A.** 7, poi 8.
- **B.** 8, poi 7.
- **C.** 8, poi 8.
- **D.** 7, poi 7.

2. Risposta aperta

Una nuova versione di `format` deve ricevere dal chiamante la trasformazione testuale da usare, invece di fissare sempre `strip`. La trasformazione riceve il testo e restituisce il testo formattato: non deve stamparlo o modificarne contenitori esterni. Deve poter essere fornita con una lambda o un riferimento a metodo.

Scegli un’interfaccia funzionale standard, definisci la nuova firma e il corpo minimo di `format` e motiva il tipo scelto. Non introdurre nuove interfacce o classi concrete.

Lunghezza indicativa: massimo 3 righe di codice e una frase. Puoi omettere eventuali nuovi import.

Mostra la soluzione dell’esercizio 3

1. Scelta multipla: B

andThen calcola prima 3 + 1, poi raddoppia ottenendo 8; `compose` raddoppia 3 e poi incrementa ottenendo 7.

- **A errata:** scambia i due ordini.
- **C errata:** la composizione non equivale sempre alla sequenza nell'ordine di scrittura.
- **D errata:** il primo calcolo incrementa prima del raddoppio.

2. Risposta aperta

```
static String format(String text, Function<String, String> operation) {
    return operation.apply(text);
}
```

`Function` esprime un ingresso e un risultato; `Consumer` non restituisce un testo. È valido anche `UnaryOperator<String>`, specializzazione per ingresso e risultato dello stesso tipo.

Esercizio 4 — Nomi selezionati

Tempo indicativo: 5–7 minuti · Java 25

Una schermata deve consultare i nomi che iniziano con A. La selezione deve preservare l'ordine e non modificare la lista originale. Il filtro stampa un messaggio diagnostico quando viene valutato, così da osservare quando il lavoro è eseguito. La prima richiesta riguarda soltanto il primo nome selezionato. I nomi sono non nulli e lo stream è sequenziale.

Il frammento è eseguito in un metodo con `java.util.List` e `java.util.stream.Stream` importate. I numeri di riga non fanno parte del codice.

```
01  List<String> names = List.of("Ada", "Linus", "Anna");
02  Stream<String> selected = names.stream()
03      .filter(name -> {
04          IO.println("Check " + name);
05          return name.startsWith("A");
06      });
07  IO.println("Ready");
08  IO.println(selected.findFirst().orElse("None"));
```

1. Scelta multipla

Seleziona l'unico output corretto, indicando le righe nell'ordine.

- **A.** Ready, poi Check Ada, poi Ada.
- **B.** Check Ada, poi Ready, poi Ada.
- **C.** Ready, poi Check Ada, Check Linus, Check Anna, infine Ada.
- **D.** Ready, poi None.

2. Risposta breve

In un'esecuzione separata, si mantengono le righe 1–6 ma si sostituiscono le righe 7–8 con:

```
09  IO.println(selected.count());
10  IO.println(selected.toList());
```

Valuta se il programma stampi sia il numero sia la lista completa dei nomi selezionati. Se necessario, riscrivi soltanto le due nuove righe affinché il filtro venga eseguito una sola volta e siano disponibili entrambi i risultati. Le stampe diagnostiche del filtro possono rimanere.

Lunghezza indicativa: una frase e massimo 3 righe di codice.

Mostra la soluzione dell'esercizio 4

1. Scelta multipla: A

La pipeline è lazy: prima si stampa Ready; `findFirst` valuta il primo elemento, stampa Check Ada e restituisce Ada senza cercare oltre.

- **B errata:** la sola costruzione della pipeline non esegue il filtro.
- **C errata:** `findFirst` interrompe la ricerca al primo risultato.
- **D errata:** esiste un elemento che soddisfa il filtro.

2. Risposta breve

Il terminale `count` consuma lo stream; non è possibile usarlo nuovamente per ottenere la lista. Si materializza una volta il risultato:

```
List<String> result = selected.toList();
IO.println(result.size());
IO.println(result);
```

I messaggi diagnostici precedono le due stampe finali; il filtro non viene eseguito due volte.

Esercizio 5 — Autori dei gruppi

Tempo indicativo: 5–7 minuti · Java 25

Un'organizzazione raccoglie gli autori presenti in diversi gruppi di lavoro. Ogni gruppo contiene una lista di nomi. Il riepilogo deve produrre ciascun nome una sola volta, nell'ordine della prima comparsa tra i gruppi. Le liste originali devono restare disponibili e non devono essere modificate. Nomi e liste sono non nulli e lo stream è sequenziale.

Il frammento è eseguito in un metodo con `java.util.List` importata. I numeri di riga non fanno parte del codice.

```
01  List<List<String>> groups = List.of(
02      List.of("Ada", "Linus"),
03      List.of("Ada", "Grace"));
04  List<String> result = groups.stream()
05      .flatMap(List::stream)
06      .distinct()
07      .toList();
08  IO.println(result);
```

1. Scelta multipla

Qual è l'output? Seleziona una sola risposta.

- **A.** `[[Ada, Linus], [Ada, Grace]]`.
- **B.** `[Ada, Linus, Ada, Grace]`.
- **C.** `[Ada, Linus, Grace]`.
- **D.** `[Ada, Grace, Linus]`.

2. Risposta breve

Un nuovo chiamante vuole ordinare `result` sul posto e aggiungervi successivamente un nome. Valuta se il risultato attuale lo consenta. Se necessario, modifica soltanto la costruzione del risultato, mantenendo la selezione e la sua indipendenza dalle liste originali.

Lunghezza indicativa: 1–2 frasi e massimo 5 righe di codice. Puoi omettere gli import aggiuntivi.

Mostra la soluzione dell'esercizio 5

1. Scelta multipla: C

`flatMap` visita i nomi dei gruppi in ordine; `distinct` mantiene la prima occorrenza di ogni stringa nello stream ordinato.

- **A errata:** il risultato non contiene più liste annidate.
- **B errata:** il duplicato Ada viene eliminato.
- **D errata:** `Linus` precede `Grace` nell'ordine di incontro.

2. Risposta breve

`Stream.toList()` restituisce una lista non modificabile. Una costruzione conforme è:

```
List<String> result = groups.stream()
    .flatMap(List::stream)
    .distinct()
    .collect(java.util.stream.Collectors.toCollection(java.util.ArrayList::new));
```

È valida anche una nuova `ArrayList` costruita dal risultato originale. Non si deve presumere che `Collectors.toList()` garantisca una particolare implementazione o la modificabilità richiesta.

Esercizio 6 — Totali delle misure

Tempo indicativo: 5–7 minuti · Java 25

Un servizio prepara totali a partire da misure già raccolte. La prima elaborazione ignora quelle negative e raddoppia le altre. Un secondo riepilogo deve considerare invece tutti gli elementi. Entrambe le elaborazioni devono lasciare invariata la lista di partenza. Le misure sono interi non nulli tra -100 e 100 e le liste contengono al massimo cento elementi, quindi non ci sono problemi di overflow.

Il frammento è eseguito in un metodo con `java.util.List` importata. I numeri di riga non fanno parte del codice.

```
01  List<Integer> values = List.of(2, -1, 4);
02  int total = values.stream()
03      .filter(value -> value >= 0)
04      .mapToInt(value -> 2 * value)
05      .sum();
06  IO.println(total);
```

1. Scelta multipla

Qual è l'output? Seleziona una sola risposta.

- **A.** 6.
- **B.** 10.
- **C.** 5.
- **D.** 12.

2. Risposta breve

Separatamente, si vuole calcolare il negativo della somma di tutti gli elementi, anche con uno stream parallelo. Un revisore propone:

```
07  int negativeTotal = values.parallelStream()
08      .reduce(0, (left, right) -> left - right);
```

Valuta se la riduzione garantisca il risultato richiesto. Motiva considerando il raggruppamento delle sottrazioni, con un piccolo esempio, e proponi un'espressione alternativa valida anche per una lista vuota.

Lunghezza indicativa: 2 frasi, un esempio numerico e una riga di codice.

Mostra la soluzione dell'esercizio 6

1. Scelta multipla: D

Restano 2 e 4, trasformati in 4 e 8: il totale è 12.

- **A errata:** omette il raddoppio.
- **B errata:** include la misura negativa.
- **C errata:** somma l'input senza filtro né trasformazione.

2. Risposta breve

La sottrazione non è associativa e lo zero non è un'identità a sinistra: `(0 - 2) - 3 = -5`, mentre `(0 - 2) - (0 - 3) = 1`. I raggruppamenti della riduzione parallela non garantiscono il risultato richiesto; una forma corretta, anche sull'input vuoto, è:

```
int negativeTotal = -values.parallelStream().mapToInt(Integer::intValue).sum();
```

Esercizio 7 — Ordine di selezione

Tempo indicativo: 5–7 minuti · Java 25

Un servizio estrae un numero limitato di misure valide. Le misure devono essere positive e conservare il loro ordine. Il chiamante richiede al massimo due risultati. La sorgente è una lista non nulla di interi non nulli. Lo stream è sequenziale e nessuna elaborazione modifica la lista.

Frammento in un metodo con `java.util.List` importata. I numeri di riga non fanno parte del codice.

```java
List<Integer> values = List.of(-1, 2, 3, 4);
List<Integer> result = values.stream()
    .limit(2)
    .filter(value -> value > 0)
    .toList();
IO.println(result);
```

1. Scelta multipla

Qual è l'output? Seleziona una sola risposta.

- **A.** `[2, 3]`.
- **B.** `[-1, 2]`.
- **C.** `[2, 3, 4]`.
- **D.** `[2]`.

2. Risposta breve

La richiesta è «le prime due misure positive, o tutte quelle disponibili se meno di due». Valuta l'ordine attuale delle operazioni e proponi la modifica minima. Che risultato deve avere una sorgente di soli valori non positivi?

Lunghezza indicativa: 2 frasi oppure una frase e 2 righe di pipeline.

Mostra la soluzione dell'esercizio 7

1. Scelta multipla: D

`limit` seleziona prima `-1` e 2; il filtro elimina `-1` e resta solo 2.

- **A errata:** richiederebbe il filtro prima del limite.
- **B errata:** il filtro rifiuta `-1`.
- **C errata:** il limite impedisce di raggiungere tutti gli elementi.

2. Risposta breve

Scambio le operazioni: prima `filter(value -> value > 0)`, poi `limit(2)`. Così il limite conta i risultati validi, non gli elementi iniziali; in assenza di positivi si ottiene una lista vuota.

Esercizio 8 — Quantità per categoria

Tempo indicativo: 5–7 minuti · Java 25

Un negozio riceve una sequenza di categorie per i prodotti venduti. Il riepilogo deve contare quante vendite appartengono a ciascuna categoria. Due stringhe uguali indicano la stessa categoria. Non interessa l'ordine delle chiavi della mappa risultante. La sorgente contiene stringhe non nulle e non deve essere modificata.

Frammento in un metodo con `java.util.*`, `java.util.function.Function` e `java.util.stream.Collect` importate. `groupingBy` raggruppa per la chiave calcolata; `counting` conta gli elementi di ogni gruppo con risultato Long. I numeri di riga non fanno parte del codice.

```
01  List<String> categories = List.of("book", "pen", "book");
02  Map<String, Long> counts = categories.stream()
03      .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
04  IO.println(counts.get("book"));
05  IO.println(counts.size());
```

1. Scelta multipla

Seleziona l'output corretto.

- **A.** 1, poi 3.
- **B.** 2, poi 2.
- **C.** 2, poi 3.
- **D.** Eccezione per la chiave duplicata book.

2. Risposta aperta

Il chiamante vuole una funzione riutilizzabile `countBy` che riceva una lista di stringhe e una regola per calcolare la categoria di ciascuna stringa. Proponi la firma con un'interfaccia funzionale standard e descrivi come riusare il collector; regola e risultati sono non nulli.

Lunghezza indicativa: una firma e 1–2 frasi.

Mostra la soluzione dell'esercizio 8

1. Scelta multipla: B

Due elementi appartengono a book; le categorie distinte sono due.

- **A errata:** raggruppare non elimina il contributo delle ripetizioni al conteggio.
- **C errata:** la dimensione della mappa conta le chiavi, non gli input.
- **D errata:** il collector gestisce proprio gruppi con la stessa chiave.

2. Risposta aperta

`static Map<String, Long> countBy(List<String> values, Function<String, String> category)` usa `groupingBy(category, counting())` al posto della funzione identità. La funzione riceve l'elemento e restituisce la sua chiave; sono valide wildcard compatibili nella firma.

Esercizio 9 — Scelta di un risultato

Tempo indicativo: 5–7 minuti · Java 25

Un componente cerca il primo nome sufficientemente lungo. L'assenza di risultati è prevista e non deve essere trattata come un guasto. Il chiamante vuole sempre una stringa da visualizzare. La lista è non nulla e contiene stringhe non nulle. Lo stream è sequenziale e segue l'ordine della lista.

Frammento in un metodo con `java.util.*` importata. I numeri di riga non fanno parte del codice.

```
01  List<String> names = List.of("Ada", "Bo");
02  Optional<String> found = names.stream()
03      .filter(name -> name.length() > 5)
04      .findFirst();
05  IO.println(found.isPresent());
06  IO.println(found.orElse("None"));
```

1. Scelta multipla

Seleziona l'output corretto.

- **A.** `false`, poi `None`.
- **B.** `true`, poi `Ada`.
- **C.** `false`, poi `null`.
- **D.** Solleva `NoSuchElementException`.

2. Risposta aperta

Una variante deve visualizzare la lunghezza del nome trovato oppure zero se assente, senza `get()` né un controllo esplicito con `if`. Proponi l'espressione e spiega perché gestisce entrambe le situazioni.

Lunghezza indicativa: una riga di codice e 1–2 frasi.

Mostra la soluzione dell'esercizio 9

1. Scelta multipla: A

Nessun nome supera cinque caratteri: l'`Optional` è vuoto e si usa il testo di riserva.

- **B errata:** Ada non supera la soglia.
- **C errata:** il ripiego esplicito è `None`, non `null`.
- **D errata:** `orElse` gestisce l'assenza senza leggere con `get()`.

2. Risposta aperta

```
int length = found.map(String::length).orElse(0);
```

`map` trasforma solo il valore presente; `orElse` fornisce zero quando il risultato è assente. Non occorre una lettura non protetta.

Esercizio 10 — Riepilogo dei codici

Tempo indicativo: 5–7 minuti · Java 25

Un esportatore deve produrre una stringa con i codici ricevuti. I codici vanno separati da una virgola, senza separatore iniziale o finale. La lista può essere vuota e in quel caso il risultato deve essere una stringa vuota. I codici sono stringhe non nulle e non contengono virgole. Il codice deve funzionare anche se la pipeline viene resa parallela.

Frammento in un metodo con `java.util.List` e `java.util.stream.Collectors` importate. I numeri di riga non fanno parte del codice.

```
01  List<String> codes = List.of("A", "B", "C");
02  String text = codes.stream()
03      .map(code -> code.toLowerCase(java.util.Locale.ROOT))
04      .collect(Collectors.joining(","));
05  IO.println(text);
```

1. Scelta multipla

Qual è l'output? Seleziona una sola risposta.

- **A.** `,a,b,c`.
- **B.** `a,b,c,`.
- **C.** `a,b,c`.
- **D.** `[a, b, c]`.

2. Risposta breve

Un revisore usa un unico `StringBuilder` esterno, condiviso dalle lambda di un `parallelStream().forEach(...)`, per aggiungere codici e virgole. Spiega due ragioni per cui tale versione non garantisce il contratto, senza proporre sincronizzazione. Quale parte della versione mostrata evita la necessità di uno stato esterno condiviso?

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 10

1. Scelta multipla: C

`joining` inserisce la virgola soltanto tra gli elementi; i codici sono trasformati in minuscolo.

- **A errata:** non viene aggiunto un delimitatore iniziale.
- **B errata:** non viene aggiunto un delimitatore finale.
- **D errata:** il risultato è una stringa unita, non la rappresentazione di una lista.

2. Risposta breve

Il builder condiviso non è sicuro per modifiche concorrenti e `forEach` parallelo non garantisce l'ordine della sorgente. Inoltre una semplice aggiunta della virgola dopo ogni codice lascerebbe un separatore finale. Il collector `joining` gestisce accumulazione, combinazione e delimitatori senza esporre un unico builder esterno alle lambda; preserva l'ordine di incontro dello stream ordinato.

Esercizio 11 — Aggiornamento dei conti scaduti

Tempo indicativo: 5–7 minuti · Java 25

Un servizio seleziona i conti con scadenza strettamente precedente alla data di riferimento. Deve aumentare del 10% il saldo dei soli conti selezionati, modificando gli oggetti originali, e restituirli in ordine di saldo aggiornato decrescente. I conti non scaduti devono rimanere invariati. Ogni test prepara nuovi oggetti e invoca il servizio una sola volta.

Liste, conti e date sono non nulli; i saldi sono centesimi interi positivi multipli di 100, senza overflow. Lo stream è sequenziale. peek esegue l'azione sugli elementi che attraversano quello stadio; `filter` lascia passare quelli che soddisfano il predicato, `sorted` li ordina e `toList` raccoglie i riferimenti senza copiare gli oggetti. Il frammento appartiene a una classe di test, con `Account` come classe annidata statica. `@Test` identifica un test e `assertEquals(atteso, effettivo)` confronta valori o contenuto e ordine delle liste. Import omessi; i numeri di riga non fanno parte del codice.

```java
static class Account {
    int balance;
    final LocalDate due;
    Account(int balance, LocalDate due) {
        this.balance = balance;
        this.due = due;
    }
}
static List<Account> applyInterest(List<Account> accounts, LocalDate today) {
    return accounts.stream()
            .peek(a -> a.balance += a.balance / 10)
            .filter(a -> a.due.isBefore(today))
            .sorted(Comparator.comparingInt((Account a) -> a.balance).reversed())
            .toList();
}
@Test
void selectedAccounts() {
    LocalDate today = LocalDate.of(2026, 5, 10);
    Account old = new Account(100, today.minusDays(1));
    Account current = new Account(200, today.plusDays(1));
    List<Account> selected = applyInterest(List.of(old, current), today);
    assertEquals(List.of(old), selected);
    assertEquals(110, old.balance);
}
@Test
void currentBalance() {
    LocalDate today = LocalDate.of(2026, 5, 10);
    Account current = new Account(200, today.plusDays(1));
    applyInterest(List.of(current), today);
    assertEquals(200, current.balance);
}
```

1. Scelta multipla

Qual è l'esito dei test?

- **A.** Passano entrambi: `filter` impedisce ogni modifica ai conti esclusi.
- **B.** Falliscono entrambi perché `toList` crea copie indipendenti dei conti.
- **C.** Passa soltanto `selectedAccounts`; in `currentBalance` il saldo effettivo è `220`.
- **D.** Passa soltanto `currentBalance`; peek non viene eseguito quando si usa `toList`.

2. Risposta aperta

Il test `selectedAccounts` verifica anche il requisito sull'ordinamento? In **1–3 frasi**, proponi dati aggiuntivi e l'ordine atteso che permettano di verificarlo, senza scrivere codice.

Mostra la soluzione dell’esercizio 11

1. Scelta multipla: C

`toList` consuma la pipeline e il peek precedente al filtro aumenta il saldo anche dei conti che saranno esclusi. Il primo test osserva soltanto la selezione e il saldo del conto scaduto, entrambi corretti; nel secondo il conto non scaduto passa da `200` a `220`, violando il contratto.

- **A errata:** il filtro non annulla gli effetti già avvenuti negli stadi precedenti.
- **B errata:** `toList` raccoglie gli stessi riferimenti, non copie indipendenti dei conti.
- **D errata:** `toList` è un'operazione terminale e, in questa pipeline, gli elementi attraversano peek prima della selezione.

2. Risposta aperta

No: una lista con un solo conto selezionato non distingue ordinamenti diversi. Si possono inserire nell'ordine due conti entrambi scaduti, con saldi `100` e `300`: dopo l'aggiornamento devono comparire prima quello con `330`, poi quello con `110`; l'implementazione mostrata soddisfa questa verifica, pur non rispettando il requisito sui conti esclusi.

Alternativa valida: altri saldi distinti di conti scaduti, inizialmente non già in ordine decrescente, con saldi aggiornati e ordine atteso corretti.

E8 - Java Concurrency

Esercizio 1 — Conteggio degli eventi

Tempo indicativo: 5–7 minuti · Java 25

Un servizio conta gli eventi prodotti da due sorgenti. Ogni sorgente esegue un solo incremento sullo stesso contatore, inizialmente pari a zero. Ogni chiamata a `increment` deve aumentarlo esattamente di uno. Il chiamante vuole leggere il totale dopo la fine delle due elaborazioni. Entrambi i thread terminano normalmente e il thread che esegue il frammento non viene interrotto.

Le istruzioni dopo la classe sono eseguite in un metodo che dichiara `throws InterruptedException`. I numeri di riga non fanno parte del codice.

```
01  class Counter {
02      private volatile int value;
03      void increment() { value++; }
04      int get() { return value; }
05  }
```

```
06  Counter counter = new Counter();
07  Thread first = new Thread(counter::increment);
08  Thread second = new Thread(counter::increment);
09  first.start();
10  second.start();
11  first.join();
12  second.join();
13  IO.println(counter.get());
```

1. Scelta multipla

Quali valori può stampare il programma secondo le garanzie di Java? Seleziona una sola risposta.

- **A.** Soltanto 2, perché il campo è `volatile`.
- **B.** `0`, 1 o 2, perché la stampa potrebbe precedere gli incrementi.
- **C.** 1 oppure 2, in base all'interleaving degli incrementi.
- **D.** Soltanto 1, perché i thread condividono lo stesso oggetto.

2. Risposta breve

Considera una versione distinta in cui `increment` è dichiarato `synchronized`, il campo resta `volatile` e i due `join` vengono eliminati. La stampa finale è garantita pari a 2? Motiva distinguendo la protezione dell'incremento dall'attesa del completamento dei thread; non modificare questa versione.

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 1

1. Scelta multipla: C

`volatile` dà visibilità ma non rende atomico il ciclo lettura-incremento-scrittura. I thread possono leggere entrambi zero e scrivere entrambi uno; altrimenti il risultato è due. I `join` impediscono la lettura prima della loro terminazione.

- **A errata:** un incremento composto non diventa atomico grazie a `volatile`.
- **B errata:** dopo entrambi i `join` la stampa non precede le scritture e zero non è un esito possibile.
- **D errata:** l'interleaving può anche serializzare gli incrementi, producendo due.

2. Risposta breve

No: `synchronized` evita la perdita di incrementi, ma non attende l'esecuzione dei thread. Senza `join` la stampa può leggere `0`, `1` o `2` dal campo ancora `volatile`. Mutua esclusione e attesa del completamento sono requisiti distinti.

Esercizio 2 — Apertura di un varco

Tempo indicativo: 5–7 minuti · Java 25

Un worker attende il permesso prima di attraversare un varco. Il varco parte chiuso e, una volta aperto, rimane aperto. `await` deve ritornare soltanto quando il permesso è disponibile. Il thread principale apre il varco e attende la conclusione del worker. Il worker non viene interrotto, termina normalmente e tutti i thread vengono prima o poi eseguiti.

Le istruzioni dopo la classe sono eseguite in un metodo che dichiara `throws InterruptedException`. I numeri di riga non fanno parte del codice.

```java
class Gate {
    private boolean open;
    synchronized void await() throws InterruptedException {
        while (!open) wait();
    }
    synchronized void open() {
        open = true;
        notifyAll();
    }
}
```

```java
Gate gate = new Gate();
Thread worker = new Thread(() -> {
    try { gate.await(); }
    catch (InterruptedException exception) { throw new AssertionError(exception); }
    IO.println("Passed");
});
worker.start();
gate.open();
worker.join();
IO.println("End");
```

1. Scelta multipla

Quale affermazione sull'output è corretta? Seleziona una sola risposta.

- **A.** Stampa sempre Passed, poi End.
- **B.** Stampa sempre End, poi Passed.
- **C.** Può stampare le due righe in qualsiasi ordine.
- **D.** Può restare in attesa per sempre soltanto perché `open()` viene eseguito prima di `await()`.

2. Risposta breve

In una versione distinta si sostituisce il `while` con `if (!open) wait();`. Considera che Java ammette risvegli di `wait` anche senza una notifica. La versione modificata garantisce ancora il contratto di `await`? Descrivi una sequenza di eventi che sostenga la tua valutazione, senza usare pause temporizzate.

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 2

1. Scelta multipla: A

Il worker stampa Passed dopo aver osservato il varco aperto; il thread principale stampa End solo dopo il suo `join`.

- **B errata:** `join` attende anche la stampa del worker.
- **C errata:** esiste un vincolo di ordine esplicito.
- **D errata:** se il varco è già aperto, `await` non entra in `wait`.

2. Risposta breve

La versione con `if` non garantisce il contratto. Il worker può osservare `open == false`, entrare in attesa e risvegliarsi senza notifica mentre il varco è ancora chiuso; con `if` proseguirebbe senza ricontrollare. Il `while` verifica nuovamente la condizione dopo ogni risveglio, sotto lo stesso monitor.

Esercizio 3 — Risultato di un'elaborazione

Tempo indicativo: 5–7 minuti · Java 25

Un servizio affida un calcolo a un executor con un solo worker. Il chiamante conserva un `Future` per recuperarne il risultato. Dopo l'invio non sono necessari altri task e viene richiesto lo shutdown. La chiusura delle risorse deve evitare di lasciare worker attivi. L'invio riesce, il calcolo termina normalmente e il chiamante non viene interrotto.

Il frammento è eseguito in un metodo che dichiara `throws Exception`, con `java.util.concurrent.Executo` `Executors` e `Future` importate. I numeri di riga non fanno parte del codice.

```
01  try (ExecutorService executor = Executors.newSingleThreadExecutor()) {
02      Future<Integer> result = executor.submit(() -> 6 * 7);
03      executor.shutdown();
04      IO.println(result.get());
05      IO.println(executor.isShutdown());
06  }
```

1. Scelta multipla

Che cosa accade? Seleziona una sola risposta.

- **A.** Stampa 42, poi `false`.
- **B.** Solleva `RejectedExecutionException` perché `shutdown` annulla il task già inviato.
- **C.** Stampa 42, poi `true`.
- **D.** Non compila perché `ExecutorService` non può essere usato in try-with-resources in Java 25.

2. Risposta breve

In un'esecuzione separata, con un executor ancora attivo, viene avviato questo task. `performStep()` è un metodo disponibile che termina ogni volta, non blocca e non modifica lo stato di interruzione.

```
07  Future<?> job = executor.submit(() -> {
08      while (true) {
09          performStep();
10      }
11  });
12  job.cancel(true);
```

Supponi che il task sia già iniziato e che `cancel(true)` restituisca `true`. È garantito che il codice del task si sia fermato quando `cancel` ritorna? Motiva e indica una modifica minima al ciclo per rendere la terminazione cooperativa rispetto all'interruzione.

Lunghezza indicativa: una riga di codice e 2 frasi.

Mostra la soluzione dell'esercizio 3

1. Scelta multipla: C

`shutdown` non annulla i task già accettati; `get` attende il risultato 42. L'executor ha ricevuto la richiesta di shutdown, quindi `isShutdown()` è `true`.

- **A errata:** lo shutdown è già stato richiesto.
- **B errata:** un task già inviato non viene rifiutato retroattivamente.
- **D errata:** in Java 25 `ExecutorService` è `AutoCloseable`.

2. Risposta breve

Il successo di `cancel(true)` non garantisce che il codice del task si sia già fermato: l'interruzione è cooperativa. Si può sostituire la condizione del ciclo con:

```
while (!Thread.currentThread().isInterrupted()) {
```

Dato che `performStep()` termina e non cancella l'interruzione, il task potrà osservare il segnale e uscire a un controllo successivo; la cancellazione non è un'attesa della sua terminazione.

Esercizio 4 — Avvio di un’attività

Tempo indicativo: 5–7 minuti · Java 25

Un’applicazione prepara un’attività che stampa un messaggio. Il chiamante crea un oggetto `Thread` per rappresentarla. Nella versione mostrata invoca direttamente `run`. Si vuole capire se l’operazione introduca un’esecuzione concorrente. Non ci sono altri thread applicativi che stampano o modificano lo stato osservato.

Frammento in un metodo. I numeri di riga non fanno parte del codice.

```
01  Thread caller = Thread.currentThread();
02  Thread worker = new Thread(() -> {
03      IO.println(Thread.currentThread() == caller);
04  });
05  worker.run();
06  IO.println("End");
```

1. Scelta multipla

Seleziona l’output corretto.

- **A.** `false`, poi End.
- **B.** End, poi `true`.
- **C.** `true`, poi End.
- **D.** Le due righe possono uscire in qualsiasi ordine.

2. Risposta aperta

Proponi la sequenza minima che esegua l’attività in un nuovo thread e garantisca che End venga stampato dopo la sua conclusione. Non usare attese temporizzate e assumi che il chiamante possa dichiarare `throws InterruptedException`.

Lunghezza indicativa: massimo 3 righe di codice e una frase.

Mostra la soluzione dell’esercizio 4

1. Scelta multipla: C

`run()` è una chiamata ordinaria eseguita dal thread chiamante; la verifica d’identità restituisce `true`.

- **A errata:** non viene avviato un nuovo thread.
- **B errata:** la chiamata termina prima della stampa successiva.
- **D errata:** qui le operazioni avvengono sequenzialmente nello stesso thread.

2. Risposta aperta

```
worker.start();
worker.join();
IO.println("End");
```

`start` avvia il nuovo thread e `join` ne attende la conclusione; la verifica nel worker diventa `false`. Non si deve chiamare anche `run` per ripetere il lavoro.

Esercizio 5 — Pubblicazione di un risultato

Tempo indicativo: 5–7 minuti · Java 25

Un worker calcola un valore e lo salva in un oggetto condiviso. Il chiamante legge il risultato solo dopo aver atteso il worker. Il campo non usa `volatile` e non ci sono altre scritture. Il worker termina normalmente. Il chiamante non viene interrotto e può dichiarare `throws InterruptedException`.

Classe e istruzioni in un metodo statico separato. I numeri di riga non fanno parte del codice.

```
01  class Result { int value; }
02  // In un altro metodo:
03  Result result = new Result();
04  Thread worker = new Thread(() -> result.value = 42);
05  worker.start();
06  worker.join();
07  IO.println(result.value);
```

1. Scelta multipla

Quale risultato è garantito? Seleziona una sola risposta.

- **A.** `0` oppure 42, perché il campo non è `volatile`.
- **B.** 42, grazie alla terminazione osservata tramite `join`.
- **C.** `0`, perché il campo appartiene al chiamante.
- **D.** Errore di compilazione per accesso da un altro thread.

2. Risposta breve

Un revisore sostituisce `join` con `Thread.sleep(100)`. Tale versione garantisce lo stesso risultato su ogni macchina? Distingui il tempo trascorso dall'osservazione del completamento.

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 5

1. Scelta multipla: B

Il completamento atteso con `join` stabilisce la relazione happens-before necessaria a rendere visibile la scrittura precedente del worker.

- **A errata:** `volatile` non è l'unico meccanismo di visibilità.
- **C errata:** l'oggetto è condiviso e il worker può modificarlo.
- **D errata:** Java permette accessi condivisi; la sincronizzazione è una responsabilità del programma.

2. Risposta breve

No: una pausa di cento millisecondi non garantisce che il worker abbia finito, né sostituisce la relazione stabilita da `join`. Lo scheduling può ritardarlo oltre la pausa. Serve attendere il completamento con un meccanismo appropriato, non stimarne la durata.

Esercizio 6 — Aggiornamento di una coppia

Tempo indicativo: 5–7 minuti · Java 25

Un componente mantiene due contatori che devono avanzare insieme. Tutti gli aggiornamenti acquisiscono il monitor dell'oggetto. Dopo la fine degli aggiornamenti i due valori devono coincidere. Il chiamante crea due worker, ciascuno con una sola chiamata. Entrambi terminano normalmente, il chiamante non viene interrotto e non si considera overflow.

Classe e istruzioni in un metodo statico che dichiara `throws InterruptedException`. I numeri di riga non fanno parte del codice.

```
01  class Pair {
02      private int first, second;
03      synchronized void advance() { first++; second++; }
04      synchronized boolean equal() { return first == second; }
05  }
06  // In un altro metodo:
07  Pair pair = new Pair();
08  Thread a = new Thread(pair::advance);
09  Thread b = new Thread(pair::advance);
10  a.start();
11  b.start();
12  a.join();
13  b.join();
14  IO.println(pair.equal());
```

1. Scelta multipla

Quale risultato è garantito? Seleziona una sola risposta.

- **A.** `false`, perché due thread eseguono incrementi.
- **B.** `true` solo se entrambi i campi sono `volatile`.
- **C.** Qualsiasi booleano, perché i thread possono alternarsi.
- **D.** `true`, perché ogni aggiornamento protegge la coppia con lo stesso monitor.

2. Risposta aperta

Una nuova API deve restituire insieme i due valori mentre gli aggiornamenti possono continuare. Proponi firma e strategia di un metodo che produca una fotografia coerente, senza esporre stato interno modificabile. Non servono altri tipi oltre a quelli standard o a un piccolo record.

Lunghezza indicativa: massimo 4 righe di codice e una frase.

Mostra la soluzione dell'esercizio 6

1. Scelta multipla: D

Le chiamate si escludono a vicenda sullo stesso oggetto; dopo i `join` entrambi gli incrementi di ciascun campo sono terminati.

- **A errata:** la concorrenza è protetta, non implica corruzione automatica.
- **B errata:** il monitor fornisce già la sincronizzazione necessaria.
- **C errata:** nessun aggiornamento parziale può essere osservato sotto quel monitor.

2. Risposta aperta

```
record Snapshot(int first, int second) { }
synchronized Snapshot snapshot() {
    return new Snapshot(first, second);
}
```

La lettura dei due campi usa lo stesso monitor degli aggiornamenti e restituisce valori primitivi in un oggetto immutabile. È valido anche un nuovo array di due interi, costruito sotto quel monitor e non conservato come stato interno.

Esercizio 7 — Due archivi condivisi

Tempo indicativo: 5–7 minuti · Java 25

Due worker devono accedere a due archivi protetti da monitor distinti. Ogni worker deve acquisire entrambi prima di svolgere il lavoro. Il programma può eseguire i worker in qualunque ordine. Non ci sono interruzioni o operazioni esterne che rilasciano forzatamente i monitor. Gli oggetti `left` e `right` sono distinti e condivisi da entrambi i worker.

Frammento in un metodo. I numeri di riga non fanno parte del codice.

```
01  Object left = new Object();
02  Object right = new Object();
03  Thread a = new Thread(() -> {
04      synchronized (left) {
05          synchronized (right) { IO.println("A"); }
06      }
07  });
08  Thread b = new Thread(() -> {
09      synchronized (right) {
10          synchronized (left) { IO.println("B"); }
11      }
12  });
13  a.start();
14  b.start();
```

1. Scelta multipla

Quale affermazione è corretta? Seleziona una sola risposta.

- **A.** È possibile un deadlock se ciascuno acquisisce prima il proprio monitor esterno.
- **B.** Il deadlock è inevitabile a ogni esecuzione.
- **C.** `synchronized` impedisce qualsiasi deadlock.
- **D.** Il programma non compila perché i monitor sono locali.

2. Risposta breve

Indica una modifica alla politica di acquisizione che elimini questo ciclo d'attesa senza rimuovere la protezione dei due archivi. Spiega perché aggiungere `sleep` non darebbe la stessa garanzia.

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 7

1. Scelta multipla: A

Un worker può trattenere `left` aspettando `right`, mentre l'altro trattiene `right` aspettando `left`.

- **B errata:** uno dei due può acquisire entrambi e completare prima dell'altro.
- **C errata:** la mutua esclusione non impone un ordine globale dei monitor.
- **D errata:** i riferimenti locali sono effettivamente finali e possono essere catturati.

2. Risposta breve

Entrambi devono acquisire prima `left` e poi `right`, secondo un unico ordine condiviso. In questo modo non può formarsi il ciclo d'attesa tra questi due monitor. Una pausa può solo cambiare la probabilità di un interleaving, non eliminarlo dalle esecuzioni possibili.

Esercizio 8 — Fallimento di un task

Tempo indicativo: 5–7 minuti · Java 25

Un servizio invia un task che può fallire. Il chiamante vuole osservare il fallimento recuperando il risultato del task. L'invio viene accettato e il worker esegue il codice mostrato. Il chiamante non viene interrotto. `ExecutionException` è il contenitore con cui `Future.get()` riporta il fallimento del task; `getCause()` restituisce l'eccezione originaria.

Frammento in un metodo che dichiara `throws InterruptedException`, con `java.util.concurrent.*` importata. I numeri di riga non fanno parte del codice.

```
01  try (ExecutorService executor = Executors.newSingleThreadExecutor()) {
02      Future<Integer> result = executor.submit(() -> {
03          throw new IllegalArgumentException("Bad input");
04      });
05      try {
06          IO.println(result.get());
07      } catch (ExecutionException exception) {
08          IO.println(exception.getCause().getMessage());
09      }
10  }
```

1. Scelta multipla

Che cosa accade? Seleziona una sola risposta.

- **A.** Stampa `null`.
- **B.** Il `catch` non vede errori avvenuti in un altro thread.
- **C.** Stampa `Bad input`.
- **D.** Non compila perché una lambda che lancia non può essere un `Callable<Integer>`.

2. Risposta breve

Se il chiamante ignora il `Future` e non ne recupera mai l'esito, il solo ritorno di `submit` dimostra il successo del calcolo? Indica dove deve essere gestito l'errore per poterlo riportare al client, senza introdurre stampe nel task.

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 8

1. Scelta multipla: C

`get` solleva `ExecutionException`; il gestore stampa il messaggio della causa `IllegalArgumentException`.

- **A errata:** un task fallito non produce un risultato nullo di successo.
- **B errata:** il `Future` trasporta esplicitamente l'esito tra i thread.
- **D errata:** un corpo che termina sempre con `throw` è compatibile con quel risultato.

2. Risposta breve

Il ritorno di `submit` dimostra solo che l'invio è stato accettato, non che il calcolo sia riuscito. Un livello che possiede il `Future` deve recuperarne l'esito e gestire o tradurre `ExecutionException`, conservando la causa. Ignorare il `Future` può lasciare il fallimento non osservato dal client.

Esercizio 9 — Passaggio di un messaggio

Tempo indicativo: 5–7 minuti · Java 25

Un produttore consegna un messaggio a un consumatore mediante una coda bloccante di capacità uno. `put` attende spazio e `take` attende un elemento, rimuovendolo. I due task devono poter procedere senza polling. Nel frammento un solo worker deve eseguirli entrambi e non ci sono produttori esterni. Nessun thread viene interrotto e i task accettati vengono prima o poi eseguiti se il worker è libero.

Frammento in un metodo con `java.util.concurrent.*` importata. I numeri di riga non fanno parte del codice.

```
01  BlockingQueue<String> queue = new ArrayBlockingQueue<>(1);
02  try (ExecutorService executor = Executors.newSingleThreadExecutor()) {
03      executor.submit(() -> {
04          IO.println(queue.take());
05          return null;
06      });
07      executor.submit(() -> {
08          queue.put("Ready");
```

```
09          return null;
10      });
11  }
```

1. Scelta multipla

Quale comportamento descrive questo programma? Seleziona una sola risposta.

- **A.** Stampa sempre Ready e termina.
- **B.** Il consumatore attende nel primo task e impedisce al produttore di partire; la chiusura attende senza terminare.
- **C.** `take` restituisce `null` perché la coda è vuota.
- **D.** Non compila perché una coda bloccante richiede thread virtuali.

2. Risposta aperta

Per questo scambio finito di un solo messaggio, scegli un'organizzazione dell'executor che permetta a entrambi i task di procedere e indica come osservare l'eventuale fallimento dei due task. Non usare `sleep`, polling o un numero illimitato di worker.

Lunghezza indicativa: una dichiarazione e 2 frasi.

Mostra la soluzione dell'esercizio 9

1. Scelta multipla: B

Il primo task occupa l'unico worker aspettando un elemento che soltanto il secondo task potrebbe produrre. La chiusura ordinata attende task che non riescono a completare.

- **A errata:** il produttore non trova un worker libero.
- **C errata:** `take` attende, non segnala la coda vuota con `null`.
- **D errata:** le code bloccanti funzionano anche con thread di piattaforma.

2. Risposta aperta

```
ExecutorService executor = Executors.newFixedThreadPool(2);
```

Con due worker, produttore e consumatore possono avanzare contemporaneamente; la capacità uno basta per il messaggio. Conservo entrambi i `Future` e ne recupero l'esito con `get`, gestendo interruzione e fallimenti; per protocolli più generali servirebbe anche definire come cancellare il task che resta in attesa se l'altro fallisce.

Esercizio 10 — Attesa con scadenza

Tempo indicativo: 5–7 minuti · Java 25

Un client non vuole attendere indefinitamente il risultato di un task. Usa quindi un `get` con timeout su un `Future` già ottenuto da un executor attivo. Per questa esecuzione si assume esplicitamente che il timeout scada mentre il task è ancora in corso. Il task non viene cancellato da altri componenti e il chiamante non viene interrotto. `get(timeout, unit)` segnala lo scadere dell'attesa con `TimeoutException`, senza cancellare il calcolo.

Frammento in un metodo che dichiara `throws InterruptedException, ExecutionException`, con `java.util.concurrent.*` importata; `result` è un `Future<Integer>` già disponibile. I numeri di riga non fanno parte del codice.

```
01  try {
02      IO.println(result.get(1, TimeUnit.MILLISECONDS));
03  } catch (TimeoutException exception) {
04      IO.println("Timeout");
05  }
06  IO.println(result.isCancelled());
```

1. Scelta multipla

Sotto le assunzioni date, seleziona l'output corretto.

- **A.** `Timeout`, poi `false`.
- **B.** `Timeout`, poi `true`.
- **C.** Soltanto `Timeout`, perché il client termina automaticamente.
- **D.** Il risultato numerico, poi `false`.

2. Risposta breve

Il client propone `result.cancel(true)` nel `catch` e sostiene che questo annulli anche tutti gli effetti già prodotti dal task. Valuta la proposta distinguendo richiesta di cancellazione, cooperazione del task e operazioni già completate.

Lunghezza indicativa: 2–3 frasi.

Mostra la soluzione dell'esercizio 10

1. Scelta multipla: A

Il timeout termina questa attesa, non il task. In assenza di cancellazioni, `isCancelled()` resta `false`, anche se nel frattempo il task completa.

- **B errata:** il timeout non cancella automaticamente.
- **C errata:** il `catch` termina normalmente e si esegue la stampa successiva.
- **D errata:** l'ipotesi della consegna esclude il ritorno di un valore da questa chiamata a `get`.

2. Risposta breve

`cancel(true)` tenta di cancellare e, se opportuno, interrompere il task; il codice deve cooperare per fermarsi. Non riporta indietro effetti già completati, come una scrittura esterna. Se il task nel frattempo è terminato, la cancellazione può anche essere rifiutata: timeout e rollback sono contratti diversi.